

Doctoral Dissertation

**Practical and Secure On/Offline
Cryptographic Protocols for Multi-User and
Multi-Device Mobile Applications**



Young Ah Shin

Department of Information Security

School of Cybersecurity

Korea University

August 2026

The dissertation of Young Ah Shin has been
approved by the dissertation committee in partial
fulfillment of the requirements for the degree of
Doctor of Engineering

June 2026



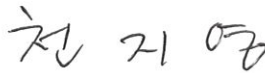
Committee Chair: Ik Rae Jeong



Committee Member: Changmin Lee



Committee Member: Seongkwang Kim



Committee Member: Ji Young Chun



Committee Member: Geontae Noh

Practical and Secure On/Offline Cryptographic Protocols for Multi-User and Multi-Device Mobile Applications

Young Ah Shin

Department of Information Security

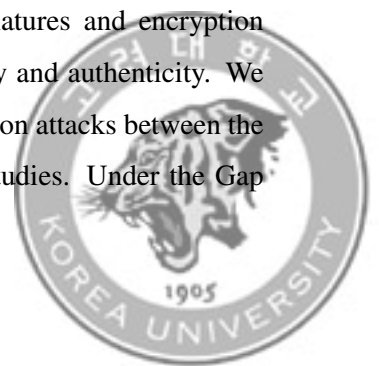
under the supervision of Professor Ik Rae Jeong

Abstract

Identity-based cryptography (IBC) provides a public-key cryptographic framework suitable for mobile environments by allowing devices' unique identifiers to serve as public keys, thereby reducing the burden of certificate management. However, many existing IBC schemes rely on bilinear pairings, limiting their practical applicability in resource-constrained environments. This dissertation addresses the design of practical and provably secure identity-based on/offline cryptographic protocols for such environments. Specifically, it focuses on a multi-proxy signature scheme for multi-user settings and on/offline signcryption schemes for multi-device settings.

An identity-based multi-proxy signature (IBMPS) scheme enables multiple proxy signers with delegated signing authority to collaboratively generate a proxy signature on behalf of an original signer when the original signer is absent. Unlike existing schemes, our scheme maintains independence between each proxy signing key and the corresponding long-term private key, ensuring that the exposure of a proxy signing key does not compromise the long-term private key. We prove that the scheme achieves existential unforgeability under the Computational Diffie–Hellman (CDH) assumption. We further extend the scheme into an on/offline protocol in which each proxy signer can generate its partial signature during the online phase using only a single elliptic curve point addition. Our implementation on a Raspberry Pi–based drone platform demonstrates the computational efficiency and practicality of the proposed construction.

Identity-based on/offline signcryption (OOSE) combines digital signatures and encryption within a single cryptographic primitive to provide message confidentiality and authenticity. We present two schemes, AOOSE and COOSE, that consider memory corruption attacks between the offline and online phases, which have not been considered in previous studies. Under the Gap



Bilinear Diffie–Hellman (GBDH) assumption, the proposed schemes are proven to achieve indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA2) for confidentiality and existential unforgeability under adaptive chosen-message attacks (EUF-CMA) for authentication. Our OMNeT++-based simulations evaluate their computational and communication performance in drone-based mobile network environments and show the practicality of the proposed schemes.

Overall, this dissertation demonstrates that identity-based on/offline cryptographic protocols can simultaneously achieve strong security against key exposure and cross-phase attacks while remaining practical for resource-constrained mobile applications.

Keywords: ID-Based Cryptography, On/Offline Protocol, Multi-Proxy Signature, On/Offline Signcryption, Mobile Application



다중 사용자 및 다중 기기 구조의 모바일 환경에서 실용적이고 안전한 온오프라인 암호 프로토콜

신 영 아
정 보 보 호 학 과
지도교수: 정 익 래

국문 초록

ID 기반 암호는 기기의 고유 식별자를 공개키로 활용함으로써 인증서 관리 부담을 줄이고, 모바일 환경에 적합한 공개키 암호 체계를 구성할 수 있다. 그러나 많은 수의 ID 기반 암호 기법은 연산 비용이 큰 겹선형 사상에 의존하므로 자원 제약 환경에 적용하는 데 한계가 있다. 본 논문은 자원 제약 모바일 환경에서 실용성과 증명 가능한 안전성을 함께 제공하는 ID 기반 온오프라인 암호 프로토콜의 설계를 다룬다. 구체적으로, 다중 사용자 환경을 위한 ID 기반 다중 대리 서명 기법과 다중 기기 환경을 위한 ID 기반 온오프라인 사인크립션 기법을 제안한다.

ID 기반 다중 대리 서명 기법(IBMPS)은 원 서명자가 직접 서명할 수 없는 상황에서, 사전에 권한을 위임받은 다수의 대리 서명자가 협력하여 원 서명자를 대신하는 대리 서명을 생성하도록 한다. 본 논문에서는 기존 기법과 달리 대리 서명키와 장기 개인키 간의 독립성을 보장함으로써 대리 서명키의 노출이 장기 개인키의 노출로 이어지지 않도록 설계하였다. 또한, CDH 가정하에서 제안 기법의 위조 불가능성을 증명하였다. 나아가 이를 온오프라인 구조로 새롭게 확장하여, 각 대리 서명자가 온라인 단계에서 타원곡선상의 덧셈 연산 한 번만으로 부분 서명을 생성할 수 있도록 하였다. 라즈베리 파이 기반 드론 환경에서의 구현 및 실험을 통해 제안 기법의 계산 효율성과 실용성을 확인하였다.

ID 기반 온오프라인 사인크립션(OOSE)은 서명과 암호화를 결합하여 하나의 기법으로 메시지의 기밀성과 인증을 동시에 제공한다. 본 논문에서는 기존 OOSE 연구에서 고려되지 않았던 오프라인 단계와 온라인 단계 사이의 메모리 노출 공격을 새롭게 정의하고, 이러한 공격에 안전한 두 가지 OOSE 기법을 제안한다. 제안 기법들은 오프라인 단계에서 생성된 중간값이 노출되더라도 오프라인 결과와 온라인 입력 간의 연계성 및 인증을 보장하도록 설계되었다. 또한, GBDH 가정하에서 기밀성에 대한 IND-CCA2 안전성을 증명하고, 인증에 대한 EUF-



CMA 안전성도 증명하였다. OMNeT++ 기반 시뮬레이션을 통해 드론 기반 모바일 네트워크 환경에서의 계산 및 통신 성능을 평가하고, 제안 기법의 실용성을 확인하였다.

종합적으로, 본 논문은 다중 사용자 및 다중 기기 구조를 갖는 자원 제약 모바일 환경에 실용적으로 적용할 수 있는 ID 기반 온오프라인 암호 프로토콜을 제안한다. 나아가 각 운용 환경에서 발생할 수 있는 대리 서명키 노출과 단계 간 메모리 노출 공격을 고려하면서도, 제안 기법들이 효율성과 증명 가능한 안전성을 함께 제공함을 보인다.

키워드: ID 기반 암호, 온오프라인 프로토콜, 다중 대리 서명, 사인크립션, 모바일 환경



Acknowledgement

This dissertation would not have been possible without the support and guidance of many people who have been part of this journey.

First and foremost, I would like to express my deepest gratitude to my advisor, Prof. Ik Rae Jeong, for his unwavering support, patience, and guidance throughout my doctoral studies. His insightful advice and encouragement have shaped not only this dissertation but also my perspective as a researcher. I am truly fortunate to have had the opportunity to learn and grow under his supervision.

I am also deeply grateful to Prof. Jin Wook Byun, Prof. Ji Young Chun, and Prof. Geontae Noh, whose guidance extended well beyond the scope of this dissertation. Prof. Byun's invaluable guidance and insightful discussions were instrumental to the completion of this research. Prof. Chun and Prof. Noh offered continued academic counsel and encouragement that helped me navigate the many challenges along the way. Learning from them has been both a privilege and a constant source of inspiration.

I would like to extend my heartfelt appreciation to all the members of the Privacy-Enhancing Technologies (PET) Laboratory. Joining this lab was a defining chapter of my life. The stimulating research environment, thoughtful discussions, and warm camaraderie we shared have profoundly shaped who I am today, both as a researcher and as a person. I will carry the memories and lessons from this place wherever I go.

To my family, I owe more than words can express. Their unconditional love, unwavering support, and willingness to think together about my future have been the very foundation upon which I could pursue this path. I am deeply grateful for their belief in me, especially during the most challenging times.

Finally, I would like to thank my best friend forever, Hyeyoung Hwang, who has shared every joy and sorrow with me since our high school days. Her quiet patience and unwavering presence throughout this long journey have been a true source of comfort and strength.

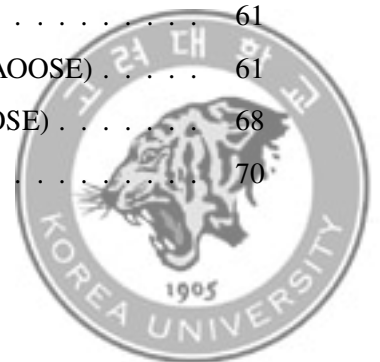


Table of Contents

Abstract	i
Acknowledgement	v
Table of Contents	vi
List of Figures	ix
List of Tables	x
Chapter 1. Introduction	1
1.1 Related Works	2
1.1.1 On/Offline Cryptographic Protocols in Signature Schemes	2
1.1.2 On/Offline Cryptographic Protocols in Encryption Schemes	5
1.1.3 On/Offline Cryptographic Protocols in Signcryption Schemes	6
1.2 Motivation	7
1.3 Contributions	9
Chapter 2. Preliminaries	11
2.1 Notation	11
2.2 ID-Based Cryptography	13
2.3 Bilinear Pairings	14
2.4 Hardness Assumptions	15



2.4.1	Computational Diffie–Hellman Problem	15
2.4.2	Gap Bilinear Diffie–Hellman Problem	15
2.4.3	Discrete Logarithm Problem	16
2.5	Chameleon Hash Function	17
Chapter 3. On/Offline Identity-Based Multi-Proxy Signature		19
3.1	Definitions	20
3.1.1	Syntax	20
3.1.2	Security Model	21
3.2	Previous Work	23
3.3	Construction	25
3.3.1	Identity-Based Multi-Proxy Signature (IBMPS)	26
3.3.2	On/Offline Identity-Based Multi-Proxy Signature (OIBMPS)	28
3.4	Security Analysis	31
3.5	Performance Analysis	40
3.5.1	Efficiency Analysis	40
3.5.2	Implementation Analysis for Mobile Applications	41
Chapter 4. On/Offline Identity-Based Signcryption		47
4.1	Definitions	48
4.1.1	Syntax	48
4.1.2	Security Model	50
4.2	Previous Work	54
4.3	Construction	55
4.3.1	Authenticated Identity-Based On/Offline Signcryption (AOOSE)	55
4.3.2	Confidentiality-Enhanced On/Offline Signcryption (COOSE)	58
4.4	Security Analysis	61
4.4.1	Authenticated Identity-Based On/Offline Signcryption (AOOSE)	61
4.4.2	Confidentiality-Enhanced On/Offline Signcryption (COOSE)	68
4.5	Performance Analysis	70



4.5.1	Efficiency Analysis	70
4.5.2	Implementation Analysis for Mobile Applications	71
Chapter 5. Conclusion		78
Bibliography		81



List of Figures

3.1	Chronological Development of IBMPS Schemes with Proxy Signing Keys	23
3.2	On/Offline IBMPS (OIBMPS)	29
3.3	Offline Protocol of OIBMPS	30
3.4	Online Protocol of OIBMPS	31
3.5	Implementation Analysis Results of IBMPS and OIBMPS	44
4.6	Performance Results for Different Numbers of Preloaded Offline Outputs	73
4.7	OMNeT++ Visualization of the Simulated IoD Network	74
4.8	Memory Usage under Different Offline-State Retrieval Strategies	75



List of Tables

2.1	Summary of Notation	12
3.2	Structure of Existing Proxy Signing Keys and Its Analysis	39
3.3	Efficiency Comparison of IBMPS Schemes	42
3.4	Comparison of Communication Overhead and Energy Consumption	43
4.5	Efficiency Comparison of On/Offline Identity-Based Signcryption Schemes . . .	71
4.6	Sizes of Algorithm Outputs and Encrypted Plaintexts	72

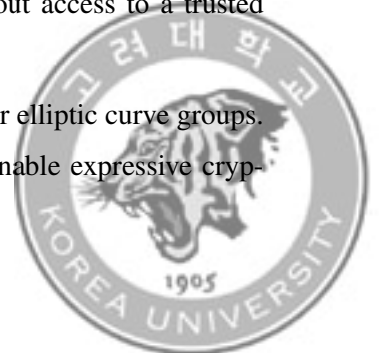


Chapter 1. Introduction

The rapid proliferation of mobile and embedded devices, coupled with the maturation of wireless communication infrastructure, has led to large-scale distributed environments in which numerous devices and users must authenticate messages and establish trust with minimal human intervention. Internet of Things (IoT) ecosystems, mobile payment platforms, Internet of Drones (IoD) networks, and vehicular ad-hoc networks (VANETs) exemplify such settings in which authentication among communicating entities is not optional but a fundamental security requirement. These environments demand cryptographic protocols that provide message authenticity, confidentiality, or both, while operating under stringent resource constraints such as limited computational power, memory, and battery capacity.

In such unmanned and heterogeneous settings, identity-based cryptography (IBC), first conceptualized by Shamir [1] in 1984 and practically realized by Boneh and Franklin [2] in 2001, offers a natural framework by allowing a user's public key to be derived directly from a unique identity string, such as a device serial number or a network address. This eliminates the need for certificates and simplifies key management inherent in traditional public-key infrastructure (PKI). While certificateless public-key cryptography [3] mitigates the key escrow problem of IBC, it requires users to manage additional public-key information, and public keys can no longer be derived solely from identities. This introduces additional communication rounds and may be impractical in scenarios where devices must communicate directly without access to a trusted intermediary during operation.

Many of the existing IBC schemes rely on bilinear pairings defined over elliptic curve groups. By exploiting the bilinearity property, pairing-based IBC constructions enable expressive cryp-



tographic designs that facilitate efficient verification procedures and compact representations of keys and signatures. However, these advantages come at a considerable computational cost, as a single pairing operation is significantly more expensive than an elliptic curve scalar multiplication. In particular, the signature generation and verification phases in pairing-based protocols typically require multiple pairing operations, which can impose a performance bottleneck on resource-constrained devices.

Therefore, preserving the advantages of pairing-based IBC while reducing its computational overhead on lightweight devices remains a key challenge for enabling its practical use in modern mobile applications. One of the promising approaches to address this problem is the use of online and offline (on/offline) cryptographic protocols, originally introduced by Even, Goldreich, and Micali [4]. These protocols shift computationally expensive operations to an offline phase performed in advance, leaving only lightweight computations in the time-critical online phase. By combining IBC with the on/offline paradigm, it becomes possible to design protocols that are both certificate-free and computationally efficient during real-time execution.

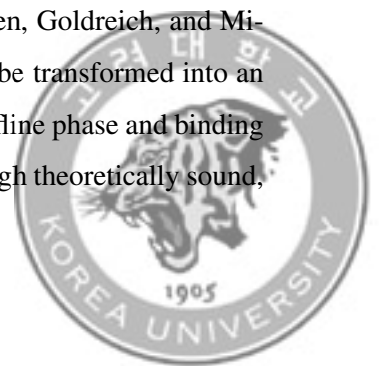
Research on on/offline cryptographic protocols in IBC can be broadly categorized into three main fields based on their security properties: on/offline signature schemes for authentication, on/offline encryption schemes for confidentiality, and on/offline signcryption schemes for both notions through an efficient composition of signature and encryption.

1.1 Related Works

In this section, we begin by reviewing prior works on on/offline cryptographic protocols in the context of public-key cryptography (PKC), including identity-based cryptography (IBC).

1.1.1 On/Offline Cryptographic Protocols in Signature Schemes

The concept of on/offline digital signatures was first introduced by Even, Goldreich, and Micali [4] in 1989. Their work demonstrated that any signature scheme can be transformed into an on/offline scheme by precomputing a one-time signature pair during the offline phase and binding it to the actual message during the online phase using a hash chain. Although theoretically sound,

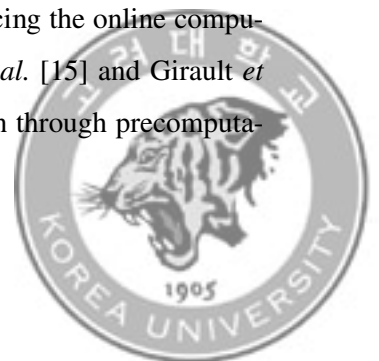


their construction incurred a significant increase in signature size due to the one-time signature overhead and was not practical for resource-constrained environments.

Shamir and Tauman [5] later proposed the *hash-sign-switch* paradigm using chameleon hash functions [6], which substantially improved the efficiency of on/offline signatures. In their approach, the signer computes a signature on a chameleon hash value during the offline phase using a random temporary message. When the actual message becomes available in the online phase, the signer uses the trapdoor key to find a collision in the chameleon hash so that the precomputed signature remains valid for the new message. The online phase thus requires only the lightweight collision-finding computation, without any expensive operations such as scalar multiplications or pairing operations. This paradigm has since become the standard technique for constructing on/offline signatures and signcryptions, and it forms a central building block in the on/offline signcrypton schemes presented in Chapter 4 of this dissertation.

Following these foundational works, several on/offline signature schemes were developed in the standard public-key setting. Kurosawa *et al.* [7] proposed efficient constructions in the standard model, while functional extensions such as threshold signatures were investigated by Crutchfield *et al.* [8] and Bresson *et al.* [9]. Yu *et al.* [10] proposed an RSA-based on/offline signature scheme, while Catalano *et al.* [11] provided both theoretical analysis and experimental evaluation. Notably, Gao *et al.* [12] introduced the concept of divisible on/offline signatures where the offline component could be made public, a notion whose relationship to conventional on/offline signatures was later critically examined by Au *et al.* [13].

Later research has further advanced on/offline cryptographic constructions along two main directions. First, studies have extended the on/offline paradigm toward quantum-resistant settings in recent years. In particular, Zheng *et al.* [14] proposed constructions based on post-quantum assumptions, aiming to preserve the benefits of the on/offline paradigm while achieving resilience against quantum adversaries. Second, considerable efforts have been devoted to enhancing efficiency and practicality in resource-constrained environments, where reducing the online computational burden remains a primary objective. Early works such as Xu *et al.* [15] and Girault *et al.* [16] explored lightweight designs that minimize real-time computation through precomputation techniques.



While most existing studies on on/offline signatures have focused on improving efficiency and extending functionality within standard signature settings [15, 16], the on/offline paradigm has been extended to specialized primitives that support delegation [17], threshold signing [15], and other forms of controlled authorization in resource-constrained environments.

Among these specialized extensions, proxy signatures are particularly relevant to distributed and multi-user environments because they allow an original signer to delegate signing authority to a designated proxy signer. This delegation capability enables flexible authorization when the original signer is absent or unable to perform signing operations directly. However, proxy signature schemes generally require additional cryptographic procedures for delegation generation and verification, proxy signing key generation, and proxy signature verification. These additional procedures may impose considerable computational overhead on resource-constrained devices.

In this regard, the on/offline paradigm provides a natural and promising approach to reducing the computational burden incurred during time-critical proxy signing operations. By shifting message-independent and computationally expensive operations, including delegation-related computations, to the offline phase, proxy signature schemes can retain their flexible authorization functionality while requiring only lightweight computation during the online phase. This separation is particularly beneficial in mobile and embedded environments where signatures must be generated promptly despite limited computational power and battery capacity. Nevertheless, the application of on/offline techniques to proxy signatures, particularly identity-based multi-proxy signatures, has remained largely underexplored.

The development of proxy signature schemes began with the work of Mambo, Usuda, and Okamoto [18], who introduced the concept in 1996. Their construction enabled an original signer to delegate signing authority to a designated proxy signer. This concept was subsequently extended to the multi-proxy setting by Hwang and Shi [19], in which multiple proxy signers cooperate to generate a single proxy signature on behalf of the original signer. Chen, Zhang, and Kim [20] later combined identity-based cryptography with multi-proxy signatures and proposed the first identity-based multi-proxy signature (IBMPS) scheme based on bilinear pairings. Since then, a series of IBMPS schemes [21, 22, 23, 24] have been proposed to improve computational efficiency and security. However, the integration of on/offline techniques into IBMPS schemes



remained largely unexplored until Shin *et al.* [17] demonstrated that such an integration could achieve remarkably lightweight online computation. In their construction, each proxy signer generates a partial multi-proxy signature during the online phase using only a single elliptic curve point addition.

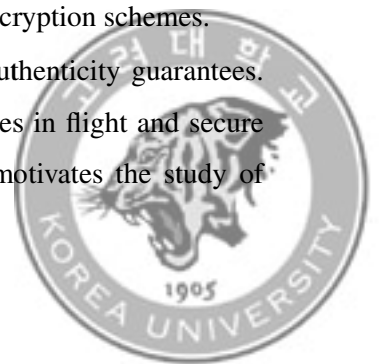
1.1.2 On/Offline Cryptographic Protocols in Encryption Schemes

On/offline techniques have also been applied to encryption schemes to reduce the computational burden on the sender side. The fundamental idea is to precompute ciphertext components that are independent of the actual message and, in some constructions, also independent of the receiver's identity.

Liu *et al.* [25] proposed an efficient identity-based on/offline encryption scheme in which the sender precomputes ciphertext components during the offline phase and completes the encryption with lightweight operations once the message and the receiver's identity are determined. Subsequent work has also focused on reducing ciphertext size to improve communication efficiency in bandwidth and resource-constrained environments. In particular, Lai *et al.* [26] proposed identity-based on/offline encryption and signcryption schemes with short ciphertexts in both the offline and online phases.

Compared to on/offline signature schemes, research on on/offline encryption remains relatively limited. Only a few works, such as Liu and Zhou [25] and Selvi *et al.* [27], have explicitly investigated identity-based on/offline encryption schemes. In many cases, encryption functionality has been studied implicitly within on/offline signcryption constructions rather than as a standalone primitive. For example, Selvi *et al.* [27] revisited the security of existing identity-based on/offline encryption and signcryption schemes, identifying vulnerabilities in several prior constructions and proposing improved schemes with formal security proofs. Their work highlights the importance of careful security analysis in the on/offline setting, where the separation of computations into two phases introduces subtle attack vectors that do not arise in standard encryption schemes.

These works primarily focus on confidentiality and do not provide authenticity guarantees. However, in many practical scenarios, including unmanned mobile devices in flight and secure mobile messaging, both properties are simultaneously required. This motivates the study of



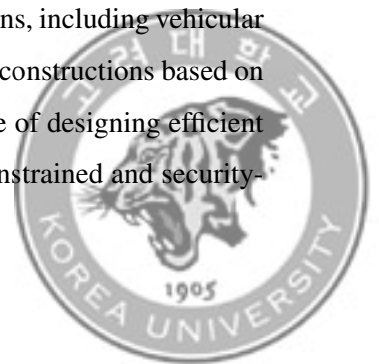
on/offline signcryption, which combines signature and encryption into a single primitive and can provide both properties more efficiently than a generic sequential composition of signature and encryption.

1.1.3 On/Offline Cryptographic Protocols in Signcryption Schemes

Signcryption, introduced by Zheng [28] in 1997, achieves both message confidentiality and authenticity at a cost significantly lower than the sequential application of a signature scheme and an encryption scheme. Broadly, approaches to combining these functionalities can be classified into sequential composition (e.g., Sign-then-Encrypt and Encrypt-then-Sign) and integrated constructions, where signing and encryption are performed in a unified manner. An *et al.* [29] formally analyzed the security of joint signature and encryption, establishing the theoretical foundations for modern signcryption security models. The concept was subsequently extended to the identity-based setting by Malone-Lee [30], Libert *et al.* [31], and Chen *et al.* [32], and later combined with on/offline techniques to reduce the computational overhead on the sender side.

The first on/offline signcryption scheme was proposed by Zhang *et al.* [33] in 2005, demonstrating the feasibility of precomputing signcryption components to reduce the sender's online workload. For the identity-based setting, Sun *et al.* [34] subsequently proposed an identity-based on/offline signcryption scheme with precomputed decryption keys, which further reduced online computation but required the receiver's identity to be fixed during the offline phase. Later, Sun *et al.* [35] presented a generic construction of identity-based on/offline signcryption (OOSE) based on the hash-sign-switch paradigm with chameleon hash functions. This framework has served as the foundation for numerous subsequent works. Liu *et al.* [36] improved efficiency, while Li *et al.* [37] optimized OOSE for low-power devices. More recent studies have focused on practical deployments and specialized constructions, including IoT [38], short-ciphertext constructions [26], and UAV communication scenarios [39].

In parallel, recent research has explored signcryption in emerging domains, including vehicular networks [40, 41], electronic payment systems [42, 43], and post-quantum constructions based on lattices [44, 45, 46]. These developments further highlight the importance of designing efficient and robust on/offline signcryption schemes for deployment in resource-constrained and security-



critical environments.

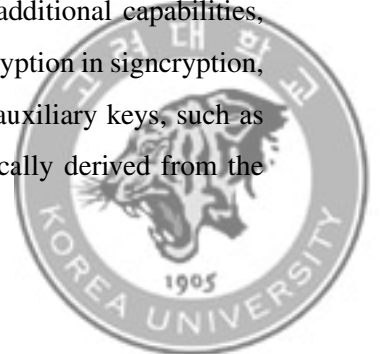
Despite these advances in efficiency, existing constructions commonly rely on an implicit assumption that the offline and online phases are executed within a single trusted environment by the same entity on the same device. However, this assumption does not hold in practical multi-device settings. As recently highlighted by Byun [47], memory corruption between phases can enable an adversary to generate valid signcryption ciphertexts using leaked offline outputs, especially in mobile applications. In particular, in the generic construction of Sun *et al.* [35], the offline output includes a precomputed decryption key and a signature on a temporary message. If these values are exposed (e.g., through device capture), an adversary can execute the online phase to generate valid ciphertexts for arbitrary messages without possessing the sender’s private key.

Therefore, on/offline signcryption schemes should not only remain practical for resource-constrained mobile applications, but also preserve security under realistic execution settings in which the offline and online phases may be separated across devices and subject to cross-phase memory exposure.

1.2 Motivation

Despite significant advances in identity-based cryptography and on/offline protocol design, the secure and practical realization of such protocols for real-world mobile applications remains a challenging problem. In emerging mobile application environments, cryptographic primitives must not only achieve strong security guarantees but also be deployable under stringent resource constraints and heterogeneous execution settings. While prior works have primarily focused on improving efficiency or extending functionality, the interplay between extended functionalities and realistic deployment conditions has not been sufficiently explored.

The first concern arises from *key exposure attacks* in cryptographic primitives that incorporate multiple functionalities. When a base primitive is extended to support additional capabilities, such as delegation in proxy signatures or combined authentication and encryption in signcryption, auxiliary keys or intermediate secret values are often introduced. These auxiliary keys, such as proxy signing keys or decryption keys, are often structurally or algebraically derived from the



user’s long-term private key. In such constructions, it is critical to ensure that the compromise of an auxiliary key does not lead to the exposure of the user’s private key. In existing identity-based multi-proxy signature schemes [20, 21, 22, 48, 49, 50], for example, the proxy signing key is formed by a simple algebraic combination of the proxy signer’s private key and the delegation, so that an adversary who obtains the proxy signing key can recover the private key. This *proxy signing key exposure attack* [51, 52] is particularly threatening in mobile applications such as Internet of Drones (IoD) systems, where drones operating in hostile territory are vulnerable to physical capture. Similarly, in existing signcryption schemes, such as Sun et al. [35], access to the sender’s auxiliary key for decryption key establishment enables unauthorized generation of a valid online ciphertext, which compromises message authentication and confidentiality. This problem becomes even more severe in multi-instance and multi-user environments, where parallel and repeated executions of the same primitive may involve shared states or related secret information. Despite its importance, such exposure resilience in the presence of functionality extensions has not been systematically addressed in existing identity-based constructions.

The second concern stems from the *security gap between the offline and online phases* in on/offline protocols deployed across multiple devices. In practical deployments, the offline and online phases may be executed in physically or logically separated contexts. As a result, intermediate outputs generated during the offline phase, which may contain secret-dependent information such as precomputed signatures or decryption keys, must be stored in the device’s memory and later retrieved during the online phase. If an adversary gains access to this memory through corruption or device compromise, the intermediate outputs may be exposed. In most existing on/offline constructions related to signcryption [35, 34, 27, 37, 36, 38, 26, 39], the exposed offline outputs contain sufficient information for any party to execute the online phase and produce a valid output without possessing the sender’s private key. This security threat, referred to as *memory corruption* [47] between phases, fundamentally undermines the authentication guarantee of on/offline protocols.

Motivated by these observations, this dissertation aims to develop identity-based on/offline cryptographic protocols that are both practically efficient and provably secure under realistic deployment conditions. Specifically, this dissertation considers two representative primitives for



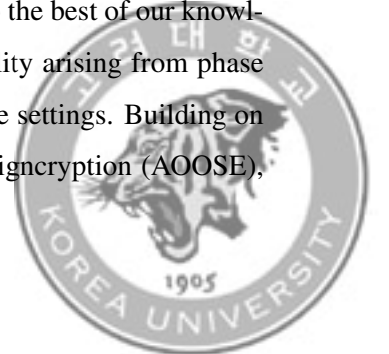
mobile applications: multi-proxy signatures, which support delegated authentication in multi-user settings, and on/offline signcryption, which provides integrated authenticity and confidentiality in multi-device settings. It also addresses the security challenges that arise when these primitives are extended to support delegation to multiple parties, integrated signing and encryption operations, and on/offline execution across multiple devices and instances.

Our goal can be categorized as follows. First, to ensure that the exposure of any auxiliary key does not compromise the user’s long-term private key. Second, to guarantee that even under memory corruption between phases, only an authorized entity in possession of the private key can produce a valid online output.

1.3 Contributions

The contributions of this dissertation are organized as follows. First, we propose a provably secure identity-based multi-proxy signature (IBMPS) scheme based on bilinear pairings tailored for multi-user environments. Our construction employs an independent random value in the generation of the proxy signing key, ensuring that the exposure of the proxy signing key does not compromise the proxy signer’s long-term private key. We provide a systematic structural analysis of existing proxy signing keys in IBMPS schemes and demonstrate that most prior constructions [20, 21, 22, 48, 49, 50] are vulnerable to the proxy signing key exposure attack. We prove that our scheme achieves existential unforgeability under the Computational Diffie–Hellman (CDH) assumption in the random oracle model. Importantly, we also extend the proposed scheme to an on/offline protocol for IoD applications, in which each proxy signer generates its partial multi-proxy signature during the online phase by performing only a single elliptic curve point addition on the elliptic curve.

Second, we introduce a formal security model for identity-based on/offline signcryption (OOSE) that captures memory corruption between the offline and online phases. To the best of our knowledge, this is the first work to formally identify and address the vulnerability arising from phase separation in on/offline signcryption under multi-device and multi-instance settings. Building on this model, we propose two OOSE schemes: Authenticated On/Offline Signcryption (AOOSE),



which guarantees authenticated linkage between the offline and online phases by relocating the decryption key generation to the online phase, and Confidentiality-enhanced On/Offline Signcryption (COOSE), which additionally encrypts the offline output to protect its confidentiality. We prove the IND-CCA2 confidentiality of both schemes under the Gap Bilinear Diffie–Hellman (GBDH) assumption in the random oracle model. We establish their EUF-CMA unforgeability based on the existential unforgeability of the underlying identity-based signature scheme and the collision resistance of the chameleon hash function. Our constructions also employ separate master secrets for identity-based signature and identity-based key agreement, ensuring that the compromise of one primitive does not propagate to another.

Third, we implement the proposed on/offline cryptographic protocols and evaluate their practicality in mobile environments. For the IBMPS scheme, we implement our construction on a Raspberry Pi Zero W-based drone platform using the Pairing-Based Cryptography (PBC) library and measure computation time, communication overhead, and energy consumption against representative baseline schemes. For the OOSE schemes, we build a wireless drone network simulation using the OMNeT++ simulator integrated with the INET framework and evaluate latency, throughput, and memory usage as the number of preloaded offline outputs scales. The experimental results demonstrate that our constructions achieve a favorable balance between enhanced security guarantees and practical efficiency, confirming their suitability for deployment in resource-constrained mobile applications.

Organization. The remainder of this dissertation is organized as follows. Chapter 2 presents the mathematical preliminaries, including identity-based cryptography, bilinear pairings, hardness assumptions, and the chameleon hash function. Chapter 3 proposes our identity-based multi-proxy signature scheme and its on/offline extension, along with the security analysis and implementation results. Chapter 4 introduces the formal security model for on/offline signcryption under memory corruption, presents the AOOSE and COOSE constructions, and provides the security proofs and performance evaluation. Chapter 5 concludes the dissertation.



Chapter 2. Preliminaries

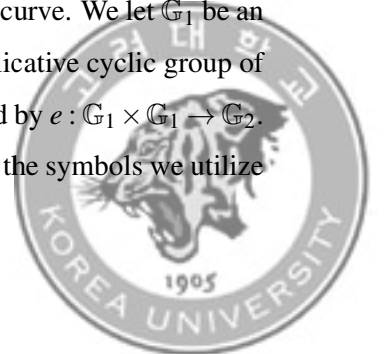
In this chapter, we present the mathematical foundations and cryptographic building blocks that underlie the schemes proposed in Chapters 3 and 4. We begin by introducing the basic notation and conventions, then present identity-based cryptography and discuss its suitability for resource-constrained environments. Next, we define the bilinear pairing setting and the computational hardness assumptions. Finally, we describe the chameleon hash function, which serves as an essential component of our on/offline signcryption framework.

2.1 Notation

Let $\lambda \in \mathbb{N}$ denote the security parameter. A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is negligible if, for every positive polynomial $p(\cdot)$, there exists $N \in \mathbb{N}$ such that $\text{negl}(\lambda) < 1/p(\lambda)$ for all $\lambda > N$. We use $\mathcal{A}$ to denote a probabilistic polynomial-time (PPT) adversary, and $\mathbf{pp}$ to denote the public system parameters.

If $\mathcal{A}$ is probabilistic, we write $y \xleftarrow{\$} \mathcal{A}(x)$. If $\mathcal{A}$ is deterministic, we write $y \leftarrow \mathcal{A}(x)$. For a finite set S , the notation $y \xleftarrow{\$} S$ means that y is sampled uniformly at random from S . For $n \in \mathbb{N}$, we write $[n]$ to denote the set $\{1, \dots, n\}$. The symbol $\parallel$ denotes concatenation.

For elliptic curve and group parameters, $E(\mathbb{F}_t)$ denotes an elliptic curve defined over the finite field $\mathbb{F}_t$ where t is a prime power and $\#E(\mathbb{F}_t)$ is the number of points on the curve. We let $\mathbb{G}_1$ be an additive cyclic group of prime order q generated by P , and $\mathbb{G}_2$ be a multiplicative cyclic group of the same prime order q , where $q \mid \#E(\mathbb{F}_t)$. A bilinear pairing map is denoted by $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$. We denote $\mathbb{Z}_q^*$ as the multiplicative group of integers modulo q . The rest of the symbols we utilize



for Identity-based key management and cryptographic hash functions are described in Table 2.1.

Table 2.1: Summary of Notation

Symbol	Description
$\mathbb{G}_1$	An additive cyclic group of prime order q , generated by P
$\mathbb{G}_2$	A multiplicative cyclic group of the same prime order q
P	A generator of $\mathbb{G}_1$
e	A bilinear pairing map $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$
$\mathbb{Z}_q^*$	Multiplicative group of integers modulo q
KGC	Key generation center, a trusted authority
$s, \bar{s}$	Master secret keys kept by KGC
$P_{pub}, \bar{P}_{pub}$	Master public keys, where $P_{pub} = sP$ and $\bar{P}_{pub} = \bar{s}P$
ID_k	An identity string of user U_k , for $k \in \{0, 1, \dots, n\}$
pk_k	A set of public keys of ID_k
sk_k	A set of private/secret keys of ID_k
n	The number of users
Q_{ID_k}	A public key of ID_k derived via IBS, where $Q_{ID_k} = H_1(ID_k)$
S_{ID_k}	A private key of ID_k derived via IBS, where $S_{ID_k} = s \cdot Q_{ID_k}$
$\bar{Q}_{ID_k}$	A public key of ID_k derived via IBK, where $\bar{Q}_{ID_k} = H_0(ID_k)$
$\bar{S}_{ID_k}$	A private key of ID_k derived via IBK, where $\bar{S}_{ID_k} = \bar{s} \cdot \bar{Q}_{ID_k}$
CK_k	Trapdoor key of ID_k for the Chameleon hash, where $CK_k = y_k \in \mathbb{Z}_q^*$
HK_k	Chameleon hash key of ID_k , where $HK_k = CK_k P = y_k P$
H_0, H_1, H_2, H_3	Cryptographic hash functions
T_M	A scalar multiplication operation in $\mathbb{G}_1$
T_P	A bilinear pairing computation
T_E	An exponentiation operation



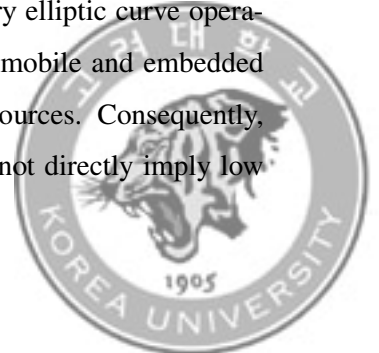
2.2 ID-Based Cryptography

Identity-based cryptography (IBC) is a public-key cryptographic paradigm in which a user's public key is derived directly from a publicly known identity string, such as an email address, a device serial number, or any other unique identifier. The concept was first proposed by Shamir [1] in 1984, who envisioned a system that would eliminate the need for digital certificates and the complex public-key infrastructure (PKI) required to manage them.

In a traditional certificate-based PKI setting, each user must obtain a digital certificate from a certificate authority (CA) that binds the user's identity to a public key. This process involves certificate generation, distribution, storage, revocation, and verification, all of which impose significant computational and communication overhead. In contrast, IBC allows any party to compute a user's public key solely from the user's identity and the system's public parameters, thereby removing the need for certificates entirely.

The practical realization of IBC remained an open problem for nearly two decades until Boneh and Franklin [2] proposed the first practical and provably secure identity-based encryption (IBE) scheme based on bilinear pairings in 2001. Their construction demonstrated that the Weil pairing on elliptic curves could be leveraged to build an IBE scheme with provable security under the Bilinear Diffie–Hellman assumption in the random oracle model. This breakthrough sparked extensive research into identity-based primitives, including identity-based signature (IBS) schemes [53, 54, 55], identity-based key agreement (IBK) [56], and identity-based signcryption [30, 31].

Although IBC simplifies public-key distribution by eliminating the need for digital certificates, its application to resource-constrained environments remains challenging. Since the KGC generates users' private keys, the system inherently requires complete trust in the KGC and is subject to the key-escrow problem. Moreover, many existing IBC constructions are built from bilinear pairings, whose computation is considerably more expensive than ordinary elliptic curve operations. Such computational overhead can be particularly burdensome for mobile and embedded devices with limited processing power, memory capacity, and battery resources. Consequently, the reduction in certificate-management overhead provided by IBC does not directly imply low



computational cost or efficient real-time execution. This limitation provides an important motivation for combining IBC with the on/offline paradigm, which moves expensive computations to a precomputation phase and leaves only lightweight operations for the online phase.

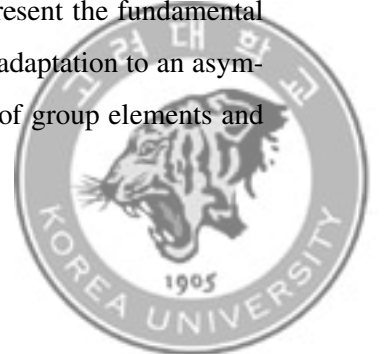
2.3 Bilinear Pairings

Definition 1 (Bilinear Pairing). *Let $\mathbb{G}_1$ be an additive cyclic group generated by P and $\mathbb{G}_2$ be a multiplicative cyclic group, both of the same prime order q . A bilinear pairing is a map $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ with the following properties.*

1. **Bilinearity.** *For all $P, Q \in \mathbb{G}_1$ and $a, b \in \mathbb{Z}_q^*$, we have $e(aP, bQ) = e(P, Q)^{ab}$.*
2. **Non-degeneracy.** *There exist $P, Q \in \mathbb{G}_1$ such that $e(P, Q) \neq 1_{\mathbb{G}_2}$.*
3. **Computability.** *There exists an efficient algorithm to compute $e(P, Q)$ for any $P, Q \in \mathbb{G}_1$.*

In this dissertation, we consider a symmetric Type-1 bilinear pairing, in which both input elements belong to the same additive group $\mathbb{G}_1$. This symmetric setting simplifies the notation and construction of the proposed schemes because public keys, private-key components, signatures, and intermediate values can be represented within a single source group. It also allows the verification equations to be expressed without separately managing elements from two distinct source groups.

Although modern implementations often employ asymmetric Type-3 pairings for improved computational efficiency and stronger security per parameter size, the proposed constructions may be considered for adaptation to an asymmetric Type-3 pairing setting. A Type-3 instantiation can be obtained by assigning each group element to an appropriate source group, defining separate hash-to-group functions, and reformulating the pairing equations accordingly, provided that no operation requires an efficiently computable isomorphism between the two source groups. Therefore, this dissertation adopts the Type-1 pairing setting to clearly present the fundamental construction and security properties of the proposed schemes, while their adaptation to an asymmetric pairing setting may be considered through a careful reassignment of group elements and reformulation of the pairing equations.



2.4 Hardness Assumptions

2.4.1 Computational Diffie–Hellman Problem

The Computational Diffie–Hellman (CDH) problem serves as the underlying hardness assumption for the identity-based multi-proxy signature (IBMPS) scheme presented in Chapter 3.

Definition 2 (CDH Problem). *Let $\mathbb{G}_1$ be an additive cyclic group of prime order q generated by P . Given $(P, aP, bP) \in \mathbb{G}_1^3$ for randomly chosen $a, b \xleftarrow{\$} \mathbb{Z}_q^*$, the CDH problem in $\mathbb{G}_1$ is to compute $abP \in \mathbb{G}_1$.*

Definition 3 (CDH Assumption). *For a probabilistic polynomial-time algorithm $\mathcal{B}$, define its advantage in solving the CDH problem in $\mathbb{G}_1$ as*

$$\text{Adv}_{\mathcal{B}}^{\text{CDH}}(\lambda) = \Pr \left[\mathcal{B}(P, aP, bP) = abP : a, b \xleftarrow{\$} \mathbb{Z}_q^* \right].$$

We say that the CDH assumption holds in $\mathbb{G}_1$ if $\text{Adv}_{\mathcal{B}}^{\text{CDH}}(\lambda)$ is negligible in the security parameter λ for every probabilistic polynomial-time algorithm $\mathcal{B}$.

2.4.2 Gap Bilinear Diffie–Hellman Problem

The Gap Bilinear Diffie–Hellman (GBDH) problem is a computational problem in which an algorithm is additionally granted access the DBDH oracle. The GBDH assumption serves as the underlying hardness assumption for the confidentiality analysis of the on/offline signcryption schemes presented in Chapter 4.

Let $\mathcal{O}_{\text{DBDH}}$ denote an oracle that, on input $(P, A, B, C, T) \in \mathbb{G}_1^4 \times \mathbb{G}_2$, returns

$$\mathcal{O}_{\text{DBDH}}(P, A, B, C, T) = \begin{cases} 1, & \text{if } T = e(P, P)^{abc} \text{ for } A = aP, B = bP, C = cP, \\ 0, & \text{otherwise.} \end{cases}$$

Definition 4 (Gap Bilinear Diffie–Hellman Problem). *Let $\mathbb{G}_1$ be an additive cyclic group and let $\mathbb{G}_2$ be a multiplicative cyclic group, both of the same prime order q . Let $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$ be a*



bilinear pairing, and let P be a generator of $\mathbb{G}_1$. Given $(P, aP, bP, cP) \in \mathbb{G}_1^4$ for randomly chosen $a, b, c \xleftarrow{\$} \mathbb{Z}_q^*$, and given access to $\mathcal{O}_{\text{DBDH}}$, the GBDH problem is to compute $e(P, P)^{abc} \in \mathbb{G}_2$.

Definition 5 (GBDH Assumption). *For a probabilistic polynomial-time algorithm $\mathcal{A}$ with access to $\mathcal{O}_{\text{DBDH}}$, define its advantage in solving the GBDH problem as*

$$\text{Adv}_{\mathcal{A}}^{\text{GBDH}}(\lambda) = \Pr \left[\mathcal{A}^{\mathcal{O}_{\text{DBDH}}}(P, aP, bP, cP) = e(P, P)^{abc} : a, b, c \xleftarrow{\$} \mathbb{Z}_q^* \right].$$

We say that the GBDH assumption holds in $(\mathbb{G}_1, \mathbb{G}_2)$ if $\text{Adv}_{\mathcal{A}}^{\text{GBDH}}(\lambda)$ is negligible in the security parameter λ for every probabilistic polynomial-time algorithm $\mathcal{A}$ with access to $\mathcal{O}_{\text{DBDH}}$.

The GBDH assumption is generally regarded as stronger than the standard BDH assumption because it assumes that the computational problem remains hard even when the adversary is additionally granted access to a DBDH oracle.

2.4.3 Discrete Logarithm Problem

The Discrete Logarithm (DL) problem in $\mathbb{G}_1$ underlies the collision resistance of the chameleon hash function used in Chapter 4.

Definition 6 (DL Problem). *Let $\mathbb{G}_1$ be an additive cyclic group of prime order q generated by P . Given $(P, Q) \in \mathbb{G}_1^2$, where $Q = xP$ for a randomly chosen $x \xleftarrow{\$} \mathbb{Z}_q^*$, the DL problem in $\mathbb{G}_1$ is to compute x .*

Definition 7 (DL Assumption). *For a probabilistic polynomial-time algorithm $\mathcal{B}$, define its advantage in solving the DL problem in $\mathbb{G}_1$ as*

$$\text{Adv}_{\mathcal{B}}^{\text{DL}}(\lambda) = \Pr \left[\mathcal{B}(P, xP) = x : x \xleftarrow{\$} \mathbb{Z}_q^* \right].$$

We say that the DL assumption holds in $\mathbb{G}_1$ if $\text{Adv}_{\mathcal{B}}^{\text{DL}}(\lambda)$ is negligible in the security parameter λ for every probabilistic polynomial-time algorithm $\mathcal{B}$.



2.5 Chameleon Hash Function

A chameleon hash function is a trapdoor hash family that allows the holder of a secret trapdoor key to efficiently generate collisions. In this dissertation, the chameleon hash function serves as an essential building block for the on/offline signcryption schemes presented in Chapter 4. Specifically, it enables a sender to sign the chameleon hash of a temporary message during the offline phase and later adapt the corresponding randomness to an actual message during the online phase while preserving the same hash value. Chameleon hashing was first introduced by Krawczyk and Rabin [6] and has since been widely used in on/offline signature and signcryption constructions through the hash-sign-switch paradigm [5].

For simplicity, we assume that messages are represented as elements of $\mathbb{Z}_q$. An arbitrary bit-string message can be mapped to $\mathbb{Z}_q$ using an appropriate cryptographic hash function.

Definition 8 (Chameleon Hash Function). *A chameleon hash function is a tuple of polynomial-time algorithms $\text{CH} = (\text{CH.Setup}, \text{CH.Hash}, \text{CH.Col})$ defined as follows.*

- $\text{CH.Setup}(\lambda)$: *Given a security parameter λ , the setup algorithm randomly selects $y \xleftarrow{\$} \mathbb{Z}_q^*$ and outputs a public hash key $HK = yP \in \mathbb{G}_1$ and a corresponding secret trapdoor key $CK = y$, where P is a generator of $\mathbb{G}_1$.*
- $\text{CH.Hash}(HK, m'; r')$: *Given the public hash key HK , a message $m' \in \mathbb{Z}_q$, and randomness $r' \in \mathbb{Z}_q$, the hash algorithm outputs $h = m'P + r'HK \in \mathbb{G}_1$.*
- $\text{CH.Col}(CK, m', r', m)$: *Given the trapdoor key CK , an original message-randomness pair (m', r') , and a target message $m \in \mathbb{Z}_q$, the collision-finding algorithm computes $r = r' + CK^{-1}(m' - m) \pmod{q}$. It outputs $r \in \mathbb{Z}_q$ such that $\text{CH.Hash}(HK, m'; r') = \text{CH.Hash}(HK, m; r)$.*

Correctness. For every key pair (HK, CK) generated by $\text{CH.Setup}(\lambda)$ and every $m, m', r' \in \mathbb{Z}_q$, let $r = r' + CK^{-1}(m' - m) \pmod{q}$.



Since $HK = CK \cdot P$, we have

$$\begin{aligned}
\text{CH.Hash}(HK, m; r) &= mP + rHK \\
&= mP + (r' + CK^{-1}(m' - m))CKP \\
&= m'P + r'HK \\
&= \text{CH.Hash}(HK, m'; r').
\end{aligned}$$

Therefore, the collision generated by CH.Col preserves the original hash value.

Collision resistance without the trapdoor. For any probabilistic polynomial-time algorithm $\mathcal{A}$ that does not possess the trapdoor key CK , the probability that $\mathcal{A}$ finds two distinct message-randomness pairs $(m_1, r_1) \neq (m_2, r_2)$ satisfying

$$\text{CH.Hash}(HK, m_1; r_1) = \text{CH.Hash}(HK, m_2; r_2)$$

is negligible in the security parameter λ . For the construction defined above, this property follows from the discrete logarithm assumption in $\mathbb{G}_1$.

Trapdoor collision finding. Given the trapdoor key CK , its holder can efficiently adapt the randomness associated with any hash value to an arbitrary target message. In particular, for

$$h = \text{CH.Hash}(HK, m'; r')$$

and any target message m , the holder of CK can invoke $\text{CH.Col}(CK, m', r', m)$ to compute r such that

$$h = \text{CH.Hash}(HK, m; r).$$

Uniformity. For any fixed message $m \in \mathbb{Z}_q$, if $r \xleftarrow{\$} \mathbb{Z}_q$, then $\text{CH.Hash}(HK, m; r)$ is uniformly distributed over $\mathbb{G}_1$. Consequently, the hash value does not reveal the encoded message solely from its distribution.

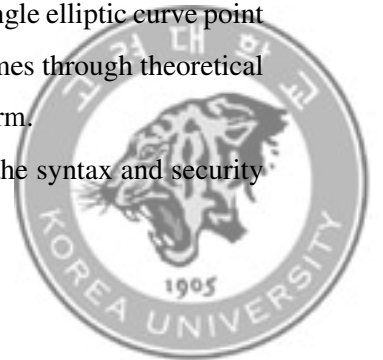


Chapter 3. On/Offline Identity-Based Multi-Proxy Signature

This chapter investigates identity-based multi-proxy signature (IBMPS) schemes that provide both provable security and practical efficiency, together with their extension to an on/offline setting for mobile applications. Previous IBMPS studies have mainly pursued computational efficiency or formal security guarantees, but relatively limited attention has been given to the consequences of proxy signing key exposure in resource-constrained deployment environments. Moreover, the pairing operations required by many existing schemes may impose a substantial computational burden on lightweight platforms. These issues are particularly relevant to Internet of Drones (IoD) systems, where mobile devices operate with limited processing capability, memory, and energy resources and may also be exposed to physical compromise.

This chapter addresses these limitations in three directions. First, we examine the structures of proxy signing keys used in representative IBMPS schemes and determine whether exposure of such keys enables recovery of the corresponding long-term private keys. Based on the resulting analysis, we construct an IBMPS scheme that resists the considered key exposure attack and prove its security in the random oracle model. Second, we extend the scheme into an on/offline protocol by moving computationally expensive operations to the offline phase. Consequently, each proxy signer completes the online generation of a partial signature using only a single elliptic curve point addition. Third, we assess the practical performance of the proposed schemes through theoretical comparisons and an implementation on a Raspberry Pi-based drone platform.

The remainder of this chapter is organized as follows. We first define the syntax and security



model of IBMPS schemes. We then review the chronology of existing IBMPS constructions and discuss their limitations. Next, we present the proposed IBMPS scheme and its on/offline extension. We subsequently establish Type I and Type II security under the CDH assumption in the random oracle model. Finally, we evaluate the proposed schemes through computational comparisons and implementation experiments.

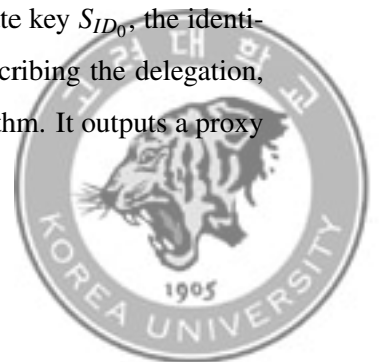
3.1 Definitions

3.1.1 Syntax

An identity-based multi-proxy signature (IBMPS) scheme involves an original signer U_0 , a set of proxy signers $\{U_i\}_{i \in N}$, where $N = \{1, \dots, n\}$, and a clerk selected from the proxy signer group. The clerk collects and verifies partial signatures and subsequently generates the final multi-proxy signature. For notational convenience, we use $M = \{0, \dots, n\}$ to index the original signer and all proxy signers collectively.

Definition 9 (Identity-Based Multi-Proxy Signature, IBMPS). *An IBMPS scheme consists of the following polynomial-time algorithms: $\text{IBMPS} = (\text{Setup}, \text{KGen}, \text{DelGen}, \text{ProxyKGen}, \text{MulProxySign}, \text{MulProxyVer})$.*

- $\text{Setup}(\lambda)$: On input of the security parameter λ , the key generation center (KGC) runs this probabilistic polynomial-time algorithm to generate the public parameters $\mathbf{pp}$ and the master secret key s . The KGC publishes $\mathbf{pp}$ and securely retains s .
- $\text{KGen}(\mathbf{pp}, ID_j)$: Given $\mathbf{pp}$ and an identity ID_j associated with user U_j , where $j \in M$, the KGC runs this probabilistic polynomial-time algorithm. It outputs the public key Q_{ID_j} and the corresponding private key S_{ID_j} .
- $\text{DelGen}(\mathbf{pp}, S_{ID_0}, \{ID_i\}_{i \in N}, w)$: Given $\mathbf{pp}$, the original signer's private key S_{ID_0} , the identities of the designated proxy signers $\{ID_i\}_{i \in N}$, and a warrant w describing the delegation, the original signer U_0 runs this probabilistic polynomial-time algorithm. It outputs a proxy delegation value σ_0 .



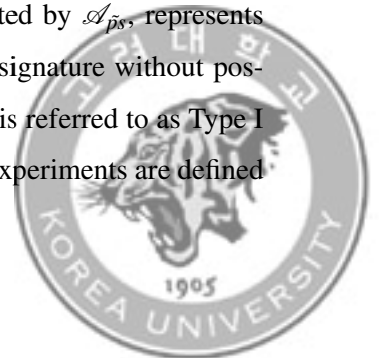
- $\text{ProxyKGen}(w, \sigma_0, S_{ID_i})$: Given w , σ_0 , and S_{ID_i} , each proxy signer U_i , for $i \in N$, runs this probabilistic polynomial-time algorithm. It outputs the proxy signing key S_{U_i} corresponding to U_i .
- $\text{MulProxySign}(\mathbf{pp}, w, S_{U_i}, \sigma_0, m, \{\sigma\}_{i \in N})$: Given $\mathbf{pp}$, w , S_{U_i} , σ_0 , a message m , and the set of partial signatures $\{\sigma_i\}_{i \in N}$, the proxy signers interactively generate their partial signatures using their respective proxy signing keys. One of the proxy signers, known as the clerk, then verifies the collected partial signatures and aggregates the valid contributions into the final multi-proxy signature σ .
- $\text{MulProxyVer}(\mathbf{pp}, \{ID_j\}_{j \in M}, m, \sigma)$: Given $\mathbf{pp}$, the identities $\{ID_j\}_{j \in M}$, a message m , and a multi-proxy signature σ , any party runs this deterministic polynomial-time algorithm. It outputs 1 if σ is valid and 0 otherwise.

Correctness. An IBMPS scheme is correct if the following condition holds:

$$\Pr \left[\begin{array}{l} \mathbf{pp} \leftarrow \text{Setup}(1^\lambda), \\ (Q_{ID_j}, S_{ID_j}) \leftarrow \text{KGen}(\mathbf{pp}, ID_j), \\ \sigma_0 \leftarrow \text{DelGen}(\mathbf{pp}, S_{ID_0}, \{ID_i\}_{i \in N}, w), \\ S_{U_i} \leftarrow \text{ProxyKGen}(w, \sigma_0, S_{ID_i}), \\ \sigma \leftarrow \text{MulProxySign}(\mathbf{pp}, w, S_{U_i}, \sigma_0, m, \{\sigma\}_{i \in N}) \end{array} \right] = 1.$$

3.1.2 Security Model

We consider two adversarial types in the IBMPS security model. The first adversary, denoted by $\mathcal{A}_{\tilde{d}}$, represents a malicious proxy signer that attempts to produce a valid multi-proxy signature without obtaining an authorized delegation. The second adversary, denoted by $\mathcal{A}_{\tilde{ps}}$, represents a malicious original signer that attempts to generate a valid multi-proxy signature without possessing the required proxy signing key. Security against these adversaries is referred to as Type I and Type II unforgeability, respectively [24]. The corresponding security experiments are defined



through $\text{Game}_{\tilde{d}}$ and $\text{Game}_{\tilde{ps}}$ as follows:

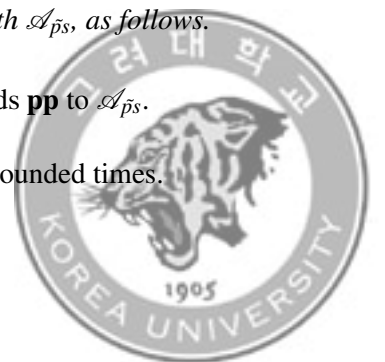
Definition 10 (Type I: Unforgeability Against $\mathcal{A}_{\tilde{d}}$). *The challenger $\mathcal{C}$ interacts with $\mathcal{A}_{\tilde{d}}$ in the $\text{Game}_{\tilde{d}}$, as follows.*

1. **Setup phase.** $\mathcal{C}$ runs $\text{Setup}(1^k)$ to generate $\mathbf{pp}$. It sends $\mathbf{pp}$ to $\mathcal{A}_{\tilde{d}}$.
2. **Training phase.** $\mathcal{A}_{\tilde{d}}$ adaptively issues a polynomial number of the following oracle queries.
 - Hash queries: On an input, $\mathcal{C}$ computes its hash value and returns it to $\mathcal{A}_{\tilde{d}}$.
 - KGen queries: On an arbitrary user identity ID_i , $\mathcal{C}$ runs $\text{KGen}(\mathbf{pp}, ID_i)$ and returns the private key S_{ID_i} to $\mathcal{A}_{\tilde{d}}$.
 - DelGen queries: On an input of (ID_0, w) , $\mathcal{C}$ runs $\text{DelGen}(\mathbf{pp}, S_{ID_0}, \{ID_i\}_{i \in N}, w)$ and returns σ_0 to $\mathcal{A}_{\tilde{d}}$.
 - ProxyKGen queries: On an input (ID_i, ID_0, w) , $\mathcal{C}$ runs $\text{ProxyKGen}(w, \sigma_0, S_{ID_i})$ and returns S_{U_i} to $\mathcal{A}_{\tilde{d}}$.
 - MulProxySign queries: On an input $(ID_0, \{ID_i\}_{i \in N}, w, m)$, $\mathcal{C}$ runs $\text{MulProxySign}(\mathbf{pp}, w, S_{U_i}, \sigma_0, m, \{\sigma_i\}_{i \in N})$ and returns σ to $\mathcal{A}_{\tilde{d}}$.
3. **Forgery phase.** $\mathcal{A}_{\tilde{d}}$ finally outputs a forged multi-proxy signature σ^* on a message m^* with a warrant w^* . We say that $\mathcal{A}_{\tilde{d}}$ wins the game if the following conditions are all satisfied:
 - σ^* is a valid multi-proxy signature, i.e., $\text{MulProxyVer}(\mathbf{pp}, \{ID_j\}_{j \in M}, m^*, \sigma^*) = 1$.
 - w^* has not been queried in any DelGen query.
 - (w^*, m^*) has not been queried in any MulProxySign query.

If the probability of $\mathcal{A}_{\tilde{d}}$ winning the game $\text{Game}_{\tilde{d}}$, denoted by $\epsilon_{\tilde{d}}$, is *negligible*, then the IBMPS scheme is said to guarantee unforgeability against $\mathcal{A}_{\tilde{d}}$, i.e., Type I security.

Definition 11 (Type II: Unforgeability Against $\mathcal{A}_{\tilde{ps}}$). *Game $_{\tilde{ps}}$ interacts with $\mathcal{A}_{\tilde{ps}}$, as follows.*

1. $\text{Game}_{\tilde{ps}}$ runs $\text{Setup}(1^k)$ to generate the public parameters $\mathbf{pp}$. It sends $\mathbf{pp}$ to $\mathcal{A}_{\tilde{ps}}$.
2. $\mathcal{A}_{\tilde{ps}}$ adaptively issues the same queries as in $\text{Game}_{\tilde{d}}$ in polynomial bounded times.



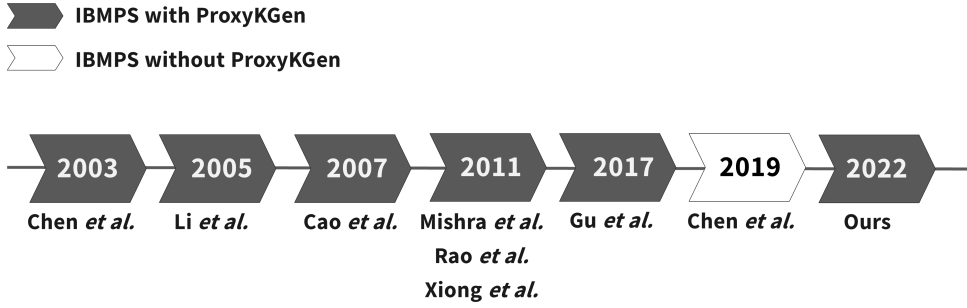


Figure 3.1: Chronological Development of IBMPS Schemes with Proxy Signing Keys

3. $\mathcal{A}_{\tilde{ps}}$ finally outputs a forged multi-proxy signature σ^* on a message m^* with a warrant w^* .

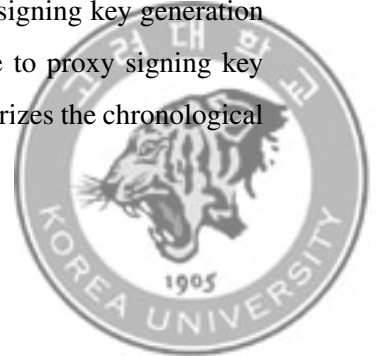
We say that $\mathcal{A}_{\tilde{ps}}$ wins the game if the following conditions are all satisfied.

- σ^* is a valid multi-proxy signature, i.e., $\text{MulProxyVer}(\mathbf{pp}, \{ID_j\}_{j \in M}, m^*, \sigma^*) = 1$.
- At least one proxy signer's identity $ID_i \in \{ID_1, ID_2, \dots, ID_n\}$ has not been queried in any KGen query.
- (w^*, m^*) has not been queried in any MulProxySign query.

If the probability of $\mathcal{A}_{\tilde{ps}}$ winning the game, denoted by $\epsilon_{\tilde{ps}}$, is *negligible*, then the IBMPS scheme is said to guarantee unforgeability against $\mathcal{A}_{\tilde{ps}}$, i.e., Type II security.

3.2 Previous Work

As discussed in Chapter 1, proxy signatures have been extended in various directions, including multi-proxy and on/offline signing. Accordingly, this section does not revisit the general development of proxy signatures. Instead, it focuses on representative identity-based multi-proxy signature (IBMPS) schemes that are directly relevant to the construction presented in this chapter. In particular, we examine whether each scheme defines an explicit proxy signing key generation procedure, denoted by ProxyKGen, and how its design affects resistance to proxy signing key exposure, formal security, and computational efficiency. Figure 3.1 summarizes the chronological development of the schemes considered in this comparison.



An explicit ProxyKGen procedure determines how a proxy signer derives its proxy signing key from the delegation, its long-term private key, and, where applicable, independently selected randomness. This procedure is particularly important when analyzing whether exposure of a proxy signing key may reveal the corresponding long-term private key. It also affects the distribution of computation between the original signer and the proxy signers.

Chen *et al.* [20] presented an early pairing-based IBMPS construction together with proxy multi-signature and multi-proxy multi-signature variants. Their work showed that identity-based key management could be incorporated into a multi-proxy signing framework. However, the scheme was not accompanied by a formal security proof. Moreover, as examined in Table 3.2, its proxy signing key can reveal the proxy signer's long-term private key when the proxy signing key and the corresponding delegation values are exposed.

Li and Chen [21] subsequently proposed another pairing-based IBMPS scheme. Although their construction was intended to improve the efficiency of identity-based multi-proxy signing, its ProxyKGen procedure requires more computation than that of the earlier construction, and its security was not established under a formal IBMPS model. Xu *et al.* [57] introduced a sequential IBMPS scheme in which an intermediate signature can be verified after each proxy signer completes its signing operation. This feature allows the signing process to terminate at an intermediate stage while retaining a verifiable result. However, the resulting signature size increases linearly with the number of participating proxy signers.

Cao and Cao [22] proposed one of the early IBMPS schemes analyzed under a formal security model. Xiong *et al.* [50] later identified a forgery vulnerability in the scheme and proposed a modification that applies domain-separation prefixes to distinguish ordinary messages from multi-proxy messages. Nevertheless, neither construction explicitly considered the exposure of proxy signing keys. IBMPS schemes proposed during the same period by Mishra *et al.* [48] and Rao and Reddy [49] were likewise not accompanied by complete proofs under a formal IBMPS security model.

Gu *et al.* [23] later proposed an IBMPS scheme with a security proof in the standard model. Their model considers the exposure of arbitrary proxy signing keys as well as self-proxy signing attacks. The scheme therefore provides stronger security treatment than many earlier construc-



tions. However, its signature size and verification cost increase with the number of proxy signers, and its final signing procedure does not include individual verification of each partial multi-proxy signature.

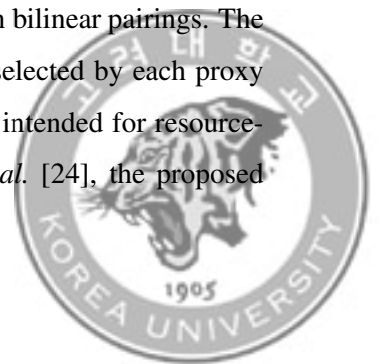
Chen *et al.* [24] subsequently presented an efficient IBMPS construction with a security proof in the random oracle model. Their scheme achieves low computation costs, but it does not define an explicit ProxyKGen procedure. Consequently, the scheme does not model proxy signing keys as separately generated credentials derived from a proxy signer's long-term private key and delegation information.

Shin *et al.* [58] presented a preliminary on/offline framework that applied the construction of Chen *et al.* [24] to an Internet of Drones environment. The framework primarily demonstrated how precomputation could reduce the computational work performed during flight. However, because the underlying construction does not include an explicit ProxyKGen procedure, the framework did not address the security of proxy signing key generation or exposure under a formal IBMPS model.

The preceding comparison reveals a trade-off among explicit proxy signing key generation, resistance to key exposure, provable security, and computational efficiency. Several early IBMPS schemes define a ProxyKGen procedure but do not provide formal security guarantees or resistance to proxy signing key exposure. Other constructions provide stronger security or improved efficiency but either incur costs that grow with the number of proxy signers or omit explicit proxy signing key generation. These limitations motivate the construction presented in this chapter, which aims to combine an explicit ProxyKGen procedure, resistance to proxy signing key exposure, provable security, and efficient signing operations suitable for mobile environments.

3.3 Construction

This section first describes the underlying IBMPS construction based on bilinear pairings. The proposed proxy signing key incorporates a random value independently selected by each proxy signer. We then extend the construction to an on/offline IBMPS protocol intended for resource-constrained mobile applications. In contrast to the scheme of Chen *et al.* [24], the proposed



construction includes an explicit proxy signing key generation procedure and prevents exposure of the resulting proxy signing key from directly revealing the corresponding long-term private key. Section 3.4 establishes the security of the construction.

3.3.1 Identity-Based Multi-Proxy Signature (IBMPS)

$\mathbf{pp} \xleftarrow{\$} \text{Setup}(\lambda)$. Given the security parameter λ , the KGC chooses two groups $\mathbb{G}_1$ and $\mathbb{G}_2$, where $\mathbb{G}_1$ is an additive group of prime order q with a generator P and $\mathbb{G}_2$ is a multiplicative group of the same prime order. The bilinear map is defined as $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$. The hash functions are chosen as $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, $H_2 : \{0, 1\}^* \times \mathbb{G}_1 \rightarrow \mathbb{G}_1$, and $H_3 : \{0, 1\}^* \times \mathbb{G}_1 \rightarrow \mathbb{G}_1$. The KGC randomly selects $s \in \mathbb{Z}_q^*$ as a master secret key and computes a system public key $P_{pub} = sP$. The KGC publishes the public parameters $\mathbf{pp} = (\mathbb{G}_1, \mathbb{G}_2, q, e, P, P_{pub}, H_1, H_2, H_3)$ and keeps s secret.

$(Q_{ID_j}, S_{ID_j}) \xleftarrow{\$} \text{KGen}(\mathbf{pp}, ID_j)$. On a user identity ID_j for $j \in M$, the KGC computes a public key Q_{ID_j} and a private key S_{ID_j} as follows:

$$\begin{aligned} Q_{ID_j} &= H_1(ID_j), \\ S_{ID_j} &= sQ_{ID_j}. \end{aligned}$$

The KGC sends the private keys to the users through a secure channel. The original signer U_0 holds the public key Q_{ID_0} and the associated private key S_{ID_0} . Similarly, each proxy signer U_i for $i \in N$ holds a key pair (Q_{ID_i}, S_{ID_i}) .

$\sigma_0 \xleftarrow{\$} \text{DelGen}(\mathbf{pp}, S_{ID_0}, \{ID_i\}_{i \in N}, w)$. To delegate the signing capability to the proxy group, the original signer U_0 generates a warrant w containing identity information for the original signer and the proxy group, a delegation period, a scope of the proxy authority, and the message types to be delegated.

1. U_0 randomly chooses $r_0 \in \mathbb{Z}_q^*$ and computes $R_0 = r_0P$.
2. U_0 computes $h_0 = H_2(w, R_0)$ and $V_0 = r_0h_0 + S_{ID_0}$.



3. For $i \in N$, U_0 sends $\sigma_0 = (R_0, V_0)$ and w to each proxy signer U_i through a secure channel.

Upon receiving (w, R_0, V_0) , each proxy signer U_i computes $h_0 = H_2(w, R_0)$ and verifies the following equation:

$$e(P, V_0) = e(P_{pub}, Q_{ID_0}) \cdot e(R_0, h_0). \quad (3.1)$$

If the equality holds, U_i accepts σ_0 as a valid delegation.

$S_{U_i} \xleftarrow{\$} \text{ProxyKGen}(w, \sigma_0, S_{ID_i})$. Each proxy signer U_i generates a proxy signing key using the delegation (R_0, V_0) and the private key S_{ID_i} as follows.

1. U_i selects a random secret $r_i \in \mathbb{Z}_q^*$, computes $R_i = r_i P$, and broadcasts R_i to the proxy group as a public proxy key.
2. U_i generates a proxy signing key $S_{U_i} = r_i h_0 + S_{ID_i} + V_0$ for $h_0 = H_2(w, R_0)$.
3. U_i publishes a public parameter list (R_i, R_0, w) for the proxy signing key S_{U_i} . Note that r_i is no longer needed for the verification process.

We design that only a proxy signer who obtains V_0 and S_{ID_i} is able to generate a valid multi-proxy signature.

$\sigma \xleftarrow{\$} \text{MulProxySign}(\mathbf{pp}, w, S_{U_i}, \sigma_0, m, \{\sigma_i\}_{i \in N})$. Given a message m to be signed, each proxy signer U_i for $i \in N$ proceeds as follows.

1. U_i randomly chooses $x_i \in \mathbb{Z}_q^*$, computes $X_i = x_i P$, and broadcasts X_i to the proxy group. Each proxy signer computes $X = \sum_{i=1}^n X_i$ and generates $V_i = x_i h + S_{U_i}$ for $h = H_3(m, X)$.
2. U_i sends the partial signature $\sigma_i = (w, R_0, R_i, X_i, V_i)$ to the clerk.
3. The clerk verifies the validity of σ_i using the following equation:

$$e(P, V_i) = e(P_{pub}, Q_{ID_0} + Q_{ID_i}) \cdot e(X_i, h) \cdot e(R_0 + R_i, h_0). \quad (3.2)$$



If the verification holds for all $i \in N$, the clerk computes $V = \sum_{i=1}^n V_i$ and $R = \sum_{i=1}^n R_i$, then outputs the final multi-proxy signature $\sigma = (w, R_0, R, X, V)$.

$\{0, 1\} \leftarrow \text{MulProxyVer}(\mathbf{pp}, \{ID_j\}_{j \in M}, m, \sigma)$. Upon receiving the multi-proxy signature $\sigma = (w, R_0, R, X, V)$ for a message m , any party verifies σ as follows.

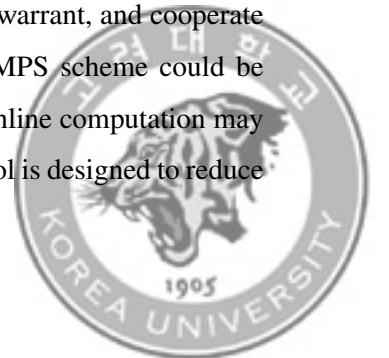
1. The verifier checks whether m conforms to the warrant w . If it does not conform, the verification fails.
2. The verifier checks whether all proxy signers $\{U_i\}_{i \in N}$ are correctly listed in w . If they are not listed in w , the verification fails.
3. The verifier accepts σ if and only if the following equation is satisfied:

$$e(P, V) = e\left(P_{pub}, nQ_{ID_0} + \sum_{i=1}^n Q_{ID_i}\right) \cdot e(X, h) \cdot e(R + nR_0, h_0). \quad (3.3)$$

3.3.2 On/Offline Identity-Based Multi-Proxy Signature (OIBMPS)

We next derive an identity-based on/offline multi-proxy signature (OIBMPS) protocol from the underlying IBMPS construction. As an illustrative mobile application, we consider an Internet of Drones (IoD) environment in which drone D_0 acts as the original signer and the authorized drones $\{D_i\}_{i \in N}$ act as proxy signers. These drones can jointly authenticate predefined operational or emergency messages on behalf of the original signer when direct participation by the original authority is unavailable.

As illustrated in Fig. 3.2, the protocol is divided into offline and online phases. Before flight, each proxy drone initializes its system parameters and public-private key pair with the KGC, receives a delegation from D_0 , and precomputes the components required for partial signing. The resulting values are stored for later use. During flight, the proxy drones communicate with one another, complete their partial signatures for a message predefined by the warrant, and cooperate in generating the final multi-proxy signature. Although an existing IBMPS scheme could be adapted to an IoD environment with minor procedural modifications, its online computation may remain burdensome for lightweight drones. The proposed OIBMPS protocol is designed to reduce



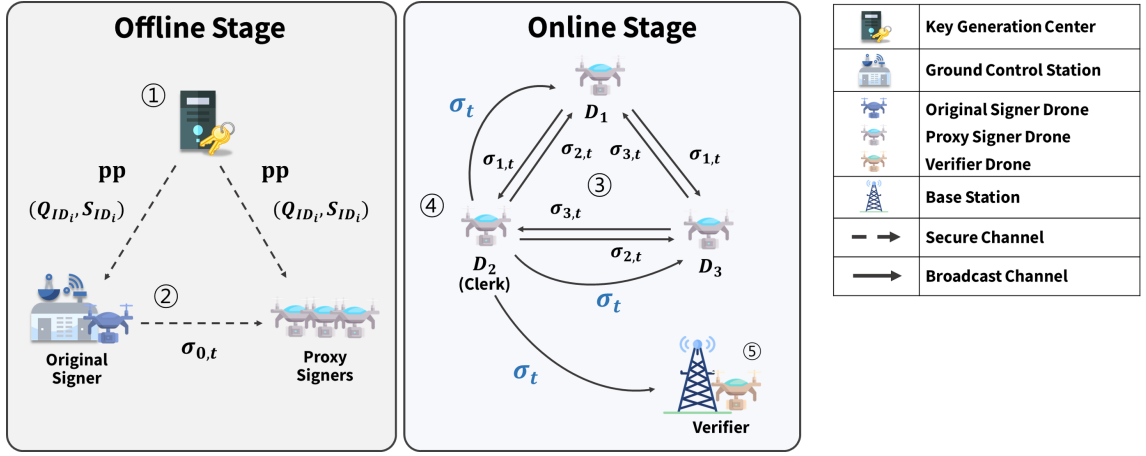


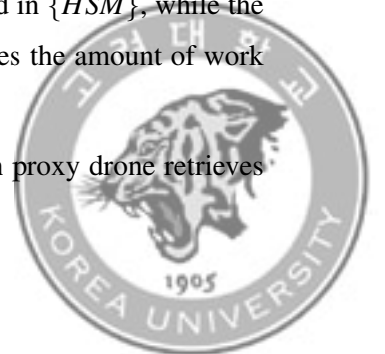
Figure 3.2: On/Offline IBMPS (OIBMPS)

this cost: once the offline values have been prepared, each proxy signer completes its online partial-signature computation using only one elliptic curve point addition. The security analysis of the underlying IBMPS scheme also covers the delegation and partial-signature queries required by the on/offline execution.

Figures 3.3 and 3.4 describe the operational procedures through which drones equipped with the proposed scheme perform the roles of proxy signers and jointly produce a final multi-proxy signature. The notation $\{Q_{ID_i}\}_{i=0}^n$ denotes the public keys associated with users ID_0 through ID_n . The index t denotes the t -th operational epoch. The notation $\{HSM\}$ represents protected hardware-backed storage used to retain secret signing components against device-capture attacks, whereas $\{VP\}$ denotes the collection of public values required for partial- and final-signature verification.

Figure 3.3 presents the pre-flight offline procedure. The original signer drone first transmits the delegation to the authorized proxy drones over a secure channel. After validating the delegation, each proxy drone executes the Preflight procedure to derive its proxy signing key $S_{U_{i,t}}$ and precompute the partial-signature component $V'_{i,t}$. Secret values are retained in $\{HSM\}$, while the public verification values are stored in $\{VP\}$. This precomputation reduces the amount of work required after the drones begin their mission.

Figure 3.4 describes the online procedure executed during flight. Each proxy drone retrieves



Protocol: Offline Protocol of On/Offline IBMPS (OIBMPS)

Input: $\mathbf{pp}$; $\{Q_{ID_i}\}_{i=0}^n$; $\{S_{ID_i}\}_{i=0}^n$; Message m ; Epoch t .

Output: Boolean.

Original Signer:

1. Runs DelGen($\mathbf{pp}, S_{ID_0}, \{ID_i\}_{i=1}^n, w$):
 - (a) Generates random $r_{0,t}$ for epoch t and computes $R_{0,t} = r_{0,t}P$.
 - (b) Computes the warrant hash $h_{0,t} = H_2(w, R_{0,t})$ and $V_{0,t} = r_{0,t}h_{0,t} + S_{ID_0}$.
 - (c) Sets $\sigma_{0,t} \leftarrow (R_{0,t}, V_{0,t})$ and sends $\sigma_{0,t}$ and w to $\{D_i\}_{i=1}^n$ through a secure channel.

Proxy Signer: $i = 1, \dots, n$:

1. Receives $\sigma_{0,t}$ and w through a secure channel and runs DelVer($\mathbf{pp}, \sigma_{0,t}, Q_{ID_0}, h_{0,t}$):
 - (a) Computes $e_1 = e(P, V_{0,t})$ and $e_2 = e(P_{pub}, Q_{ID_0}) \cdot e(R_{0,t}, h_{0,t})$.
 - (b) If $e_1 = e_2$, runs Preflight($\mathbf{pp}, S_{ID_i}, \sigma_{0,t}, w, m$). Otherwise, returns False.
 2. Runs Preflight($\mathbf{pp}, S_{ID_i}, \sigma_{0,t}, w, m$):
 - (a) Generates random $r_{i,t}$ for epoch t and computes $R_{i,t} = r_{i,t}P$.
 - (b) Computes the proxy signing key $S_{U_{i,t}} \leftarrow r_{i,t}h_{0,t} + S_{ID_i} + V_{0,t}$.
 - (c) Generates random $x_{i,t}$ for epoch t and computes $X_{i,t} = x_{i,t}P$.
 - (d) Stores $\{HSM\} \leftarrow S_{U_{i,t}}, r_{i,t}, x_{i,t}$.
 - (e) Sends $R_{i,t}$ and $X_{i,t}$ to the proxy group and receives $\{R_{i,t}\}_{i=1}^{n-1}$ and $\{X_{i,t}\}_{i=1}^{n-1}$ from $n-1$ proxy signers. Sets $Arr1[i] \leftarrow R_{i,t}$ and $Arr2[i] \leftarrow X_{i,t}$ for $i \in \{1, \dots, n\}$.
 - (f) If $|Arr1| = n$ and $|Arr2| = n$:
 - Computes $R_t = \sum_{i=1}^n R_{i,t}$, $X_t = \sum_{i=1}^n X_{i,t}$, and the message hash $h_t = H_3(m, X_t)$.
 - Generates the pre-partial multi-proxy signature $V'_{i,t} = x_{i,t}h_t$ and stores $\{HSM\} \leftarrow V'_{i,t}$.
 - Stores $\{VP\} \leftarrow (\mathbf{pp}, Q_{ID_0}, \{Q_{ID_i}\}, h_{0,t}, h_t, Arr1, R_t, Arr2, X_t)$ at storage and $\{HSM\}$ at secure storage.
 - Returns True.
- Otherwise, returns False.

Figure 3.3: Offline Protocol of OIBMPS



Protocol : Online Protocol of On/Offline IBMPS (OIBMPS)

Input : Epoch t .

Output : σ_t .

Proxy Signer: $i = 1, \dots, n$:

1. Fetches $\{VP\}$ and fetches $V'_{i,t}$ and $S_{U_{i,t}}$ from $\{HSM\}$.
2. Computes $V_{i,t} = V'_{i,t} + S_{U_{i,t}}$ (only one addition is needed).
3. Sets $\sigma_{i,t} \leftarrow (R_{i,t}, X_{i,t}, V_{i,t})$ and sends $\sigma_{i,t}$ through a broadcast channel.

Clerk:

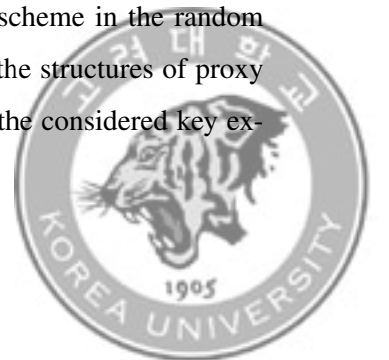
1. Receives $\sigma_{i,t}$ from $n-1$ proxy signers and runs $\text{PMPSVer}(\{VP\}, \sigma_{i,t})$ for $i \in \{1, \dots, n\}$:
 - (a) Computes $e_1 = e(P, V_{i,t})$ and $e_2 = e(P_{pub}, Q_{ID_0} + Q_{ID_i}) \cdot e(X_{i,t}, h_t) \cdot e(R_{0,t} + R_{i,t}, h_{0,t})$.
 - (b) If $e_1 = e_2$, sets $\text{Arr}[i] \leftarrow \sigma_{i,t}$. Otherwise, returns False.
2. Runs $\text{FMPSGen}(\{VP\}, \text{Arr})$:
 - (a) Aggregates $V = \sum_{i=1}^n V_{i,t}$ and $R = \sum_{i=1}^n R_{i,t}$.
 - (b) Returns the final multi-proxy signature $\sigma_t = (w, R_{0,t}, R, X_t, V)$.

Figure 3.4: Online Protocol of OIBMPS

$S_{U_{i,t}}$ and $V'_{i,t}$ from protected storage and obtains the corresponding public values from $\{VP\}$. It then computes $V_{i,t} = V'_{i,t} + S_{U_{i,t}}$, which requires one elliptic curve point addition, and forms the partial signature $\sigma_{i,t}$. After collecting the partial signatures, the clerk verifies each contribution and aggregates the valid values into the final multi-proxy signature σ_t .

3.4 Security Analysis

We first establish Type I and Type II unforgeability of the proposed scheme in the random oracle model [59] by reductions to the CDH problem. We then compare the structures of proxy signing keys in existing IBMPS schemes and examine their resistance to the considered key exposure attack.



Theorem 1. Suppose that a probabilistic polynomial-time adversary $\mathcal{A}_{\tilde{d}}$ issues at most q_{H_1} , q_{H_2} , q_{H_3} , q_K , q_D , q_P , and q_S queries to the hash functions (H_1, H_2, H_3) and the KGen, DelGen, ProxyKGen, and MulProxySign oracles, respectively, and runs in time at most $t_{\tilde{d}}$. If $\mathcal{A}_{\tilde{d}}$ breaks Type I security with non-negligible probability $\epsilon_{\tilde{d}}$, then one can construct a PPT algorithm $\mathcal{C}$ that solves the CDH problem with non-negligible probability ϵ_c satisfying

$$\epsilon_c \geq \epsilon_{\tilde{d}} \frac{1}{(q_K + q_D + n + 1)e}$$

within time t_c satisfying

$$t_c \leq t_{\tilde{d}} + (q_{H_1} + q_{H_2} + q_{H_3} + q_K + 3q_D + nq_P + 2nq_S + n + 3)t_s + t_i,$$

where t_s is the execution time of a scalar multiplication in $\mathbb{G}_1$, t_i is the time required for an inversion in $\mathbb{Z}_q^*$, and n is the number of proxy signers.

Proof. Let (P, aP, bP) be the CDH instance given to the challenger $\mathcal{C}$. The challenger uses $\mathcal{A}_{\tilde{d}}$ as a subroutine and simulates $\text{Game}_{\tilde{d}}$ in order to derive abP . The simulation proceeds as follows.

- **Setup phase.** $\mathcal{C}$ sets the system public key as $P_{\text{pub}} = aP$ and generates the public parameters $\mathbf{pp} = (\mathbb{G}_1, \mathbb{G}_2, q, e, P, P_{\text{pub}}, H_1, H_2, H_3)$. It forwards $\mathbf{pp}$ to $\mathcal{A}_{\tilde{d}}$.
- **Training phase.**
 - H_1 queries: Upon receiving an H_1 query on ID_i from $\mathcal{A}_{\tilde{d}}$ for $j \in M$, $\mathcal{C}$ responds with $Q_{ID_i} = H_1(ID_i)$ while maintaining H_1 -list $(ID_i, Q_{ID_i}, t_i, c_i)$, as follows.
 - (1) If ID_i exists in H_1 -list, $\mathcal{C}$ returns Q_{ID_i} from the list.
 - (2) Otherwise, $\mathcal{C}$ randomly chooses $t_i \in \mathbb{Z}_q^*$ and flips a coin $c_i \in \{0, 1\}$ with $\Pr[c_i = 0] = \delta$ and $\Pr[c_i = 1] = 1 - \delta$, where $0 < \delta < 1$. Then, $\mathcal{C}$ sets

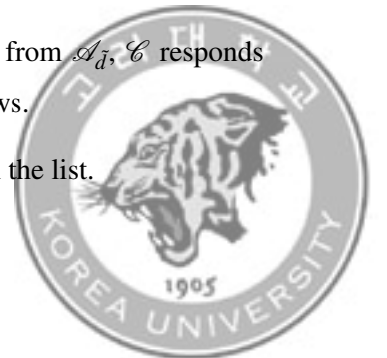
$$Q_{ID_i} = H_1(ID_i) = \begin{cases} t_i P, & \text{if } c_i = 0, \\ t_i (bP), & \text{if } c_i = 1. \end{cases}$$



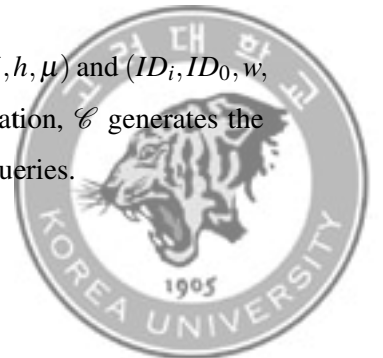
- adds $(ID_i, Q_{ID_i}, t_i, c_i)$ to H_1 -list, and returns Q_{ID_i} to $\mathcal{A}_{\tilde{d}}$.
- H_2 queries: Upon receiving an H_2 query on (w, R_0) from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with h_0 while maintaining H_2 -list (w, R_0, h_0, η) , as follows.
 - (1) If (w, R_0) exists in H_2 -list, $\mathcal{C}$ returns h_0 from the list.
 - (2) Otherwise, $\mathcal{C}$ randomly chooses $\eta \in \mathbb{Z}_q^*$, computes $h_0 = \eta P$, adds (w, R_0, h_0, η) to H_2 -list, and returns h_0 to $\mathcal{A}_{\tilde{d}}$.
 - H_3 queries: Upon receiving an H_3 query on (m, X) from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with h while maintaining H_3 -list (m, X, h, μ) , as follows.
 - (1) If (m, X) exists in H_3 -list, $\mathcal{C}$ returns h from the list.
 - (2) Otherwise, $\mathcal{C}$ randomly chooses $\mu \in \mathbb{Z}_q^*$, computes $h = \mu P$, adds (m, X, h, μ) to H_3 -list, and returns h to $\mathcal{A}_{\tilde{d}}$.
 - KGen queries: Upon receiving a KGen query on ID_i from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with S_{ID_i} while maintaining **UK**-list $(ID_i, S_{ID_i}, t_i, c_i)$, as follows.
 - (1) If ID_i exists in **UK**-list, $\mathcal{C}$ returns S_{ID_i} from the list.
 - (2) When $\mathcal{C}$ first searches with ID_i and it does not exist in H_1 -list, $\mathcal{C}$ creates the tuple $(ID_i, Q_{ID_i}, t_i, c_i)$ as defined in (2) of H_1 queries.
 - a. If $c_i = 0$, $\mathcal{C}$ computes $S_{ID_i} = t_i(aP)$ and adds it to **UK**-list.
 - b. Otherwise if $c_i = 1$, including the case that c_0 of the original signer ID_0 is 1, the simulation halts.

$$S_{ID_i} = aQ_{ID_i} = \begin{cases} at_iP, & \text{if } c_i = 0, \\ \perp, & \text{if } c_i = 1. \end{cases}$$

- (3) Finally, $\mathcal{C}$ returns S_{ID_i} to $\mathcal{A}_{\tilde{d}}$.
- DelGen queries: Upon receiving a DelGen query on (ID_0, w) from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with σ_0 while maintaining **DG**-list (ID_0, w, R_0, V_0, r_0) , as follows.
 - (1) If (ID_0, w) exists in **DG**-list, $\mathcal{C}$ returns $\sigma_0 = (R_0, V_0)$ from the list.



- (2) When $\mathcal{C}$ searches with ID_0 and there is no record, it gets $(ID_0, Q_{ID_0}, t_0, c_0)$ by following step (2) in KGen queries. If $c_0 = 0$, $\mathcal{C}$ randomly chooses $r_0 \in \mathbb{Z}_q^*$ and computes $R_0 = r_0P$.
 - (3) $\mathcal{C}$ searches (w, R_0) in H_2 -list and obtains h_0 for η . Then $\mathcal{C}$ computes $V_0 = r_0\eta P + t_0(aP)$ by retrieving S_{ID_0} from **UK**-list. Otherwise if c_i of ID_i in H_1 -list is 1, the simulation halts. Finally, $\mathcal{C}$ returns (R_0, V_0) to $\mathcal{A}_{\tilde{d}}$.
- ProxyKGen queries: Upon receiving a ProxyKGen query on (ID_i, ID_0, w) from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with S_{U_i} while maintaining **PKG**-list $(ID_i, ID_0, w, S_{U_i}, r_i)$, as follows:
- (1) If (ID_i, ID_0, w) exists in **PKG**-list, $\mathcal{C}$ returns S_{U_i} from the list.
 - (2) When $\mathcal{C}$ searches with ID_i and it does not exist in H_1 -list, $\mathcal{C}$ follows step (2) of H_1 queries to generate the tuple $(ID_i, Q_{ID_i}, t_i, c_i)$. If $c_0 = 0$, $\mathcal{C}$ searches (w, R_0) for h_0 in H_2 -list. If it does not exist, $\mathcal{C}$ creates a new list (w, R_0, h_0, η) as defined in step (2) of H_2 queries and obtains h_0 for η .
 - (3) If $c_i = 0$ for $i \in N$, $\mathcal{C}$ makes $R_i = r_iP$ for a random value $r_i \in \mathbb{Z}_q^*$ and asks KGen queries on ID_i to obtain S_{ID_i} .
 - (4) $\mathcal{C}$ computes $S_{U_i} = r_i\eta P + t_i(aP) + r_0\eta P + t_0(aP)$ and returns $(ID_i, ID_0, w, S_{U_i}, r_i)$ to $\mathcal{A}_{\tilde{d}}$. Otherwise if $c_i = 1$, the simulation halts.
- MulProxySign queries: Upon receiving a MulProxySign query on $(ID_0, \{ID_i\}_{i \in N}, w, m)$ from $\mathcal{A}_{\tilde{d}}$, $\mathcal{C}$ responds with σ as follows:
- (1) For $j \in M$, $\mathcal{C}$ searches with ID_0 or $\{ID_i\}_{i \in N}$ in H_1 -list to obtain the tuple $(ID_i, Q_{ID_i}, t_i, c_i)$ through steps (1) and (2) of H_1 queries. If $c_0 = 1$ or $c_i = 1$ for any $i \in N$, the simulation halts.
 - (2) Otherwise, $\mathcal{C}$ picks a random value $x_i \in \mathbb{Z}_q^*$ for each $i \in N$ and computes $X = \sum_{i=1}^n X_i$ where $X_i = x_iP$.
 - (3) $\mathcal{C}$ searches H_3 -list and **PKG**-list to recover the tuples (m, X, h, μ) and $(ID_i, ID_0, w, S_{U_i}, r_i)$, respectively. If there is no corresponding information, $\mathcal{C}$ generates the values by following KGen, H_2 , DelGen, and ProxyKGen queries.



(4) Using S_{U_i} , $\mathcal{C}$ computes a partial multi-proxy signature V_i as follows:

$$\begin{aligned} S_{U_i} &= r_i h_0 + S_{ID_i} + V_0 \\ &= r_i \eta P + t_i(aP) + r_0 \eta P + t_0(aP), \\ V_i &= x_i h + S_{U_i} \\ &= x_i \mu P + r_i \eta P + t_i(aP) + r_0 \eta P + t_0(aP). \end{aligned}$$

Then, $\mathcal{C}$ aggregates $V = \sum_{i=1}^n V_i$ and returns $\sigma = (w, R_0, R, X, V)$ to $\mathcal{A}_{\tilde{d}}$.

- **Forgery.** At the end of the simulation, $\mathcal{A}_{\tilde{d}}$ outputs a forged multi-proxy signature $\sigma^* = (w^*, R_0, R, X, V)$ on message m^* with warrant w^* that has not been queried in any MulProxy-Sign query. By the assumption that $\mathcal{A}_{\tilde{d}}$ has recovered the tuple $(ID_i, Q_{ID_i}, t_i, c_i)$ from H_1 -list for $j \in M$, the process goes as follows.

- (1) To ensure that w^* has not been queried in any DelGen query, the simulation fails if $c_0 = 0$. Likewise, if $c_i = 1$ for any $i \in N$, the simulation fails and halts.
- (2) Otherwise, if $c_0 = 1$ and $c_i = 0$ for all $i \in N$, $\mathcal{C}$ searches for the tuples (w, R_0, h_0, η) and (m, X, h, μ) in H_2 -list and H_3 -list, respectively.
- (3) The output of $\mathcal{C}$ is as follows:

$$\begin{aligned} e(P, V) &= e\left(P_{pub}, nQ_{ID_0} + \sum_{i=1}^n Q_{ID_i}\right) \cdot e(nR_0 + R, h_0) \cdot e(X, h) \\ &= e\left(aP, nt_0bP + \sum_{i=1}^n t_iP\right) \cdot e(nR_0 + R, \eta P) \cdot e(X, \mu P) \\ &= e\left(P, nt_0abP + \sum_{i=1}^n t_i aP + n\eta R_0 + \eta R + \mu X\right). \end{aligned}$$

Finally, $\mathcal{C}$ solves abP by computing:

$$abP = (nt_0)^{-1} \cdot \left(V - \sum_{i=1}^n t_i aP - \eta(nR_0 + R) - \mu X \right).$$



We next analyze the success probability during the queries of $\mathcal{A}_{\tilde{d}}$ in the above simulation. To solve the CDH problem, the original signer's delegation must not be queried. In other words, the coin c_0 of ID_0 must not be 0 while the private keys of other proxy signers are allowed to be accessed. Therefore, its probability is at least

$$\Pr[c_0 = 1 \wedge c_i = 0] = (1 - \delta)\delta^n, \quad (3.4)$$

where $0 \leq \delta \leq 1$.

In KGen queries, since $c_i = 1$ causes a halt, it succeeds with at least $\Pr[c_i = 0] = \delta^{q_K}$. In DelGen queries, since $c_0 = 1$ causes a halt, it succeeds with at least $\Pr[c_0 = 0] = \delta^{q_D}$. In ProxyKGen queries and MulProxySign queries, the probability for $\mathcal{C}$ to not abort is 1. When the probability of $\mathcal{A}_{\tilde{d}}$ forging a valid multi-proxy signature is $\varepsilon_{\tilde{d}}$, the success probability $\Pr[\text{Succ}]$ that $\mathcal{C}$ outputs abP is at least

$$\Pr[\text{Succ}] = \varepsilon_{\tilde{d}}(1 - \delta)\delta^{n+q_K+q_D}. \quad (3.5)$$

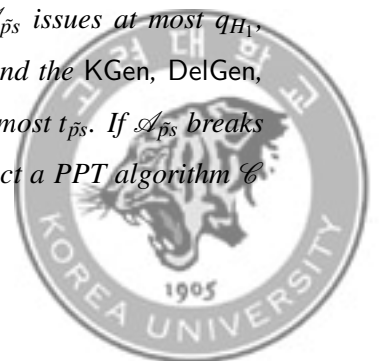
By using the mathematical constant e [60], we set $\delta = \frac{q_K+q_D+n}{q_K+q_D+n+1}$ and obtain the following lower bound: $\varepsilon_c \geq \varepsilon_{\tilde{d}} \frac{1}{(q_K+q_D+n+1)^e}$.

The total running time of $\mathcal{C}$, measured in terms of scalar multiplications in $\mathbb{G}_1$ and inversions in $\mathbb{Z}_q^*$, is at most

$$t_c \geq t_{\tilde{d}} + (q_{H_1} + q_{H_2} + q_{H_3} + q_K + 3q_D + nq_P + 2nq_S + n + 3)t_s + t_i.$$

□

Theorem 2. Suppose that a probabilistic polynomial-time adversary $\mathcal{A}_{\tilde{p}s}$ issues at most q_{H_1} , q_{H_2} , q_{H_3} , q_K , q_D , q_P , and q_S queries to the hash functions (H_1, H_2, H_3) and the KGen, DelGen, ProxyKGen, and MulProxySign oracles, respectively, and runs in time at most $t_{\tilde{p}s}$. If $\mathcal{A}_{\tilde{p}s}$ breaks Type II security with non-negligible probability $\varepsilon_{\tilde{p}s}$, then one can construct a PPT algorithm $\mathcal{C}$



that solves the CDH problem with non-negligible probability ϵ_c satisfying

$$\epsilon_c \geq \epsilon_{\tilde{p}s} \frac{1}{(q_K + q_D + n + 1)e}$$

within time t_c satisfying

$$t_c \geq t_{\tilde{p}s} + (q_{H_1} + q_{H_2} + q_{H_3} + q_K + 3q_D + nq_P + 2nq_S + n + 3)t_s + t_i,$$

where e denotes the base of the natural logarithm, t_s is the execution time of a scalar multiplication in $\mathbb{G}_1$, t_i is the time required for an inversion in $\mathbb{Z}_q^*$, and n is the number of proxy signers.

Proof. Let (P, aP, bP) be the CDH instance given to $\mathcal{C}$. The simulation follows the same overall structure as that used in the proof of Theorem 1. Therefore, we omit the repeated oracle simulations and describe only the distinct extraction step.

- **Output.** At the end of the simulation, $\mathcal{A}_{\tilde{p}s}$ outputs a forged multi-proxy signature $\sigma^* = (w^*, R_0, R, X, V)$. By the assumption that $\mathcal{A}_{\tilde{p}s}$ has queried Q_{ID_i} via the H_1 oracle, the process goes as follows.

- (1) If $c_0 = 1$ or $c_i = 0$ for all $i \in N$, the simulation fails and halts.
- (2) Otherwise, if $c_0 = 0$ and at least one $c_i = 1$ for $i \in N$, indicating that queries for $n - 1$ proxy signers are possible, $\mathcal{C}$ searches for the tuples (w, R_0, h_0, η) and (m, X, h, μ) in H_2 -list and H_3 -list, respectively.
- (3) Without loss of generality, we assume $c_n = 1$. The output of $\mathcal{C}$ is as follows:

$$\begin{aligned} e(P, V) &= e\left(P_{pub}, nQ_{ID_0} + \sum_{i=1}^n Q_{ID_i}\right) \cdot e(nR_0 + R, h_0) \cdot e(X, h) \\ &= e\left(aP, nt_0P + \sum_{i=1}^{n-1} t_iP + t_nbP\right) \cdot e(nR_0 + R, \eta P) \cdot e(X, \mu P) \\ &= e\left(P, nt_0aP + \sum_{i=1}^{n-1} t_iaP + t_nabP + \eta(nR_0 + R) + \mu X\right). \end{aligned}$$



Combining the above relations, $\mathcal{C}$ finally computes:

$$abP = t_n^{-1} \left(V - nt_0aP - \sum_{i=1}^{n-1} t_iaP - \eta(nR_0 + R) - \mu X \right).$$

Considering that one of the proxy signer's private keys must not be queried, the probability is at least

$$\Pr[c_0 = 0 \wedge c_j = 0 \wedge c_n = 1] = \delta^n(1 - \delta), \quad (3.6)$$

where $1 \leq j \leq n - 1$ and $0 \leq \delta \leq 1$.

By analyzing the success probability similarly to the proof of Theorem 1, the probability that $\mathcal{C}$ succeeds in solving the CDH problem is at least

$$\Pr[\text{Succ}] = \epsilon_{\tilde{p}s}(1 - \delta)\delta^{n+q_K+q_D}.$$

Therefore, the resulting lower bound on the success probability of $\mathcal{C}$ is $\epsilon_c \geq \epsilon_{\tilde{p}s} \frac{1}{(q_K+q_D+n+1)^e}$.

The total running time of $\mathcal{C}$ is at most

$$t_c \geq t_{\tilde{p}s} + (q_{H_1} + q_{H_2} + q_{H_3} + q_K + 3q_D + nq_P + 2nq_S + n + 3)t_s + t_i,$$

required to transform $\mathcal{A}_{\tilde{p}s}$'s forgery into the CDH solution. □

Structural Analysis of Proxy Signing Keys in Existing Schemes. We next examine the algebraic structures of proxy signing keys used in representative IBMPS schemes. Table 3.2 shows whether an insider that obtains a proxy signing key together with publicly available delegation values can recover the proxy signer's long-term private key. The comparison indicates that a substantial number of previous constructions [20, 21, 57, 22, 48, 49, 50, 23, 24] are vulnerable to this form of proxy signing key exposure [51, 52].

To facilitate comparison across schemes, we express their key structures using a common notation. The symbol S_{U_i} denotes the proxy signing key of U_i , whereas Q_{ID_i} and S_{ID_i} denote its public and private keys, respectively. The values R_0 , V_0 , and $\{T_{i,0}, T_{i,1}, T_{i,2}\}$ represent delegation

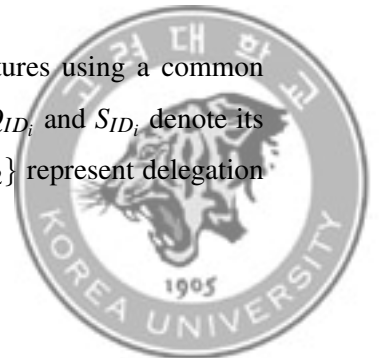
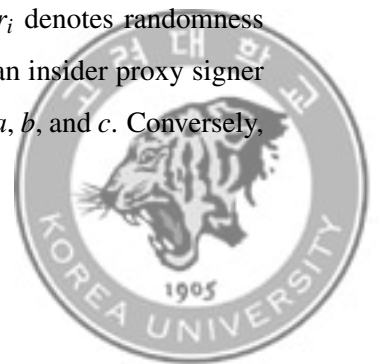


Table 3.2: Structure of Existing Proxy Signing Keys and Its Analysis

Scheme	Proxy Signing Key	Analysis
Chen <i>et al.</i> [20]	$S_{U_i} = R_0 S_{ID_i} + V_0$ $R_0 = H_2(w, r_A)$, where $r_A = e(P, P)^{r_0}$, $V_0 = R_0 S_{ID_0} + r_0 P$	$\{R_0, V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Li <i>et al.</i> [21]	$S_{U_i} = \{d_{P_i}, Q_{P_i}\} = \{V_0 + H_2(w, R_0) S_{ID_i}, H_1(w, R_0)(S_{ID_0} + S_{ID_i}) + R_0\}$ $R_0 = r_0 P$, $V_0 = H_1(w, R_0) S_{ID_0} + r_0 P_{pub}$	$\{R_0, V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Xu <i>et al.</i> [57] ¹	$S_{U_i} = \{X'_{ID_i}, S_i, h_i\} = \{r_i Q_{ID_0}, (r_i - h_i) S_{ID_0}, H_2(w_i, T_i, X'_{ID_i})\}$ $T_i = e(X'_{ID_i}, P_{pub})$, where $X'_{ID_i} = r_i Q_{ID_i}$	$\{S_i, h_i, S_{U_i}\}_{ID_j} \nrightarrow S_{ID_i}$
Cao <i>et al.</i> [22]	$S_{U_i} = V_0 + H_2(ID_0, w, R_0) S_{ID_i}$ $R_0 = r_0 P$, $V_0 = S_{ID_0} + r_0 H_1(ID_0, w, R_0)$	$\{R_0, V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Mishra <i>et al.</i> [48]	$S_{U_i} = V_0 + S_{ID_i}$ $R_0 = g^{r_0}$, $V_0 = (r_0 + H_2(w)) S_{ID_0}$	$\{V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Rao <i>et al.</i> [49]	$S_{U_i} = R_0 S_{ID_i} + V_0$ $R_0 = H_3(U_0, h_0)$, where $U_0 = e(w, P) \cdot e(Q_{ID_0}, P_{pub})^{-V_0}$, $h_0 = H_1(ID_0, w, U_0)$ $V_0 = R_0 S_{ID_0} + r_0 P$	$\{R_0, V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Xiong <i>et al.</i> [50]	$S_{U_i} = V_0 + H_2(ID_0, w, R_0) S_{ID_i}$ $R_0 = r_0 P$, $V_0 = S_{ID_0} + r_0 H_1(00, ID_0, w, R_0)$	$\{R_0, V_0, S_{U_i}\}_{ID_j} \rightarrow S_{ID_i}$
Gu <i>et al.</i> [23]	$S_{U_i} = \{y_{i,0}, y_{i,1}, y_{i,2}, y_{i,3}, y_{i,4}\} = \{S_{ID_{i,0}} T_{i,0}, S_{ID_{i,1}}, T_{i,1}, T_{i,2}\}$ $T_{i,0} = \left(S_{ID_{0,0}}^{\frac{1}{r_i}} \cdot v^{r_i H_2(w_i)} \cdot \kappa^{r_i} \right)$, where v, κ are public parameters $T_{i,1} = S_{ID_{0,1}}$, $T_{i,2} = g^{r_i}$	$\{T_{i,0}, T_{i,1}, S_{U_i}\}_{ID_j} \nrightarrow S_{ID_i}^2$
Chen <i>et al.</i> [24]	N/A (No proxy signing key)	N/A (No proxy signing key)
Our Scheme	$S_{U_i} = r_i h_0 + S_{ID_i} + V_0$ $R_0 = r_0 P$, $V_0 = r_0 h_0 + S_{ID_0}$, where $h_0 = H_2(w, R_0)$	$\{h_0, V_0, S_{U_i}\}_{ID_j} \nrightarrow S_{ID_i}$

¹ Unlike the other IBMPS schemes, the original signer computes and sends each proxy signer's proxy signing key directly to each proxy signer. ² In the scheme of Gu *et al.*, each user has a pair of secret keys $S_{ID_i} = \{S_{ID_{i,0}}, S_{ID_{i,1}}\}$ and no public key.

components generated by the original signer. For the hash functions, we use the unified notation $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$, $H_2 : \{0, 1\}^* \rightarrow \mathbb{Z}_q^*$, $H_3 : \{0, 1\}^* \times \mathbb{G}_2 \rightarrow \mathbb{Z}_q^*$, and $H_4 : \{0, 1\}^* \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$. The value r_0 denotes randomness selected by the original signer, while r_i denotes randomness selected for proxy signer U_i . The notation $\{a, b, c\}_{ID_j} \rightarrow d$ indicates that an insider proxy signer associated with ID_j can derive the secret value d from the available values a, b , and c . Conversely, $\nrightarrow$ indicates that the analyzed algebraic relation does not directly reveal d .



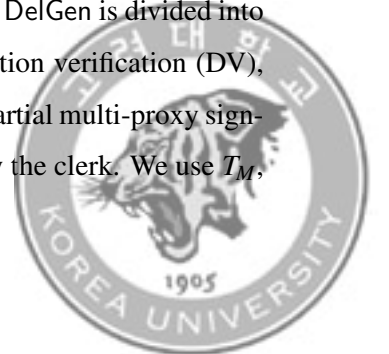
The comparison in Table 3.2 identifies three constructions for which the analyzed exposure does not directly reveal a proxy signer’s private key: the schemes of Xu *et al.* [57] and Gu *et al.* [23], and the proposed scheme. These cases, however, achieve this property through different structures. In the construction of Xu *et al.*, the original signer generates and distributes the proxy signing keys without using the proxy signers’ long-term private keys. Consequently, the considered algebraic recovery attack is not directly applicable to that construction. Nevertheless, the scheme was not accompanied by a formal proof under the security model considered here.

In the scheme of Gu *et al.* [23], the proxy signing key is represented by the five components $\{y_{i,0}, y_{i,1}, y_{i,2}, y_{i,3}, y_{i,4}\}$. Its construction incorporates signer-specific randomness through the delegation-related value $T_{i,2}$. As a result, disclosure of the proxy signing key does not directly provide the insider with the proxy signer’s long-term private key. The proposed scheme adopts a related principle by allowing each proxy signer to independently select r_i during proxy signing key generation. This random contribution prevents the exposed key and delegation values from being directly rearranged to recover S_{ID_i} . Among the compared schemes in which a proxy signer derives its own proxy signing key using its long-term private key, the construction of Gu *et al.* and the proposed scheme resist the analyzed exposure attack.

3.5 Performance Analysis

3.5.1 Efficiency Analysis

We evaluate efficiency by comparing the operations performed by the original signer, individual proxy signers, and the clerk during each phase of an IBMPS scheme. Table 3.3 summarizes the resulting computational costs and relevant functional properties of the compared constructions. Because the schemes commonly rely on bilinear pairings, the comparison focuses on the dominant group operations. To distinguish the costs incurred by different participants, we decompose selected algorithms into participant-specific procedures. In particular, DelGen is divided into delegation generation (DG), performed by the original signer, and delegation verification (DV), performed by each proxy signer. Likewise, MulProxySign is divided into partial multi-proxy signing (PMPS) by the proxy signers and final multi-proxy signing (FMPS) by the clerk. We use T_M ,



T_E , and T_P to denote the costs of a scalar multiplication in $\mathbb{G}_1$, an exponentiation in $\mathbb{G}_2$, and one bilinear pairing evaluation, respectively, and n denotes the number of proxy signers. Hash computations are omitted from the symbolic comparison because their measured costs are substantially smaller than those of the dominant group operations in the implementation considered here.

The proposed IBMPS scheme combines a formal security proof with competitive computation costs among schemes that include an explicit ProxyKGen model. Although the construction of Chen *et al.* [24] is slightly more efficient for selected operations, it does not define a separate proxy signing key generation procedure and therefore represents a different design setting.

Among the schemes with formal security analyses and an explicit ProxyKGen procedure, the constructions of Cao *et al.* [22] and Gu *et al.* [23] provide the most relevant comparisons. The scheme of Cao *et al.* was among the early IBMPS constructions defined under a formal security model, although a subsequent study identified a forgery vulnerability. From an efficiency perspective, the proposed scheme requires $nT_P + nT_E$ fewer operations during final multi-proxy signing and one fewer scalar multiplication during final verification than the scheme of Cao *et al.*

The construction of Gu *et al.* is notable because its security is proven in the standard model and its clerk-side aggregation cost is relatively low. However, it does not include a procedure for verifying individual partial signatures before aggregation. Consequently, when final verification fails, the verifier cannot determine which proxy signer contributed an invalid partial signature. The proposed scheme retains partial-signature verification while maintaining efficient final aggregation. All compared IBMPS schemes except those of Gu *et al.* and Xu *et al.* [57] include partial-signature verification. For the scheme of Xu *et al.*, the cost of proxy signing key generation is included in DelGen because the original signer generates and distributes each proxy signing key.

3.5.2 Implementation Analysis for Mobile Applications

The implementation evaluation measures how the execution times of the complete and participant-specific procedures change as the proxy group grows, as reported in Fig. 3.5. We additionally compare communication overhead, estimated computation-related energy consumption, and key-storage requirements in Table 3.4. To approximate an IoD deployment, the compared schemes

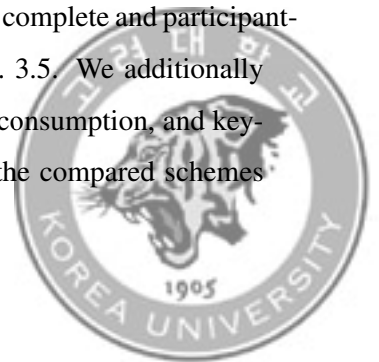


Table 3.3: Efficiency Comparison of IBMPs Schemes

Scheme	ProxyKGen ¹	ATT ²	DelGen ³	DV ⁴	ProxyKGen Cost ⁵	Com. Cost ⁶	PMPS ⁷	PMPV ⁸	FMPS ⁹	MulProxyVer ¹⁰	PS ¹¹
Chen <i>et al.</i> [20]	✓	✓	$2T_M + T_P + T_E$	$2nT_P + nT_E$	nT_M	2	$2nT_M + nT_P + nT_E$	✓	$2nT_P + 2nT_E$	$3T_P + 3T_E$	✗
Li <i>et al.</i> [21]	✓	✓	$3T_M$	$3nT_P + nT_E$	$2nT_M$	2	$3nT_M$	✓	$3nT_P + nT_E$	$nT_M + 3T_P + T_E$	✗
Xu <i>et al.</i> [57]	✓	✓	$3nT_M + nT_P$	$nT_M + 3nT_P$	N/A ¹²	2	$2nT_M$	✗	N/A	$2T_M + 3T_P$	✗
Cao <i>et al.</i> [22]	✓	✓	$2T_M$	$3nT_P$	nT_M	2	$2nT_M$	✓	$5nT_P + nT_E$	$3T_M + 4T_P$	✓
Mishra <i>et al.</i> [48]	✓	✓	$T_M + T_E$	$nT_P + 2nT_E$	— ¹³	2	$2nT_M + nT_E$	✓	$nT_P + 2nT_E$	$T_P + 2T_E$	✗
Rao <i>et al.</i> [49]	✓	✓	$2T_M + T_P + T_E$	$2nT_P + nT_E$	nT_M	2	$3nT_M + 2nT_P + nT_E$	✓	$2nT_P + 2nT_E$	$3T_P + 3T_E$	✗
Xiong <i>et al.</i> [50]	✓	✓	$2T_M$	$3nT_P$	nT_M	2	$2nT_M$	✓	$5nT_P + nT_E$	$3T_M + 4T_P$	✗
Gu <i>et al.</i> [23]	✓	✓	$4nT_E$	$4nT_P + 4nT_E$	nT_M	2	$2nT_M + 3nT_E$	✗	$3nT_M$	$2T_M + 6T_P + (n+3)T_E$	✓
Chen <i>et al.</i> [24]	✗	✓	$2T_M$	$3nT_P$	N/A	2	$3nT_M$	✓	$nT_M + 3nT_P$	$3T_M + 3T_P$	✓
Our Scheme	✓	✗	$2T_M$	$3nT_P$	$2nT_M$	2	$2nT_M$	✓	$4nT_P$	$2T_M + 4T_P$	✓

¹ ProxyKGen: whether the scheme employs a ProxyKGen model.

² ATT: whether the ProxyKGen model in the scheme is susceptible to a proxy signing key exposure attack.

³ DelGen: computation cost of DelGen by the original signer.

⁴ DV: computation cost of delegation verification by the proxy signer.

⁵ ProxyKGen Cost: computation cost of ProxyKGen by the proxy signer.

⁶ Com. Cost: communication cost required by a proxy signer in the MulProxySign phase.

⁷ PMPS: computation cost of partial multi-proxy signing by the proxy signer.

⁸ PMPV: whether the scheme includes verification of each partial multi-proxy signature.

⁹ FMPS: computation cost of final multi-proxy signing by the clerk.

¹⁰ MulProxyVer: computation cost of MulProxyVer by any verifier.

¹¹ PS: whether the scheme is proposed with provable security.

¹² N/A: the scheme of Xu *et al.* generates each proxy signing key at the original signer side.

¹³ —, the cost in terms of T_M , T_P , and T_E is not applicable.

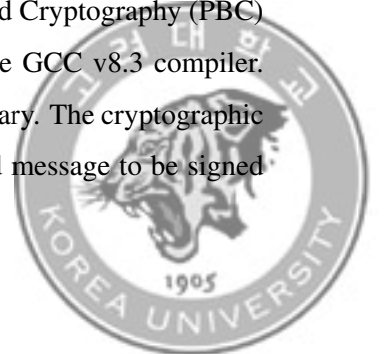


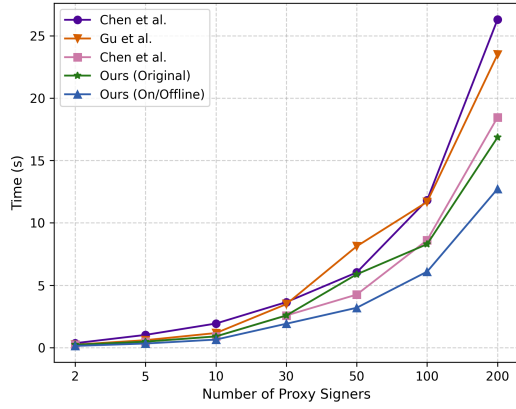
Table 3.4: Comparison of Communication Overhead and Energy Consumption

Scheme	Phase	Bandwidth (Byte)	CPU Time (ms)	Energy (mJ)	Public Key Size (Byte)	Private Key Size (Byte)
Chen <i>et al.</i> [20]	MulProxySign	256	0.350	173.407	128	128
	MulProxyVer	256	0.064	31.895		
Gu <i>et al.</i> [23]	MulProxySign	384	0.233	115.472	256	—
	MulProxyVer	768	0.198	97.899		
Chen <i>et al.</i> [24]	MulProxySign	256	0.219	108.542	128	128
	MulProxyVer	256	0.063	31.201		
IBMPS	MulProxySign	384	0.223	110.541	128	128
	MulProxyVer	384	0.063	31.114		
OIBMPS	MulProxySign	384	0.132	65.377	128	128
	MulProxyVer	384	0.063	31.068		

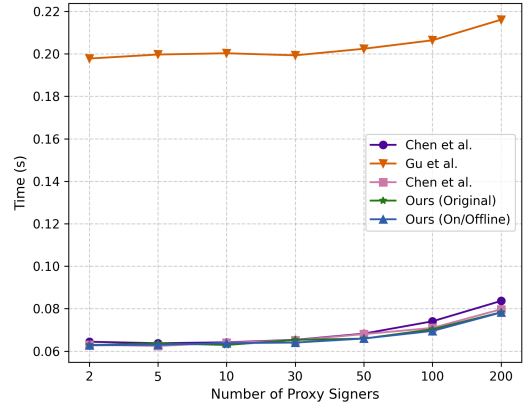
are implemented on a Raspberry Pi-based drone. The three baselines represent distinct stages in IBMPS development: Chen *et al.* [20] as an early IBMPS construction, Gu *et al.* [23] as a scheme proven in the standard model, and Chen *et al.* [24] as a recent efficient construction. Although the first scheme is not accompanied by a formal security proof, it is retained as a historical reference point.

In the experiment, we use a Raspberry Pi Zero W (Rev 1.1)-based drone, Air Copter [61]. The quadcopter weighs 100 g with a size of 22.5 cm and shows a maximum flight time of 5 to 7 minutes. The system configuration is: ARM v6 1 GHz Single Core, 512 MB RAM, and Raspbian v10. The protocols are implemented in C using the Pairing-Based Cryptography (PBC) library and the GNU Multiple Precision Arithmetic (GMP) library on the GCC v8.3 compiler. For pairings and parameters, we utilize Type-A internals from the PBC library. The cryptographic hash function is instantiated with SHA-256. The sizes of the warrant and message to be signed

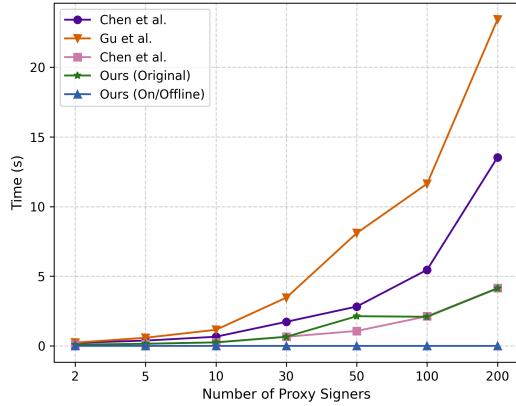




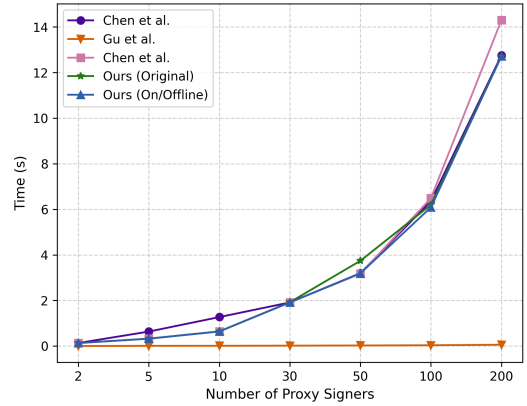
(a) Time Difference in MulProxySign



(b) Time Difference in MulProxyVer



(c) Time Difference in PMulProxySign

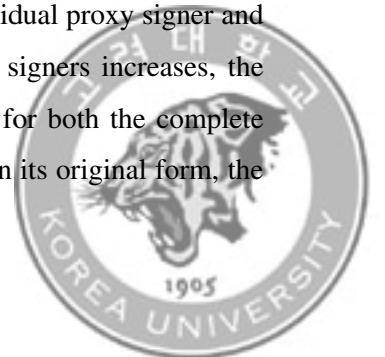


(d) Time Difference in FMulProxySign

Figure 3.5: Implementation Analysis Results of IBMPS and OIBMPS

are both set to 7 bytes.

Figure 3.5 presents the measured computation times as the number of proxy signers increases. For the IoD-oriented evaluation, the MulProxySign phase is decomposed into PMPS and FMPS, where PMPS denotes partial multi-proxy signature generation by an individual proxy signer and FMPS denotes final aggregation by the clerk. As the number of proxy signers increases, the proposed IBMPS scheme exhibits the slowest growth in execution time for both the complete MulProxySign procedure in Fig. 3.5(a) and MulProxyVer in Fig. 3.5(b). In its original form, the



proposed scheme has a MulProxySign cost comparable to that of Chen *et al.* [24], while requiring less time for FMPS as shown in Fig. 3.5(d).

The advantage of the on/offline extension is most evident in the partial-signature results shown in Fig. 3.5(c). By reusing the values generated before flight, a proxy signer completes the on-line computation with a single elliptic curve point addition. The resulting execution time grows only marginally as the proxy group expands, supporting the applicability of the protocol to time-sensitive IoD operations.

The clerk-side cost of the scheme of Gu *et al.* [23] remains lower because the clerk aggregates the received contributions without individually verifying each partial signature. This design reduces the measured FMPS execution time, but it also limits fault identification. If final verification fails after values have been transmitted over a broadcast channel, the scheme provides no direct means of identifying the invalid contribution. By contrast, the proposed scheme verifies each partial signature before aggregation while retaining a moderate growth rate in clerk-side computation.

Table 3.4 reports communication overhead and estimated energy consumption when two proxy signers jointly generate a multi-proxy signature. Communication overhead is measured as the total number of bytes transmitted through broadcast messages during the MulProxySign and MulProxyVer procedures. In the proposed scheme, each proxy signer broadcasts R_i , X_i , and V_i during MulProxySign, and the clerk transmits the aggregated values required for final verification. Although the proposed scheme includes partial-signature verification and a formal security proof, its communication cost remains within the same practical range as those of the compared constructions.

We estimate computation-related energy consumption using $E = VIt$ [62, 63], where the supply voltage is fixed at $V = 3.3$ V, the current draw is assumed to be $I = 150$ mA, and t is the execution time measured on the ARMv6 platform. The results show that the proposed schemes require less estimated energy than most of the compared schemes, except for the MulProxySign phase of Chen *et al.* [24]. The on/offline construction yields the lowest estimated signing energy among the evaluated schemes. Public-key sizes are also reported to indicate the storage required when a drone swarm maintains the keys of multiple participants.



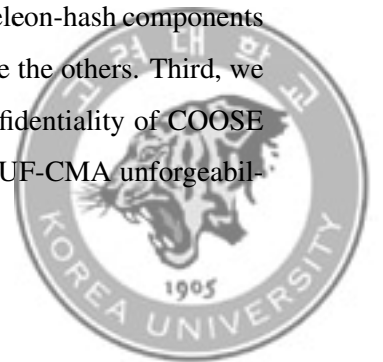
Because IBMPS execution requires cooperation among several drones, interaction delay [64] is also relevant to IoD deployment. The proposed architecture allows the proxy drones to cooperate directly instead of transmitting every signing request to a distant ground control station. This design can reduce dependence on long-distance communication and is particularly useful in time-sensitive missions such as surveillance and rescue. Once the delegation and offline values have been prepared, the authorized drones can jointly generate and authenticate a message on behalf of the original signer.



Chapter 4. On/Offline Identity-Based Signcryption

This chapter investigates identity-based on/offline signcryption (OOSE) schemes for mobile and multi-device environments in which offline computations may be performed repeatedly and their outputs may remain in device memory until the online phase. Existing OOSE constructions primarily reduce online computation by moving expensive operations to the offline phase. Their security analyses, however, generally presume that the intermediate offline state remains protected until it is consumed. This presumption may not hold in deployment settings such as Internet of Drones (IoD) and mobile edge systems, where device compromise or memory corruption may expose values generated between the two phases.

This chapter addresses this problem in four directions. First, we extend the OOSE security model to capture memory corruption between the offline and online phases when offline computations may be executed repeatedly. The model introduces MemCorrupt queries to represent the exposure of offline outputs retained in device memory. Second, we construct two identity-based OOSE schemes: authenticated OOSE (AOOSE) and confidentiality-enhanced OOSE (COOSE). AOOSE authenticates the offline state before it is used online, whereas COOSE additionally protects the confidentiality of the stored offline state. Both constructions separate the identity-based key-agreement, identity-based signature, symmetric-encryption, and chameleon-hash components so that exposure affecting one function does not automatically compromise the others. Third, we prove the IND-CCA2 confidentiality of AOOSE and the offline-state confidentiality of COOSE under the GBDH assumption in the random oracle model. We establish EUF-CMA unforgeabil-



ity from the existential unforgeability of the underlying identity-based signature scheme and the collision resistance of the chameleon hash function. Finally, we evaluate the proposed schemes through implementation and discrete-event network simulation in an IoD environment.

The remainder of this chapter is organized as follows. We first present the syntax and security model of OOSE schemes, including the multi-instance and memory-corruption settings. We then examine the structural assumptions of closely related OOSE constructions. Next, we present our proposed schemes, AOOSE and COOSE. We subsequently provide the security analyses. Finally, we evaluate the computational cost and implementation performance.

4.1 Definitions

4.1.1 Syntax

In a generic on/offline identity-based signcryption (OOSE) scheme, a user may execute the offline algorithm repeatedly before invoking the online algorithm. In the security experiment below, each query is interpreted with respect to the offline state currently stored for the queried sender. Let $\{U_k\}_{k \in N}$ be a set of users indexed by $N = \{1, \dots, n\}$. Any user may act as a sender or a receiver; in a particular execution, U_i denotes the sender and U_j denotes the receiver, where $i \neq j$.

Definition 12 (On/Offline Identity-Based Signcryption, OOSE). *A generic OOSE is defined as a tuple $\text{OOSE} = (\text{Setup}, \text{KGen}, \text{OffSigncrypt}, \text{OnSigncrypt}, \text{UnSigncrypt})$, consisting of the following polynomial-time algorithms:*

- $\text{Setup}(\lambda)$: On input the security parameter λ , the key generation center (KGC) runs this probabilistic polynomial-time algorithm to generate the public parameters $\mathbf{pp}$ and the master secret values used by the underlying identity-based primitives. The public parameters describe the elliptic curve, cryptographic hash functions, identity-based key-agreement scheme $\text{IBK} = (\text{Setup}, \text{Extract}, \text{Exchange})$, identity-based signature scheme $\text{IBS} = (\text{Setup}, \text{Extract}, \text{Sign}, \text{Verify})$, symmetric-key encryption scheme $\text{SE} = (\text{KGen}, \text{Enc}, \text{Dec})$, and chameleon hash function $\text{CH} = (\text{Setup}, \text{Hash}, \text{Col})$ under discrete logarithm assumption



upon the sender's request. For IBK, it generates a master secret $\bar{s}$ and the corresponding master public key $\bar{P}_{pub}$. For IBS, it generates a master secret s and the corresponding master public key P_{pub} . The KGC publishes **pp** and securely retains the master secret values s and $\bar{s}$.

- **KGen(pp, ID_k)**: Given **pp** and an identity ID_k where $k \in N$, the KGC runs this probabilistic polynomial-time algorithm to generate a public key **pk_k** and a corresponding private key **sk_k**. The key material for IBK and IBS is generated independently. For IBK, it runs **IBK.Extract(·)** to generate a public key $\bar{Q}_{ID_k}$ and a private key $\bar{S}_{ID_k}$. For IBS, the KGC runs **IBS.Extract(·)** to obtain a public key Q_{ID_k} and a private key S_{ID_k} . When U_k acts as a sender U_i , it additionally runs **CH.Setup(λ)** to generate a public hash key HK_i and a trapdoor key CK_i . Thus,

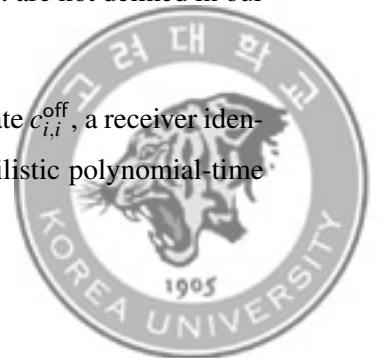
$$\mathbf{pk}_i = (Q_{ID_i}, \bar{Q}_{ID_i}, HK_i), \quad \mathbf{sk}_i = (S_{ID_i}, \bar{S}_{ID_i}, CK_i),$$

whereas a receiver U_j uses

$$\mathbf{pk}_j = (Q_{ID_j}, \bar{Q}_{ID_j}), \quad \mathbf{sk}_j = (S_{ID_j}, \bar{S}_{ID_j}).$$

For the sender, the public key is $\mathbf{pk}_i = (Q_{ID_i}, \bar{Q}_{ID_i}, HK_i)$ and the corresponding private key is $\mathbf{sk}_i = (S_{ID_i}, \bar{S}_{ID_i}, CK_i)$. For a receiver U_j , the public key is defined as $\mathbf{pk}_j = (Q_{ID_j}, \bar{Q}_{ID_j})$ and the corresponding private key is $\mathbf{sk}_j = (S_{ID_j}, \bar{S}_{ID_j})$. In OOSE, we novelly emphasize the separation of the two identity-based primitives to prevent compromise of one primitive from directly leading to the compromise of another.

- **OffSigncrypt(pp, ID_i, pk_i, sk_i)**: During the offline phase, sender U_i where $i \in N$ runs this probabilistic polynomial-time algorithm without receiving the actual message or the receiver identity. It outputs an offline state $c_{i,i}^{\text{off}}$ and stores the state in the local memory M_i for later use. We stress that the receiver's identity ID_j and the message m are not defined in our OOSE algorithm.
- **OnSigncrypt(pp, ID_i, pk_i, sk_i, c_{i,i}^{off}, ID_j, m)**: Given a stored offline state $c_{i,i}^{\text{off}}$, a receiver identity ID_j where $j \in N$, and a message m , sender U_i runs this probabilistic polynomial-time



algorithm during the online phase. It outputs a final ciphertext $c_{i,j}^{\text{on}}$ for receiver U_j .

- $\text{UnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, ID_j, c_{i,j}^{\text{on}}, \mathbf{sk}_j)$: Given $\mathbf{pp}$, ID_i , $\mathbf{pk}_i$, ID_j , $c_{i,j}^{\text{on}}$, and a receiver's private key $\mathbf{sk}_j$, a receiver U_j runs this deterministic polynomial-time algorithm to derive the decryption key, recover the plaintext, and verify the signature σ on m . It outputs the recovered message m if decryption and verification succeed and outputs $\perp$ otherwise.

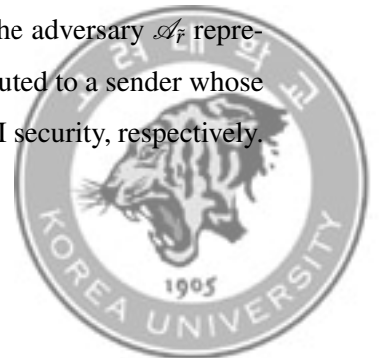
Correctness. An OOSE scheme is correct if, for every pair of distinct users U_i and U_j and every message m ,

$$\Pr \left[\begin{array}{l} \text{UnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, ID_j, c_{i,j}^{\text{on}}, \mathbf{sk}_j) = m : \\ \mathbf{pp} \leftarrow \text{Setup}(\lambda), \\ (\mathbf{pk}_i, \mathbf{sk}_i) \leftarrow \text{KGen}(\mathbf{pp}, ID_i), \\ (\mathbf{pk}_j, \mathbf{sk}_j) \leftarrow \text{KGen}(\mathbf{pp}, ID_j), \\ c_{i,i}^{\text{off}} \leftarrow \text{OffSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i), \\ c_{i,j}^{\text{on}} \leftarrow \text{OnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i, c_{i,i}^{\text{off}}, ID_j, m) \end{array} \right] = 1.$$

4.1.2 Security Model

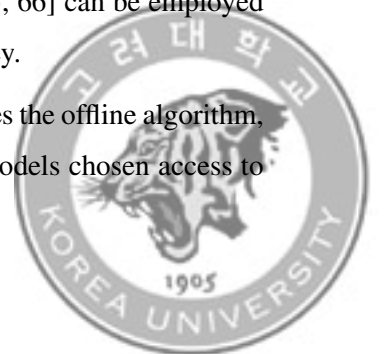
We extend conventional OOSE security notions to model exposure of the intermediate state retained between the offline and online phases. Following standard notions for on/offline signatures and signcryption [5, 29, 35, 31, 4], we consider confidentiality and unforgeability. Confidentiality is formalized as indistinguishability under adaptive chosen-ciphertext attacks (IND-CCA2), whereas unforgeability is formalized as existential unforgeability under adaptive chosen-message attacks (EUF-CMA).

We consider two insider adversaries. The adversary $\mathcal{A}_s$ represents a legitimate sender that attempts to distinguish a challenge ciphertext intended for another user. The adversary $\mathcal{A}_r$ represents a legitimate receiver that attempts to produce a valid ciphertext attributed to a sender whose signing key it does not possess. These adversaries define Type I and Type II security, respectively.



Definition 13 (Type I: Confidentiality Against $\mathcal{A}_{\bar{s}}$). *The challenger $\mathcal{C}$ interacts with $\mathcal{A}_{\bar{s}}$ in the $\text{Game}_{\bar{s}}$, as follows. The experiment below primarily formalizes the confidentiality of the final online ciphertext generated by AOOSE. For COOSE, we additionally consider the confidentiality of the stored offline state by instantiating the same experiment with an offline ciphertext as the challenge. Except for the challenge object and the corresponding memory-corruption restriction, the remaining oracle access and advantage definitions are unchanged.*

1. **Setup phase.** $\mathcal{C}$ runs $\text{Setup}(\lambda)$ to generate $\mathbf{pp}$ and forwards $\mathbf{pp}$ to $\mathcal{A}_{\bar{s}}$.
2. **Training phase 1.** $\mathcal{A}_{\bar{s}}$ adaptively issues a polynomial number of the following oracle queries.
 - H_0 queries: On input of user index k for ID_k where $k \in N$, $\mathcal{C}$ computes its hash value and returns $\bar{Q}_{ID_k}$ to $\mathcal{A}_{\bar{s}}$.
 - H_1 queries: On input of user index k for ID_k where $k \in N$, $\mathcal{C}$ computes its hash value and returns Q_{ID_k} to $\mathcal{A}_{\bar{s}}$.
 - H_2 queries: On input of the value $k_{i,j}$ for sender ID_i and receiver ID_j , $\mathcal{C}$ computes its hash value and returns $dk_{i,j}$ to $\mathcal{A}_{\bar{s}}$.
 - H_3 queries: On input of the value $k_{i,i}$ for sender ID_i , $\mathcal{C}$ computes its hash value and returns $dk_{i,i}$ to $\mathcal{A}_{\bar{s}}$. Note that this random oracle is only defined in our second scheme, COOSE.
 - IBK.Extract queries: On input ID_k for $k \in N$, $\mathcal{C}$ runs $\text{IBK.Extract}(\mathbf{pp}, ID_k)$ and returns the private key $\bar{S}_{ID_k}$ of IBK to $\mathcal{A}_{\bar{s}}$. Note that $\mathcal{A}_{\bar{s}}$ is prohibited from querying $\bar{S}_{ID_j}$ to avoid the trivial case.
 - IBS.Extract queries: On input ID_k for $k \in N$, $\mathcal{C}$ runs $\text{IBS.Extract}(\mathbf{pp}, ID_k)$ and returns the private key S_{ID_k} of IBS to $\mathcal{A}_{\bar{s}}$. Note that $\mathcal{A}_{\bar{s}}$ is not prohibited from querying any S_{ID_k} throughout the $\text{Game}_{\bar{s}}$. A forward-secure IBS scheme [65, 66] can be employed to enhance OOSE security under the exposure of the private key.
 - OffSigncrypt queries: On input a sender identity ID_i , $\mathcal{C}$ executes the offline algorithm, stores the resulting state in M_i , and returns $c_{i,i}^{\text{off}}$. This oracle models chosen access to



normal offline executions.

- MemCorrupt queries: On input ID_i , $\mathcal{C}$ returns the offline state currently stored in M_i . If no offline state has been generated for ID_i , it returns $\perp$. This oracle models exposure of stored memory rather than initiation of a new offline execution.
- OnSigncrypt queries: On inputs of $(ID_i, ID_j, m, c_{i,i}^{\text{off}})$, $\mathcal{C}$ runs $\text{OnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i, c_{i,i}^{\text{off}}, ID_j, m)$ and returns $c_{i,j}^{\text{on}}$ to $\mathcal{A}_s$.
- UnSigncrypt queries: On input $(ID_i, ID_j, c_{i,j}^{\text{on}})$, $\mathcal{C}$ runs $\text{UnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, ID_j, c_{i,j}^{\text{on}}, \mathbf{sk}_j)$ and returns the recovered message or $\perp$.

3. **Challenge.** For AOOSE, after **Training phase 1**, $\mathcal{A}_s$ submits two equal-length messages (m_0, m_1) together with a sender identity ID_i and a receiver identity ID_j . The challenger selects $b \xleftarrow{\$} \{0, 1\}$, runs the OOSE algorithms on m_b , and returns the challenge final ciphertext $c_{i,j}^{\text{on}*}$. For the additional offline-state confidentiality property of COOSE, the same experiment is instantiated with two equal-length candidate offline plaintexts (μ_0, μ_1) and a target sender identity ID_i . The challenger selects $b \xleftarrow{\$} \{0, 1\}$ and returns a challenge offline ciphertext $c_{i,i}^{\text{off}*}$ that encrypts μ_b .

4. **Training phase 2.** $\mathcal{A}_s$ continues to issue a polynomially bounded number of queries as in **Training phase 1**. However, in Phase 2, $\mathcal{A}_s$ is not allowed to query $\text{IBK.Extract}(\mathbf{pp}, ID_k)$, and not allowed to ask UnSigncrypt on the challenge ciphertext $c_{i,j}^{\text{on}*}$. For AOOSE, $\mathcal{A}_s$ may continue to issue the queries allowed in **Training phase 1**, except that it may not query $\text{IBK.Extract}(\mathbf{pp}, ID_j)$ for the challenge receiver or submit the challenge ciphertext $c_{i,j}^{\text{on}*}$ to the UnSigncrypt oracle. The adversary may continue to issue MemCorrupt queries, including queries involving the challenge sender. For COOSE, $\mathcal{A}_s$ may continue to issue the same queries, except that it may not obtain the private key required to decrypt the challenge offline state or submit the challenge offline ciphertext $c_{i,i}^{\text{off}*}$ to the MemCorrupt oracle. These restrictions prevent trivial recovery of the corresponding challenge plaintext.

5. **Output.** $\mathcal{A}_s$ outputs a bit b' . The experiment outputs 1 if $b' = b$, and 0, otherwise. The adversary's advantage is



$$\text{Adv}_{\mathcal{A}_S}^{\text{IND-CCA2}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|. \quad (4.7)$$

The scheme provides Type I security if this advantage is negligible in λ for every PPT adversary $\mathcal{A}_S$.

Definition 14 (Type II: Unforgeability Against $\mathcal{A}_T$). *The challenger $\mathcal{C}$ interacts with $\mathcal{A}_T$ in Game_T , as follows.*

1. **Setup phase.** $\mathcal{C}$ runs $\text{Setup}(\lambda)$ to generate $\mathbf{pp}$ and forwards $\mathbf{pp}$ to $\mathcal{A}_T$.
2. **Training phase.** $\mathcal{A}_T$ adaptively issues a polynomially bounded number of the same oracle queries as in Game_S .
3. **Forgery.** After the **Training phase**, $\mathcal{A}_T$ outputs a forged tuple $(ID_i^*, ID_j^*, m^*, c_{i,j}^{\text{on}*})$. We say that $\mathcal{A}_T$ wins Game_T if all of the following conditions are satisfied.
 - The forged ciphertext is valid for m^* , i.e., $\text{UnSigncrypt}(\mathbf{pp}, ID_i^*, \mathbf{pk}_i^*, ID_j^*, c_{i,j}^{\text{on}*}, \mathbf{sk}_j^*) = m^*$.
 - $\mathcal{A}_T$ has not queried $\text{IBS.Extract}(\mathbf{pp}, ID_i^*)$ to obtain $S_{ID_i^*}$.
 - $\mathcal{A}_T$ has not queried OnSigncrypt on (ID_i^*, ID_j^*, m^*) and obtained $c_{i,j}^{\text{on}*}$ as the corresponding response.
4. **Output.** The experiment outputs 1 if $\mathcal{A}_T$ satisfies all of the above conditions and outputs 0 otherwise. The adversary's advantage is defined as

$$\text{Adv}_{\mathcal{A}_T}^{\text{EUF-CMA}}(\lambda) = \Pr[\mathcal{A}_T \text{ wins } \text{Game}_T]. \quad (4.8)$$

The scheme provides Type II security if this advantage is negligible in λ for every PPT adversary $\mathcal{A}_T$.



4.2 Previous Work

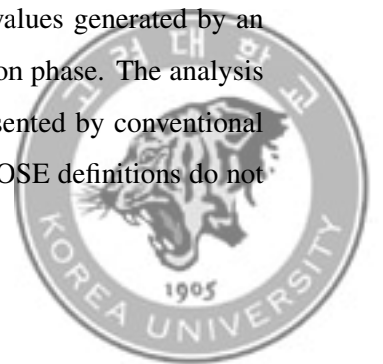
Chapter 1 reviewed the broader development of on/offline encryption and signcryption schemes. Accordingly, this section focuses on the OOSE constructions most relevant to the security problem addressed in this chapter: whether an offline state remains secure when it is exposed before the legitimate execution of the online phase.

Sun *et al.* [35] presented a generic identity-based OOSE construction based on the hash-sign-switch paradigm. Unlike their earlier construction, in which the receiver identity is determined during precomputation [34], the generic construction allows the offline phase to be executed independently of both the actual message and the receiver identity. The resulting flexibility has influenced subsequent OOSE schemes designed to reduce computation costs, shorten ciphertexts, or support resource-constrained applications [36, 27, 37, 67, 38, 39].

A common structural feature of these constructions is that the offline phase produces an intermediate state that is retained until the actual message and receiver become available. Depending on the construction, this state may include a signature on a temporary chameleon-hash input, auxiliary cryptographic values, or a precomputed key component. Storing such values substantially reduces the work performed online. However, conventional OOSE security models generally assume that the intermediate state remains under the exclusive control of the legitimate sender until it is consumed by the online algorithm.

This assumption becomes problematic when the offline and online phases are separated across time, devices, or execution environments. If an adversary obtains the stored offline state through memory corruption or device compromise, the exposed values may provide sufficient information to invoke or emulate the online computation. In particular, an adversary may attempt to reuse a valid offline signature or auxiliary key material to construct a ciphertext without possessing all of the sender's long-term secret information.

Byun [47] examined security problems that arise when intermediate values generated by an on/offline signcryption scheme are exposed outside their intended execution phase. The analysis indicates that offline-state exposure can enable attacks that are not represented by conventional signcryption, extraction, and decryption queries. Nevertheless, existing OOSE definitions do not



explicitly provide an oracle for exposing stored offline states, nor do they model such exposure independently across multiple users and executions.

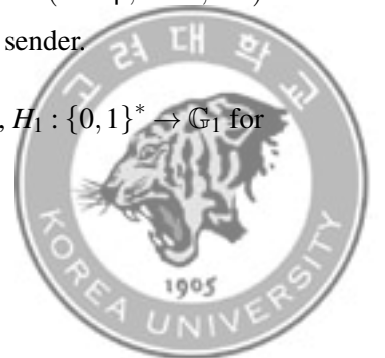
The above limitation motivates both the security model and the constructions presented in this chapter. The proposed model introduces MemCorrupt queries to represent adversarial access to offline states retained by different users between the offline and online phases. AOOSE verifies the authenticity of the offline state before using it in the online phase and binds the legitimate transition between the two phases. COOSE further protects the confidentiality of the stored offline state by encrypting it before storage. The two constructions therefore address different levels of protection against cross-phase memory exposure while preserving the computational benefits of on/offline signcryption.

4.3 Construction

4.3.1 Authenticated Identity-Based On/Offline Signcryption (AOOSE)

$\mathbf{pp} \xleftarrow{\$} \text{Setup}(\lambda)$. Given a security parameter λ , the KGC generates a set of public parameters $\mathbf{pp}$ as follows.

1. Define an elliptic curve $E(\mathbb{F}_t)$ over the finite field $\mathbb{F}_t$ where t is a prime power. Let $P \in E(\mathbb{F}_t)$ be a point of prime order q , such that $q \mid \#E(\mathbb{F}_t)$ and $\#E(\mathbb{F}_t)$ is the number of points on E over $\mathbb{F}_t$. The subgroup $\mathbb{G}_1$ is generated by P . Let $\mathbb{G}_2$ be the finite group of order q .
2. Initialize an identity-based key agreement scheme $\text{IBK} = (\text{Setup}, \text{Extract}, \text{Exchange})$, an identity-based signature scheme $\text{IBS} = (\text{Extract}, \text{Sign}, \text{Verify})$, and symmetric-key encryption scheme $\text{SE} = (\text{KGen}, \text{Enc}, \text{Dec})$.
3. Upon sender's request, choose a chameleon hash function $\text{CH} = (\text{Setup}, \text{Hash}, \text{Col})$ based on the discrete logarithm assumption and send CH to the sender.
4. Select a cryptographic hash function $H_0 : \{0, 1\}^* \rightarrow \mathbb{G}_1$ for IBK, $H_1 : \{0, 1\}^* \rightarrow \mathbb{G}_1$ for IBS, and $H_2 : \mathbb{G}_2 \rightarrow \{0, 1\}^*$ for decryption key $dk_{i,j}$.



Then, the KGC publishes $\mathbf{pp} = (E(\mathbb{F}_t), t, q, P, \mathbb{G}_1, \mathbb{G}_2, H_0, H_1, H_2, \text{IBS}, \text{IBK})$ and secretly keeps s and $\bar{s}$ to itself.

$(\mathbf{pk}_k, \mathbf{sk}_k) \xleftarrow{\$} \text{KGen}(\mathbf{pp}, ID_k)$. Given $\mathbf{pp}$ and a user identity ID_k for $k \in N$ where $N = \{1, \dots, n\}$, the KGC generates the user's independent public-private key pairs as follows.

1. Execute $\text{IBS.Extract}(\cdot)$ to obtain a public key Q_{ID_k} and a private key S_{ID_k} , such that $S_{ID_k} = sQ_{ID_k} = sH_1(ID_k)$.
2. Execute $\text{IBK.Extract}(\cdot)$ to obtain a public key $\bar{Q}_{ID_k}$ and a private key $\bar{S}_{ID_k}$, such that $\bar{S}_{ID_k} = \bar{s}\bar{Q}_{ID_k} = \bar{s}H_0(ID_k)$.
3. If U_k acts as a sender U_i where $i \in N$, U_i additionally runs $\text{CH.Setup}(\cdot)$ to obtain a hash-trapdoor key pair (HK_i, CK_i) , such that $HK_i = CK_i P = yP$.

For the sender U_i , the public key is $\mathbf{pk}_i = (Q_{ID_i}, \bar{Q}_{ID_i}, HK_i)$ and the corresponding private key is $\mathbf{sk}_i = (S_{ID_i}, \bar{S}_{ID_i}, CK_i)$. For a receiver U_j where $j \in N$, the public key is $\mathbf{pk}_j = (Q_{ID_j}, \bar{Q}_{ID_j})$ and the corresponding private key is $\mathbf{sk}_j = (S_{ID_j}, \bar{S}_{ID_j})$.

$c_{i,i}^{\text{off}} \xleftarrow{\$} \text{OffSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i)$. Given $\mathbf{pp}$, ID_i , $\mathbf{pk}_i$, and $\mathbf{sk}_i$, the sender U_i generates a signature σ on an arbitrary message pair (m', r') using the IBS private key S_{ID_i} without being given the real message to be signed. This is called the *Hash-Sign-Switch* paradigm [5]. Unlike the prior scheme [35], we emphasize that the receiver's identity is not designated and no decryption key is produced in this phase.

1. Randomly pick a pair (m', r') from a message space $\mathbb{M}$ and a random space $\mathbb{R}$, respectively. Compute the chameleon hash $h = \text{CH.Hash}(HK_i, m'; r') = m'P + r'HK_i$.
2. Compute $\sigma = \text{IBS.Sign}(S_{ID_i}, h \parallel \text{CH})$, where CH is an algorithm description of the chameleon hash family.

The output is $c_{i,i}^{\text{off}} = \{(m', r'), \sigma\}$, which is stored in the memory storage M_i of U_i .

$c_{i,j}^{\text{on}} \xleftarrow{\$} \text{OnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i, c_{i,i}^{\text{off}}, ID_j, m)$. Given $\mathbf{pp}$, ID_i , $\mathbf{pk}_i$, $\mathbf{sk}_i$, $c_{i,i}^{\text{off}}$ retrieved from M_i , a receiver's identity ID_j , and a message m , the sender U_i proceeds as follows.



1. Parse $c_{i,i}^{\text{off}}$ to obtain (m', r') and σ . Compute $h = \text{CH.Hash}(HK_i, m'; r')$, where HK_i is obtained from $\mathbf{pk}_i$.
2. Verify the signature by checking $\text{IBS.Verify}(Q_{ID_i}, \sigma, h \| \text{CH}) = 1$. This ensures the sender's authenticity from the offline phase.
3. Randomly select $x \xleftarrow{\$} \mathbb{Z}_q^*$ and compute $X_{i,j} = xP$. Compute the decryption key $dk_{i,j} = H_2(k_{i,j})$, where $k_{i,j} = e(x\bar{P}_{\text{pub}}, \bar{Q}_{ID_j})$ and $\bar{P}_{\text{pub}}$ is the master public key of IBK. Here, we designate the receiver's identity when generating $dk_{i,j}$.
4. Compute the inverse of CK_i , i.e., y^{-1} , retrieved from the secure long-term key storage. With (m', r') and the input m , compute a collision $r = y^{-1}(m' - m) + r' \pmod{q}$.
5. Encrypt $c_{i,j} = \text{SE.Enc}(dk_{i,j}, \sigma \| ID_i \| m' \| r' \| m \| r \| \text{CH})$. Note that the plaintext also includes (m', r') , along with the message pair (m, r) .

The output is $c_{i,j}^{\text{on}} = (c_{i,j}, X_{i,j})$, which is sent to the designated receiver U_j .

$m/\perp \leftarrow \text{UnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, ID_j, c_{i,j}^{\text{on}}, \mathbf{sk}_j)$. Upon receiving $c_{i,j}^{\text{on}} = (c_{i,j}, X_{i,j})$, receiver U_j derives the decryption key, recovers the plaintext, and verifies the signature as follows.

1. Compute $k_{i,j} = e(X_{i,j}, \bar{S}_{ID_j})$ and $dk_{i,j} = H_2(k_{i,j})$.
2. Decrypt

$$\sigma \| ID_i \| m' \| r' \| m \| r \| \text{CH} = \text{SE.Dec}(dk_{i,j}, c_{i,j}).$$

If decryption fails or $m' = m$, output $\perp$.

3. Compute $h = \text{CH.Hash}(HK_i, m; r) = mP + rHK_i$.
4. If $\text{IBS.Verify}(Q_{ID_i}, \sigma, h \| \text{CH}) = 1$, output m ; otherwise, output $\perp$.

Correctness. The sender and receiver derive the same key-agreement value because

$$e(x\bar{P}_{\text{pub}}, \bar{Q}_{ID_j}) = e(xP, \bar{S}_{ID_j}) = e(X_{i,j}, \bar{S}_{ID_j}). \quad (4.9)$$



Hence, both parties obtain the same symmetric key $dk_{i,j}$. Moreover, the collision value produced with CK_i satisfies

$$\text{CH.Hash}(HK_i, m'; r') = \text{CH.Hash}(HK_i, m; r),$$

so the signature generated offline verifies for the message recovered online. Correctness then follows from the correctness of IBK, CH, SE, and IBS.

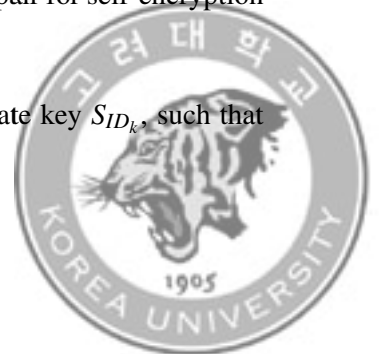
Remark. Unlike the prior scheme [35], the plaintext in OnSigncrypt is structured to include both (m', r') and (m, r) . This design enables the receiver to detect unauthorized executions of OnSigncrypt at a later phase. During decryption in UnSigncrypt, the receiver checks whether $m \neq m'$ and outputs $\perp$ if the check fails. In general, generating a valid collision value r when $m \neq m'$ requires knowledge of $CK_i \in \mathbf{sk}_i$. However, this attack remains feasible under memory corruption, where an adversary may set m to the corrupted value m' that can be obtained from M_i . In this case, the attack can be mounted without requiring any secret key material, including CK_i . To mitigate this issue and guarantee *authentication linkage*, we incorporate additional verification steps in both OnSigncrypt and UnSigncrypt. Furthermore, we observe that existing constructions, including the scheme of Sun *et al.*, may remain vulnerable to such attacks under memory corruption.

4.3.2 Confidentiality-Enhanced On/Offline Signcrypt (COOSE)

$\mathbf{pp} \xleftarrow{\$} \text{Setup}(\lambda)$. The Setup algorithm of COOSE is identical to that of AOOSE, except for the following. The KGC additionally defines $H_3 : \mathbb{G}_2 \rightarrow \{0, 1\}^{\ell_{\text{SE}}}$ for deriving the self-decryption key used in OffSigncrypt. The public parameters are thus $\mathbf{pp} = (E(\mathbb{F}_t), t, q, P, \mathbb{G}_1, \mathbb{G}_2, e, H_0, H_1, H_2, H_3, \text{IBS}, \text{IBK}, \text{SE}, \text{CH})$, and the KGC securely retains s and $\bar{s}$.

$(\mathbf{pk}_k, \mathbf{sk}_k) \xleftarrow{\$} \text{KGen}(\mathbf{pp}, ID_k)$. Given $\mathbf{pp}$ and a user identity ID_k for $k \in N$, the KGC generates the user's independent public-private key pairs in the same manner as in AOOSE. In addition, if U_k acts as a sender U_i where $i \in N$, U_i additionally generates a key pair for self-encryption as follows.

1. Execute $\text{IBS.Extract}(\cdot)$ to obtain a public key Q_{ID_k} and a private key S_{ID_k} , such that $S_{ID_k} = sQ_{ID_k} = sH_1(ID_k)$.



2. Execute $\text{IBK.Extract}(\cdot)$ to obtain a public key $\bar{Q}_{ID_k}$ and a private key $\bar{S}_{ID_k}$, such that $\bar{S}_{ID_k} = \bar{s}\bar{Q}_{ID_k} = \bar{s}H_0(ID_k)$.
3. If U_k acts as a sender U_i , U_i additionally runs $\text{CH.Setup}(\cdot)$ to obtain a hash-trapdoor key pair (HK_i, CK_i) , such that $HK_i = CK_i P = yP$.
4. U_i randomly selects $a \xleftarrow{\$} \mathbb{Z}_q^*$ and computes $P_{pub_A} = aP$. U_i secretly keeps a and includes P_{pub_A} in $\mathbf{pk}_i$.

For the sender U_i , the public key is $\mathbf{pk}_i = (Q_{ID_i}, \bar{Q}_{ID_i}, HK_i, P_{pub_A})$ and the corresponding private key is $\mathbf{sk}_i = (S_{ID_i}, \bar{S}_{ID_i}, CK_i, a)$. For a receiver U_j , the public key is $\mathbf{pk}_j = (Q_{ID_j}, \bar{Q}_{ID_j})$ and the corresponding private key is $\mathbf{sk}_j = (S_{ID_j}, \bar{S}_{ID_j})$.

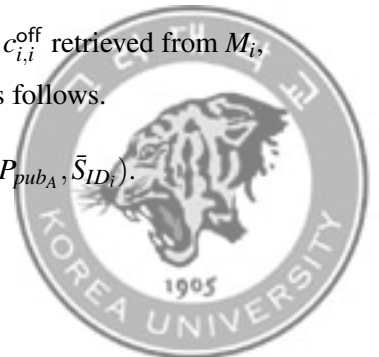
$c_{i,i}^{\text{off}} \xleftarrow{\$} \text{OffSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i)$. Given $\mathbf{pp}$, ID_i , $\mathbf{pk}_i$, and $\mathbf{sk}_i$, the sender U_i generates a signature σ on an arbitrary message pair (m', r') using S_{ID_i} and encrypts it with newly generated decryption key $dk_{i,i}$ to provide the confidentiality of $c_{i,i}^{\text{off}}$. This prevents the attacker with access to M_i under memory corruption from naturally executing the next phase.

1. Randomly pick a pair (m', r') from a message space $\mathbb{M}$ and a random space $\mathbb{R}$, respectively. Compute the chameleon hash $h = \text{CH.Hash}(HK_i, m'; r') = m'P + r'HK_i$.
2. Compute $\sigma = \text{IBS.Sign}(S_{ID_i}, h \| \text{CH})$, where CH is an algorithm description of the chameleon hash family.
3. Compute the self-decryption key $dk_{i,i} = H_3(k_{i,i})$, where $k_{i,i} = e(a\bar{P}_{pub}, \bar{Q}_{ID_i})$ and $\bar{P}_{pub}$ is the master public key of IBK.
4. Encrypt as $c_{i,i} = \text{SE.Enc}(dk_{i,i}, \sigma \| m' \| r' \| h \| \text{CH})$.

The output is $c_{i,i}^{\text{off}} = (c_{i,i}, P_{pub_A})$, where P_{pub_A} is obtained from $\mathbf{pk}_i$, which is stored in the memory storage M_i of U_i .

$c_{i,j}^{\text{on}} \xleftarrow{\$} \text{OnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, \mathbf{sk}_i, c_{i,i}^{\text{off}}, ID_j, m)$. Given $\mathbf{pp}$, ID_i , $\mathbf{pk}_i$, $\mathbf{sk}_i$, $c_{i,i}^{\text{off}}$ retrieved from M_i , a receiver's identity ID_j , and a message m , the sender U_i proceeds as follows.

1. Recover the self-decryption key $dk_{i,i} = H_3(k_{i,i})$, where $k_{i,i} = e(P_{pub_A}, \bar{S}_{ID_i})$.



2. Decrypt $\sigma \| m' \| r' \| \hat{h} \| \text{CH} = \text{SE.Dec}(dk_{i,i}, c_{i,i})$. If decryption fails, output $\perp$.
3. Recompute the chameleon hash as $h = \text{CH.Hash}(HK_i, m'; r') = m'P + r'HK_i$. If $h \neq \hat{h}$, output $\perp$.
4. Verify the offline signature. If $\text{IBS.Verify}(Q_{ID_i}, \sigma, h \| \text{CH}) \neq 1$, output $\perp$.
5. Randomly select $x \xleftarrow{\$} \mathbb{Z}_q^*$ and compute $X_{i,j} = xP$. Compute the receiver-dependent decryption key $dk_{i,j} = H_2(k_{i,j})$, where $k_{i,j} = e(x\bar{P}_{pub}, \bar{Q}_{ID_j})$.
6. Compute the inverse of $CK_i = y$ and derive the collision value $r = y^{-1}(m' - m) + r' \pmod{q}$.
7. Encrypt $c_{i,j} = \text{SE.Enc}(dk_{i,j}, \sigma \| ID_i \| m \| r \| \text{CH})$.

The output is $c_{i,j}^{\text{on}} = (c_{i,j}, X_{i,j})$, which is sent to the designated receiver U_j .

$m/\perp \leftarrow \text{UnSigncrypt}(\mathbf{pp}, ID_i, \mathbf{pk}_i, ID_j, c_{i,j}^{\text{on}}, \mathbf{sk}_j)$. Upon receiving $c_{i,j}^{\text{on}} = (c_{i,j}, X_{i,j})$, receiver U_j proceeds as follows.

1. Compute $k_{i,j} = e(X_{i,j}, \bar{S}_{ID_j})$, $dk_{i,j} = H_2(k_{i,j})$.
2. Decrypt $\sigma \| ID_i \| m \| r \| \text{CH} = \text{SE.Dec}(dk_{i,j}, c_{i,j})$. If decryption fails, output $\perp$.
3. Compute $h = \text{CH.Hash}(HK_i, m; r)$.
4. If $\text{IBS.Verify}(Q_{ID_i}, \sigma, h \| \text{CH}) = 1$, output m ; otherwise, output $\perp$.

Remark. In COOSE, the final online plaintext does not include (m', r') . Because the offline output is encrypted before storage, an adversary that merely obtains M_i cannot recover the values required to execute the online phase. Consequently, generation of a valid ciphertext $c_{i,j}^{\text{on}}$ requires the secret material contained in $\mathbf{sk}_i$.



4.4 Security Analysis

We prove the IND-CCA2 confidentiality of AOOSE and the offline-state confidentiality of COOSE under the GBDH assumption in the random oracle model. We establish Type II unforgeability from the existential unforgeability of the underlying identity-based signature scheme and the collision resistance of the chameleon hash function.

4.4.1 Authenticated Identity-Based On/Offline Signcrypt (AOOSE)

Theorem 3 (Type I: Confidentiality against $\mathcal{A}_s$ in AOOSE). *In the random oracle model, assume that there exists an IND-CCA2 adversary $\mathcal{A}_s$ against AOOSE scheme under adaptive chosen ciphertext attacks with non-negligible advantage ϵ_s . Suppose $\mathcal{A}_s$ makes at most q_{h_0} , q_{h_1} , and q_{h_2} queries to H_0 , H_1 , and H_2 , respectively, q_{off} queries to the OffSigncrypt oracle, q_{mc} queries to the MemCorrupt oracle, q_{on} queries to the OnSigncrypt oracle, and q_{us} queries to the UnSigncrypt oracle. Then, there exists a PPT challenger $\mathcal{C}$ that, by simulating Game_s , solves the GBDH problem with probability at least*

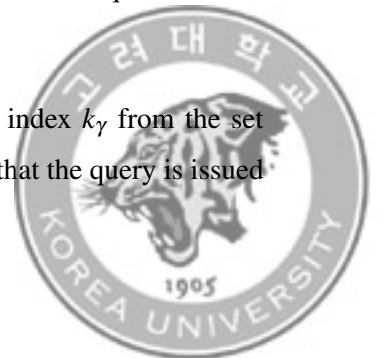
$$\text{Adv}_{\mathcal{C}}^{\text{GBDH}}(\lambda) \geq \epsilon_s \cdot \frac{1}{q_{h_0}} \cdot \left(1 - \frac{1}{q_{h_0}}\right) \cdot \frac{1}{2(q_{off} + q_{mc})} \cdot \frac{1}{q_{h_2}}.$$

Proof. Let (P, aP, bP, cP) be the GBDH instance given to $\mathcal{C}$. To solve the GBDH problem, $\mathcal{C}$'s goal is to finally compute $e(P, P)^{abc} \in \mathbb{G}_2$, where $a, b, c \in \mathbb{Z}_q^*$ and P is a generator of $\mathbb{G}_1$. The challenger uses $\mathcal{A}_s$ as a subroutine in Game_s . During the simulation, $\mathcal{C}$ initializes and maintains lists L_0 , L_1 , L_2 , and L_m to track queries. L_0 , L_1 , and L_2 correspond to oracles H_0 , H_1 , and H_2 , respectively, while L_m records the output of OffSigncrypt and MemCorrupt queries.

Setup Phase. $\mathcal{C}$ sets the master public key of IBK as $\bar{P}_{pub} = bP$. Along with system public parameters pp , $\mathcal{C}$ forwards $\bar{P}_{pub}$ to $\mathcal{A}_s$.

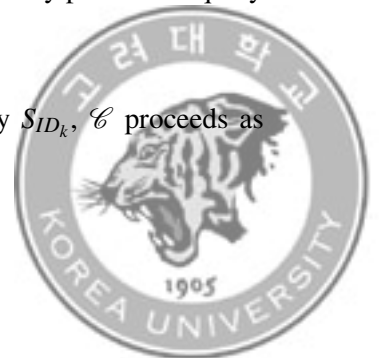
Training Phase 1. In this phase, $\mathcal{A}_s$ makes a polynomially-bounded number of queries to the defined oracles through $\mathcal{C}$.

- H_0 queries: At the start of the simulation, $\mathcal{C}$ randomly selects an index k_γ from the set $\{1, \dots, q_{h_0}\}$, which is used to guess the challenge identity. Suppose that the query is issued

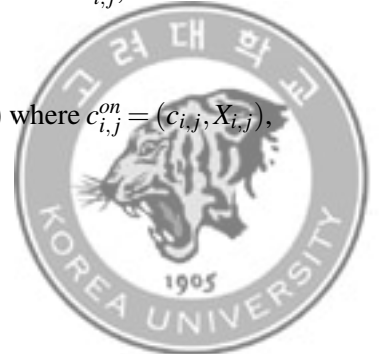


on a legal identity ID_k for $k \in \{i, j\}$.

- 1) If $k = k_\gamma$, $\mathcal{C}$ replies with the GBDH instance aP as $\bar{Q}_{ID_k}$. Observe that the corresponding private key is implicitly $\bar{S}_{ID_k} = abP$, which is unknown to $\mathcal{C}$. $\mathcal{C}$ then updates the tuple $(ID_U, \bar{Q}_{ID_U}, \perp, \perp)$ in L_0 -list. Here, U denotes a generic index of an entry stored in the list.
 - 2) Otherwise, $\mathcal{C}$ randomly chooses $\eta \xleftarrow{\$} \mathbb{Z}_q^*$ and sets $\bar{Q}_{ID_k} = \eta P$ and $\bar{S}_{ID_k} = \eta \bar{P}_{pub} = \eta bP$. Then, $\mathcal{C}$ updates $(ID_U, \bar{Q}_{ID_U}, \bar{S}_{ID_U}, \eta_U)$ in L_0 -list.
- H_1 queries: On receiving the request with k for ID_k from $\mathcal{A}_s$, $\mathcal{C}$ responds with Q_{ID_k} . Note that no restriction exists during the query.
- 1) If an entry $(ID_k, Q_{ID_k}, S_{ID_k}, \theta_k)$ exists in L_1 , return Q_{ID_k} .
 - 2) Otherwise, $\mathcal{C}$ computes $\theta \xleftarrow{\$} \mathbb{Z}_q^*$ and sets $Q_{ID_k} = \theta P$ and $S_{ID_k} = \theta P_{pub} = \theta sP$. Then, $\mathcal{C}$ updates $(ID_U, Q_{ID_U}, S_{ID_U}, \theta_U)$ to L_1 -list.
- H_2 queries: Upon receiving a query on input $k_{i,j}$ for $H_2(k_{i,j})$, $\mathcal{C}$ proceeds as follows.
- 1) If there exists a tuple $(k_{i,j}, dk_{i,j})$ in the L_2 -list, $\mathcal{C}$ returns $dk_{i,j}$.
 - 2) Otherwise, $\mathcal{C}$ randomly selects $\mu \xleftarrow{\$} \mathbb{Z}_q^*$, stores the tuple $(k_{i,j}, dk_{i,j})$ in the L_2 -list, and returns as $dk_{i,j}$.
- IBK.Extract queries: On receiving the request for the private key $\bar{S}_{ID_k}$ from $\mathcal{A}_s$, $\mathcal{C}$ proceeds as follows by looking up the corresponding entry in L_0 -List.
- 1) If the entry of ID_k exists and $ID_k = ID_{k_\gamma}$, $\mathcal{C}$ aborts the simulation.
 - 2) If the entry of ID_k exists and $ID_k \neq ID_{k_\gamma}$, $\mathcal{C}$ responds with $\bar{S}_{ID_k}$ to $\mathcal{A}_s$.
 - 3) Otherwise, $\mathcal{C}$ proceeds to access H_0 oracle to generate new IBK key pair on the query ID_k which has not been made in advance.
- IBS.Extract queries: Upon receiving the query for the signing key S_{ID_k} , $\mathcal{C}$ proceeds as follows.



- 1) If the entry of ID_k is found in L_1 -List, $\mathcal{C}$ responds with S_{ID_k} .
 - 2) Otherwise, $\mathcal{C}$ creates the tuple as defined in H_1 queries and responds with S_{ID_k} . Note that extracting signing keys is not restricted in Game_s , including $ID_k = ID_{k'}$.
- OffSigncrypt queries: Upon receiving a query with an identity ID_k , $\mathcal{C}$ proceeds as follows.
- 1) If there exists an entry $(ID_k, c_{k,k}^{off})$ in the L_m -list, $\mathcal{C}$ returns $c_{k,k}^{off}$ to $\mathcal{A}_s$.
 - 2) Otherwise, $\mathcal{C}$ checks whether a signing key S_{ID_k} is defined by looking up an entry corresponding to ID_k in the L_1 -list.
 - (a) If no such entry exists, the simulation halts, since OffSigncrypt cannot be consistently simulated without S_{ID_k} .
 - (b) Otherwise, $\mathcal{C}$ runs the $\text{OffSigncrypt}(\cdot, ID_k, \cdot)$ algorithm to generate $c_{k,k}^{off} = \{(m', r'), \sigma\}$, stores $(ID_k, c_{k,k}^{off})$ in the L_m -list, and returns $c_{k,k}^{off}$ to $\mathcal{A}_s$.
- MemCorrupt queries: Upon receiving the query on an identity ID_k , $\mathcal{C}$ proceeds as follows. Here, we do not assume that the other queries should be made in advance.
- 1) If the entry $(ID_k, c_{k,k}^{off})$ is found in the L_m -list, $\mathcal{C}$ returns $c_{k,k}^{off}$.
 - 2) Otherwise, $\mathcal{C}$ returns $\perp$ because the requested offline state has not been generated and stored.
- OnSigncrypt queries: On receiving the query of the form (ID_i, ID_j, m) for $i \neq j$, $\mathcal{C}$ first searches for the entry $(ID_i, c_{i,i}^{off})$ in L_m -List. If no such entry exists, the simulation halts. Otherwise, $\mathcal{C}$ proceeds by the following. Here, $\mathcal{C}$ can answer all OffSigncrypt queries normally without using the challenge instance components at most q_{on} times.
- 1) $\mathcal{C}$ randomly selects $\rho \xleftarrow{\$} \mathbb{Z}_q^*$ and computes $X_{i,j} = \rho P$.
 - 2) $\mathcal{C}$ computes the session key $k_{i,j} = e(\rho \bar{P}_{pub}, \bar{Q}_{ID_j})$ and queries the H_2 oracle with the input $k_{i,j}$ for $dk_{i,j}$. Then, $\mathcal{C}$ encrypts the message including m with $dk_{i,j}$, and returns $c_{i,j}^{on} = (c_{i,j}, X_{i,j})$ to $\mathcal{A}_s$.
- UnSigncrypt queries: Upon receiving the query of the form $(ID_k, c_{i,j}^{on})$ where $c_{i,j}^{on} = (c_{i,j}, X_{i,j})$, $\mathcal{C}$ proceeds as follows.

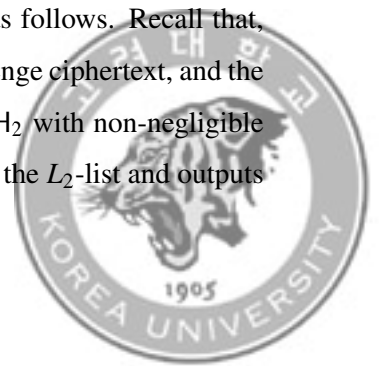


- 1) If $ID_k \neq ID_{k_\gamma}$, $\mathcal{C}$ looks up the entry $(ID_k, \bar{Q}_{ID_k}, \bar{S}_{ID_k}, \eta_k)$ in the L_0 -list. If no such entry exists, the simulation halts. Otherwise, $\mathcal{C}$ computes $k_{i,j} = e(X_{i,j}, \bar{S}_{ID_k})$. Then $\mathcal{C}$ queries H_2 oracle on input $k_{i,j}$ to obtain $dk_{i,j}$, and decrypts $c_{i,j}$ using $dk_{i,j}$. The resulting plaintext is forwarded to $\mathcal{A}_S$.
- 2) Otherwise, if $ID_k = ID_{k_\gamma}$, $\mathcal{C}$ simulates the decryption with access to the DBDH oracle. That is, $\mathcal{C}$ first searches through H_2 -List for each tuple $(k_{i,j}, dk_{i,j})$ and queries the DBDH oracle with the tuple $(P, \bar{Q}_{ID_{k_\gamma}}, \bar{P}_{pub}, X_{U,k_\gamma}, k_{U,k_\gamma})$. Here, the DBDH oracle is an integral part of the GBDH assumption, which is required to simulate UnSigncrypt for the challenge identity, as presented in 5.
 - (a) If the oracle returns 1, it implies that $k_{U,k_\gamma} = e(X_{U,k_\gamma}, \bar{S}_{ID_{k_\gamma}})$ is satisfied. Then, $\mathcal{C}$ decrypts the ciphertext with dk_{U,k_γ} .
 - (b) Otherwise, if DBDH oracle responds with 0, $\mathcal{C}$ randomly chooses (k_{U,k_γ}, ζ) that does not exist in H_2 -List. Then, $\mathcal{C}$ decrypts the ciphertext with ζ and forwards the result to $\mathcal{A}_S$.

Challenge. $\mathcal{A}_S$ outputs two chosen messages of equal length (m_0, m_1) along with the identity, ID_γ . If $ID_\gamma \neq ID_{k_\gamma}$, the simulation aborts. Otherwise, $\mathcal{C}$ proceeds as follows. $\mathcal{C}$ first samples a random challenge bit $b \xleftarrow{\$} \{0, 1\}$. It then sets $X_{U,\gamma}^* = cP$, where cP is taken from the given GBDH instance. Next, $\mathcal{C}$ chooses a random string $\tau \xleftarrow{\$} \{0, 1\}^n$ and forms the challenge ciphertext as $c_{U,\gamma}^{on} = (c_{U,\gamma}^*, X_{U,\gamma}^*) = (\tau, cP)$. Implicitly, $e(X_{U,\gamma}^*, \bar{S}_{ID_\gamma}^*) = e(cP, abP) = e(P, P)^{abc}$. The challenge ciphertext $c_{U,\gamma}^{on}$ is given to $\mathcal{A}_S$.

Training Phase 2. After receiving the challenge ciphertext $c_{U,\gamma}^{on}$, $\mathcal{A}_S$ continues to issue a polynomially bounded number of oracle queries. These queries are answered by $\mathcal{C}$ in the same manner as in **Training Phase 1**.

Output. At the end of the simulation, $\mathcal{A}_S$ outputs a bit $b' \in \{0, 1\}$ as its guess for the challenge bit b . If $b' = b$, $\mathcal{C}$ proceeds to extract a solution to the GBDH instance as follows. Recall that, during the challenge phase, the value $X_{U,\gamma}^* = cP$ is embedded into the challenge ciphertext, and the corresponding pairing value $k_{U,\gamma} = e(cP, \bar{S}_{ID_\gamma}^*) = e(P, P)^{abc}$ is queried to H_2 with non-negligible probability. $\mathcal{C}$ therefore selects the corresponding entry $(k_{U,\gamma}, dk_{U,\gamma})$ from the L_2 -list and outputs



$k_{U,\gamma}$ as its solution to the given GBDH instance (P, aP, bP, cP) . Otherwise, $\mathcal{C}$ outputs a random value in $\mathbb{G}_2$.

Now, we analyze the advantage of $\mathcal{C}$ to solve the GBDH problem. First, we define the following events that may cause the simulation to abort and halt.

- E_1 : $\mathcal{C}$ does not select ID_γ as the identity of the challenged receiver.
- E_2 : $\mathcal{C}$ receives a query on ID_{k_γ} in IBK.Extract query.
- E_3 : $\mathcal{C}$ receives the challenged ciphertext without having issued OffSigncrypt or MemCorrupt queries for ID_i by $\mathcal{A}_S$, so that no entry $(ID_i, c^{off}_{fi,i})$ exists in L_m -list at the time of the challenge.
- E_4 : $\mathcal{C}$ searches through L_2 -list during the UnSigncrypt query to ask DBDH oracle to get the GBDH problem solution. Despite the probability of simulation aborts, we say that this event proceeds if $\mathcal{A}_S$ has any advantage in Game_S .

From the analysis, we can calculate the probability that the simulation does not abort and halt as $\Pr[\neg E_1 \wedge \neg E_2 \wedge \neg E_3 \wedge \neg E_4]$. For each event, we have $\Pr[\neg E_1] = 1/q_{h_0}$, the probability that $\mathcal{C}$ correctly guesses the challenge identity as $ID_\gamma = ID_{k_\gamma}$. As $\Pr[E_2]$ is at most $1/q_{h_0}$, we have $\Pr[\neg E_2] = 1 - 1/q_{h_0}$. To confirm the validity of the challenged ciphertext, q_{off} or q_{mc} queries for m_b are required at the previous stage. Therefore, we get $\Pr[\neg E_3] = 1/2(q_{off} + q_{mc})$. Lastly, we have $\Pr[\neg E_4] = 1/q_{h_2}$ to find the solution of the GBDH problem in L_2 -list. The resulting probability loss is polynomial and does not affect the non-negligibility of the overall advantage bound.

Conditioned on these events, $\mathcal{C}$ finally outputs $k_{U,\gamma} = e(P, P)^{abc}$ whenever $\mathcal{A}_S$ wins the IND-CCA2 game, which happens with probability ϵ_S . As a result, the advantage of solving GBDH problem is $\text{Adv}_{\mathcal{C}}^{\text{GBDH}}(\lambda) \geq \epsilon_S \cdot \frac{1}{q_{h_0}} \cdot \left(1 - \frac{1}{q_{h_0}}\right) \cdot \frac{1}{2(q_{off} + q_{mc})} \cdot \frac{1}{q_{h_2}}$.

□

Theorem 4 (Type II : Unforgeability against $\mathcal{A}_T$ in AOOSE). *In the random oracle model, assume that there exists a probabilistic polynomial-time (PPT) adversary $\mathcal{A}_T$ that breaks the EUF-CMA security of the AOOSE scheme with non-negligible advantage ϵ_T , while making at most q_{h_i}*



queries to each hash oracle $H_{i=0,1,2}$, q_{off} queries to the OffSigncrypt oracle, q_{mc} queries to the MemCorrupt oracle, q_{on} queries to the OnSigncrypt oracle, and q_{us} queries to the UnSigncrypt oracle. Then, there exists a PPT challenger $\mathcal{C}$ that breaks the EUF-CMA security of the underlying identity-based signature scheme IBS or finds a collision in the chameleon hash function CH with a probability related to $\epsilon_{\tilde{r}}$.

Proof. Let $\mathcal{A}_{\tilde{r}}$ be an insider adversary that breaks EUF-CMA security of the AOOSE scheme with the output of a valid forgery $c_{i,j}^{on*}$. Then, we show how to simulate the $\text{Game}_{\tilde{r}}$ by $\mathcal{C}$ that breaks EUF-CMA security of IBS and prove the theorem by showing the contradiction.

Setup. At the beginning of the simulation, the challenger $\mathcal{C}$ interacts with the IBS challenger to receive the public parameters $\mathbf{pp}_{\text{IBS}}$. Then, $\mathcal{C}$ runs $\text{AOOSE.Setup}(\lambda)$ and gives the system public parameters $\mathbf{pp}$ to $\mathcal{A}_{\tilde{r}}$.

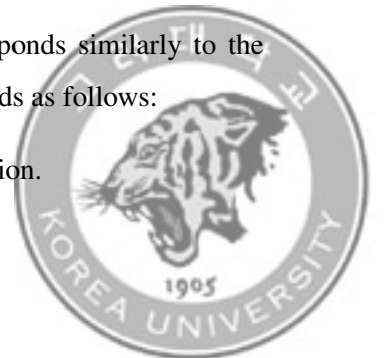
Training. $\mathcal{C}$ answers queries from $\mathcal{A}_{\tilde{r}}$ in the same manner as **Theorem 3** except the following oracles. $\mathcal{C}$ maintains lists L_0, L_1, L_2 , and L_m to store query-response pairs. At the same time, $\mathcal{C}$ can access IBS.Sign oracle during OffSigncrypt requests. Here, we do not utilize UnSigncrypt oracle.

- H_0 queries: Upon receiving a query for ID_k , $\mathcal{C}$ checks L_0 -list. If an entry exists, it returns the stored value. Otherwise, it generates $\bar{Q}_{ID_k}$, updates to the $(ID_U, \bar{Q}_{ID_U}, \bar{S}_{ID_U}, \eta_U)$ in L_0 -list, and returns the result. Note that no restriction exists during the query.
- H_1 queries: Upon the requests, $\mathcal{C}$ randomly picks an index k_γ from the set $\{1, \dots, q_{h_1}\}$, which represents the guess for the target identity index.

- 1) If $k = k_\gamma$, $\mathcal{C}$ randomly picks $\theta \xleftarrow{\$} \mathbb{Z}_q^*$ and sets $Q_{ID_k} = \theta P$. $\mathcal{C}$ then updates the tuple $(ID_U, Q_{ID_U}, \perp, \perp)$ in L_1 -list.
- 2) Otherwise, $\mathcal{C}$ randomly chooses $\theta \xleftarrow{\$} \mathbb{Z}_q^*$ and sets $Q_{ID_k} = \theta P$ and $S_{ID_k} = \theta P_{pub} = \theta sP$. Then, $\mathcal{C}$ updates to $(ID_U, Q_{ID_U}, S_{ID_U}, \theta_U)$ in L_1 -list.

- IBS.Extract queries: Upon receiving the request from $\mathcal{A}_{\tilde{r}}$, $\mathcal{C}$ responds similarly to the IBK.Extract queries in **Theorem 3**. $\mathcal{C}$ checks the L_1 -list and proceeds as follows:

- 1) If an entry for ID_k exists and $ID_k = ID_{k_\gamma}$, $\mathcal{C}$ aborts the simulation.



- 2) If an entry for ID_k exists and $ID_k \neq ID_{k_\gamma}$, $\mathcal{C}$ returns S_{ID_k} to $\mathcal{A}_F$.
 - 3) Otherwise, $\mathcal{C}$ queries the H_1 oracle on ID_k to generate a new tuple and then returns S_{ID_k} as in step 2).
- OffSigncrypt queries: On receiving the query with ID_k , $\mathcal{C}$ proceeds as follows.
 - 1) If an entry $(ID_k, c_{k,k}^{off})$ exists in the L_m -list, $\mathcal{C}$ returns it.
 - 2) Otherwise, $\mathcal{C}$ computes $h = \text{CH.Hash}(m', r')$ for a random pair (m', r') . Then, $\mathcal{C}$ generates σ by querying the external IBS.Sign oracle on $(ID_k, h || \text{CH})$ if $k = k_\gamma$, or by using the private key S_{ID_k} otherwise. Finally, $\mathcal{C}$ stores $(ID_k, c_{k,k}^{off})$ with $c_{k,k}^{off} = \{(m', r'), \sigma\}$ in the L_m -list and returns it to $\mathcal{A}_F$.
 - OnSigncrypt queries: Upon receiving the query (ID_i, ID_j, m) where $i \neq j$, $\mathcal{C}$ first searches for the entry $(ID_k, c_{k,k}^{off})$ in L_m -list. If no corresponding tuple exists, the simulation halts. Otherwise, $\mathcal{C}$ runs OnSigncrypt($\cdot$) and returns the output $c_{i,j}^{on}$ to $\mathcal{A}_F$.

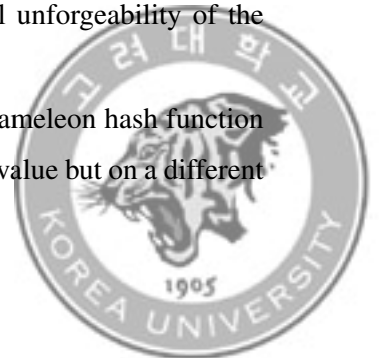
Forgery. When $\mathcal{A}_F$ terminates with the output of $(ID_i^*, ID_j^*, c_{i,j}^{on*})$ on chosen message m^* that has not been queried during OnSigncrypt, $\mathcal{C}$ runs the UnSigncrypt algorithm and parses $\sigma^* || ID_i^* || m^* || r^* || \text{CH}$ to obtain the tuple $\{\sigma^*, (m^*, r^*)\}$.

For the simulation to be successful, the identity ID_i^* chosen by $\mathcal{A}_F$ for the forgery must correspond to the index k_γ that $\mathcal{C}$ selected during the H_1 queries. If $ID_i^* \neq ID_{k_\gamma}$, $\mathcal{C}$ aborts the simulation. The probability that $\mathcal{C}$ guesses correctly is $1/q_{h_1}$.

If the guess is correct, we analyze the validity of the forged output. Let ε_F denote the non-negligible advantage of $\mathcal{A}_F$. There are two possibilities for the adversary to successfully generate a valid forgery.

The first case is where the adversary outputs a valid signature σ^* for the identity ID^* on the chameleon hash value $h^* = \text{CH.Hash}(m^*, r^*)$. Since $\mathcal{A}_F$ does not have the knowledge of the signing key S_{ID} , this event implies a successful breach of the existential unforgeability of the underlying IBS scheme.

The second case occurs when the adversary finds a collision for the chameleon hash function CH. Specifically, if the signature corresponds to a previously queried hash value but on a different



message-randomness pair, it implies that $\mathcal{A}_{\tilde{r}}$ has found a valid collision pair $(m^*, r^*) \neq (m, r)$ such that $\text{CH.Hash}(m^*, r^*) = \text{CH.Hash}(m, r)$. This constitutes a breach of the collision resistance property of CH without possessing the trapdoor key CK_i .

Therefore, the success of $\mathcal{A}_{\tilde{r}}$ directly contributes to the success of $\mathcal{C}$ in breaking either the IBS scheme or the CH scheme. Let ϵ_{IBS} and ϵ_{CH} be the advantages of breaking the EUF-CMA security of IBS and the collision resistance of CH, respectively. Considering that $\mathcal{C}$ succeeds only when it correctly guesses the target identity with probability $1/q_{h_1}$, the advantage of $\mathcal{A}_{\tilde{r}}$ is bounded by $\epsilon_{\tilde{r}} \leq q_{h_1} (\epsilon_{\text{IBS}} + \epsilon_{\text{CH}})$.

However, this leads to a contradiction. Since the underlying IBS is existentially unforgeable and CH is collision resistant against PPT adversaries $\mathcal{A}_{\tilde{r}}$, both ϵ_{IBS} and ϵ_{CH} are negligible. Consequently, this leads to the conclusion that $\epsilon_{\tilde{r}}$ is also negligible. □

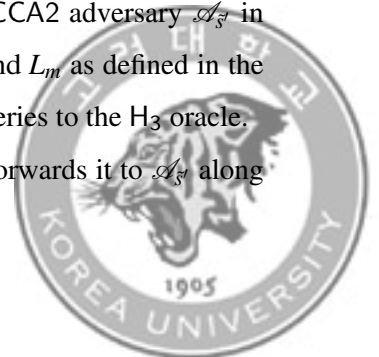
4.4.2 Confidentiality-Enhanced On/Offline Signcryption (COOSE)

Theorem 5 (Type I: Confidentiality against $\mathcal{A}_{\mathcal{S}}$ in COOSE). *In the random oracle model, assume that there exists a PPT adversary $\mathcal{A}_{\mathcal{S}}$ that breaks the confidentiality of the stored offline state generated by COOSE with non-negligible advantage $\epsilon_{\mathcal{S}}$. Suppose $\mathcal{A}_{\mathcal{S}}$ makes at most q_{h_i} queries to hash oracles $H_{i=0,1,3}$, q_{off} queries to the OffSigncrypt oracle, q_{mc} queries to the MemCorrupt oracle, q_{on} queries to the OnSigncrypt oracle, and q_{us} queries to the UnSigncrypt oracle. Then, there exists a PPT challenger $\mathcal{C}$ that, by simulating $\text{Game}_{\mathcal{S}}$, solves the GBDH problem with probability at least*

$$\text{Adv}_{\mathcal{C}}^{\text{GBDH}}(\lambda) \geq \epsilon_{\mathcal{S}} \cdot \frac{1}{q_{h_0}} \cdot \left(1 - \frac{1}{q_{h_0}}\right) \cdot \frac{1}{2(q_{\text{off}} + q_{\text{mc}})} \cdot \frac{1}{q_{h_3}}.$$

Proof. Given a GBDH instance (P, aP, bP, cP) , where the goal is to compute $e(P, P)^{abc} \in \mathbb{G}_2$, the challenger $\mathcal{C}$ attempts to solve the instance by interacting with an IND-CCA2 adversary $\mathcal{A}_{\mathcal{S}}$ in $\text{Game}_{\mathcal{S}}$. During the simulation, $\mathcal{C}$ initializes and maintains lists L_0 , L_1 , and L_m as defined in the security proof of AOOSE. In addition, $\mathcal{C}$ maintains a list L_3 to manage queries to the H_3 oracle.

Setup Phase. $\mathcal{C}$ sets the master public key of IBK as $\bar{P}_{\text{pub}} = bP$ and forwards it to $\mathcal{A}_{\mathcal{S}}$ along



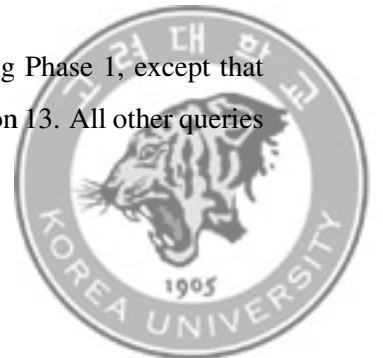
with the system parameters $\mathbf{pp}$.

Training Phase 1. Similar to the **Training Phase 1** of **Theorem 3**, $\mathcal{A}_S$ can access to the H_0 , H_1 , IBK.Extract , IBS.Extract , and MemCorrupt oracles except for the followings. Here, H_2 and UnSigncrypt oracles are not required to analyze the IND security of $c_{i,i}^{\text{off}}$.

- H_3 queries: Upon receiving a query on input $k_{i,i}$ for $H_3(k_{i,i})$, $\mathcal{C}$ proceeds as follows.
 - 1) If there exists a tuple $(k_{k,k}, dk_{k,k})$ in the L_3 -list, $\mathcal{C}$ returns $dk_{k,k}$.
 - 2) Otherwise, $\mathcal{C}$ randomly selects $\mu \xleftarrow{\$} \mathbb{Z}_q^*$, stores the tuple $(k_{k,k}, dk_{k,k})$ in the L_3 -list, and returns as $dk_{k,k}$.
- OffSigncrypt queries: On input ID_k , $\mathcal{C}$ checks if an entry (ID_k, c_k^{off}) exists in L_m . If so, it returns $c_{k,k}^{\text{off}}$. Otherwise, $\mathcal{C}$ generates a valid ciphertext as follows. $\mathcal{C}$ can answer all OffSigncrypt queries normally without using the challenge instance components.
 - 1) $\mathcal{C}$ randomly selects $\rho \xleftarrow{\$} \mathbb{Z}_q^*$ and sets the ephemeral public key $P_{\text{pub}_A} = \rho P$.
 - 2) $\mathcal{C}$ computes $k_{k,k} = e(\rho \bar{P}_{\text{pub}}, \bar{Q}_{ID_k})$ and queries the H_3 oracle to obtain $dk_{k,k}$. Then, $\mathcal{C}$ encrypts an arbitrary message m' , stores the result in L_m , and returns $c_{k,k}^{\text{off}} = (c_{k,k}, P_{\text{pub}_A})$.
- OnSigncrypt queries: Upon receiving the query as $(ID_k, c_{k,k}^{\text{off}})$, $\mathcal{C}$ proceeds as follows.
 - 1) If $ID_k \neq ID_{k_\gamma}$, $\mathcal{C}$ finds the associated information in L_0 -list and computes $k_{k,k} = e(P_{\text{pub}_A}, \bar{S}_{ID_k})$. Then, decrypt the given ciphertext with the help of H_3 oracle.
 - 2) If $ID_k = ID_{k_\gamma}$, $\mathcal{C}$ searches through L_3 -list and decrypts the final ciphertext with the help of DBDH oracle similar to the process of UnSigncrypt oracle in **Theorem 3**.

Challenge. After $\mathcal{A}_S$ outputs two chosen messages (m_0, m_1) , $\mathcal{C}$ randomly selects $b \xleftarrow{\$} \{0, 1\}$ and sets $c_{k,k}^{\text{off}*} = (c_{k,k}^*, P_{\text{pub}_A}^*) = (\tau, cP)$ where $\tau \xleftarrow{\$} \{0, 1\}^n$. Implicitly, $e(P_{\text{pub}_A}^*, \bar{S}_{ID_\gamma}^*) = e(cP, abP) = e(P, P)^{abc}$.

Training Phase 2. $\mathcal{A}_S$ continues to issue oracle queries as in Training Phase 1, except that MemCorrupt queries with ID_i are now prohibited, consistent with Definition 13. All other queries are answered by $\mathcal{C}$ in the same manner as before.



Output. The simulation ends and $\mathcal{A}_{\mathcal{S}}$ outputs a guess b' . If $\mathcal{A}_{\mathcal{S}}$ has a non-negligible advantage, it must have queried the random oracle H_3 with the correct session key $e(P, P)^{abc}$ to distinguish the message. $\mathcal{C}$ picks a random entry from the L_3 -list as the solution to the GBDH problem. The probability analysis considering the events of correct simulation and the selection from L_3 yields the advantage bound as

$$\text{Adv}_{\mathcal{C}}^{\text{GBDH}}(\lambda) \geq \epsilon_{\mathcal{S}'} \cdot \frac{1}{q_{h_0}} \cdot \left(1 - \frac{1}{q_{h_0}}\right) \cdot \frac{1}{2(q_{off} + q_{mc})} \cdot \frac{1}{q_{h_3}}.$$

□

Remark on Unforgeability against $\mathcal{A}_{\mathcal{F}}$. Since the unforgeability of the COOSE scheme essentially relies on the same underlying hard problems and security structure as the AOOSE scheme, the proof for *unforgeability against $\mathcal{A}_{\mathcal{F}}$ in COOSE* is analogous to **Theorem 4**. Therefore, we omit the detailed proof to avoid redundancy.

4.5 Performance Analysis

This section evaluates the proposed schemes from two complementary perspectives. First, we compare the symbolic computation costs of representative identity-based OOSE schemes and identify the additional operations required to protect the transition between the offline and on-line phases. Second, we assess the practical behavior of AOOSE and COOSE in an IoD-oriented mobile environment by combining cryptographic microbenchmarks with discrete-event network simulation. The implementation study uses the generic construction of Sun *et al.* [35] as the principal baseline because its offline structure is most closely related to that of the proposed schemes.

4.5.1 Efficiency Analysis

Table 4.5 compares the operations performed in the offline, online, and unsigncrypt phases. The proposed schemes deliberately move part of the protection against cross-phase exposure into the online phase. Consequently, their online costs are higher than those of schemes whose security models assume that the offline output remains inaccessible to an adversary.

The generic scheme of Sun *et al.* [35] provides the closest structural baseline. Relative to that construction, AOOSE adds $T_{\text{Ver}} + 2T_M + T_P$ to OnSigncrypt. These operations verify the authen-



Table 4.5: Efficiency Comparison of On/Offline Identity-Based Signcryption Schemes

Scheme	OffSigncrypt	Security Against Offline-State Exposure	OnSigncrypt	UnSigncrypt
Sun <i>et al.</i> [34]	$2T_M + T_P + T_E$	✗	$T_{\text{Enc}} + T_M$	$T_{\text{Dec}} + 4T_M + 2T_P$
Sun <i>et al.</i> [35]	$4T_M + T_P + T_{\text{Sign}}$	✗	$T_{\text{Coll}} + T_{\text{Enc}}$	$2T_M + T_P + T_{\text{Dec}} + T_{\text{Ver}}$
Liu <i>et al.</i> [25]	$7T_M + T_P + T_E$	✗	$3T_m$	$4T_M + 3T_P$
Selvi <i>et al.</i> [27]	$5T_M + 2T_E$	✗	$2T_m + T_{\text{ex}} + 2T_H$	$T_m + 5T_M + 3T_P + 2T_E$
Li <i>et al.</i> [37]	$4T_M + T_E$	✗	$T_{\text{ex}} + 2T_m + T_H$	$2T_M + T_P + T_{\text{ex}} + T_H$
Liu <i>et al.</i> [36]	$4T_M + T_{ME}$	✗	$3T_m + T_{\text{ex}}$	$3T_P + 5T_E + T_{ME}$
Naresh <i>et al.</i> [38]	$2T_M + T_P + T_H$	✗	$T_P + T_{\text{ex}} + T_H$	$T_M + 4T_P + T_{\text{ex}} + 3T_H$
Lai <i>et al.</i> [67]	$T_M + T_E + T_m$	✗	$T_m + T_H$	$2T_M + T_P + T_E + 2T_H$
Ali <i>et al.</i> [39]	$2T_M$	✗	$2T_H$	$4T_M$
Ours (AOOSE)	$2T_M + T_{\text{Sign}}$	✓	$T_{\text{Ver}} + 2T_M + T_P + T_{\text{Coll}} + T_{\text{Enc}}$	$T_P + T_{\text{Dec}} + 2T_M + T_{\text{Ver}}$
Ours (COOSE)	$3T_M + T_{\text{Sign}} + T_P + T_{\text{Enc}}$	✓	$2T_P + T_{\text{Dec}} + T_{\text{Ver}} + 2T_M + T_{\text{Enc}} + T_{\text{Coll}}$	$2T_M + T_P + T_{\text{Dec}} + T_{\text{Ver}}$

T_M : scalar multiplication in $\mathbb{G}_1$; T_P : one pairing evaluation $e : \mathbb{G}_1 \times \mathbb{G}_1 \rightarrow \mathbb{G}_2$; T_E : exponentiation in $\mathbb{G}_2$; T_{ex} : exclusive-OR operation; T_H : hash computation; T_m : modular computation; T_{ME} : mixed multiplication; T_{Coll} : collision finding for the chameleon hash; T_{Sign} : cost of IBS.Sign; T_{Ver} : cost of IBS.Verify; T_{Enc} : symmetric-key encryption; T_{Dec} : symmetric-key decryption.

ticity of the retrieved offline state and derive a fresh receiver-specific decryption key. COOSE further encrypts the offline state before storage. Its online phase therefore requires an additional $T_P + T_{\text{Dec}}$ beyond AOOSE to reconstruct the self-decryption key and recover the protected offline contents before performing the remaining checks.

The added operations represent the cost of security against offline-state exposure rather than an unqualified loss of efficiency. None of the compared prior schemes explicitly models memory corruption between the two phases, whereas AOOSE authenticates the stored state before online use and COOSE additionally conceals that state. Because both constructions are modular, their concrete running times can also be reduced by instantiating the underlying IBS and SE components with more efficient implementations.

4.5.2 Implementation Analysis for Mobile Applications

We evaluate the proposed schemes using a two-stage methodology. Cryptographic and retrieval operations are first measured in a native C implementation. The measured execution times

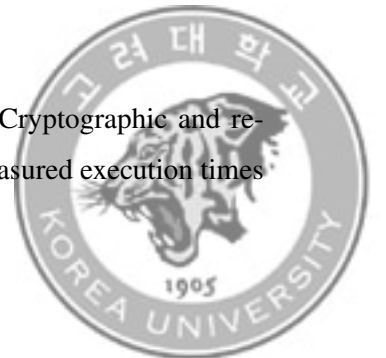


Table 4.6: Sizes of Algorithm Outputs and Encrypted Plaintexts

Value	Sun <i>et al.</i> [35]	AOOSE	COOSE
Serialized size of $c_{i,i}^{\text{off}}$	220 B	188 B	346 B
Structure of $c_{i,i}^{\text{off}}$	$(\sigma, m', r', dk_{i,j})$	(σ, m', r')	$(c_{i,i}, P_{\text{pub}_A})$
Encrypted plaintext size	222 B	262 B	210 B
Encrypted plaintext	$\sigma \ ID_i \ m \ r \ \text{CH}$	$\sigma \ ID_i \ m' \ r' \ m \ r \ \text{CH}$	$\sigma \ ID_i \ m \ r \ \text{CH}$
Serialized size of $c_{i,j}^{\text{on}}$	366 B	406 B	406 B
Structure of $c_{i,j}^{\text{on}}$	$(c_{i,j}, X_{i,j})$	$(c_{i,j}, X_{i,j})$	$(c_{i,j}, X_{i,j})$

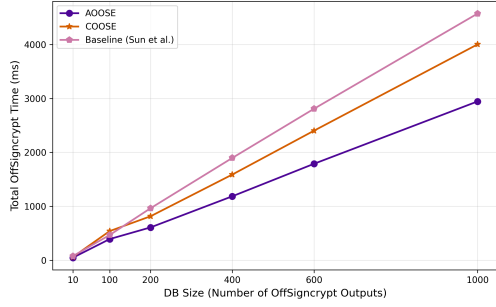
are then introduced as processing delays in a discrete-event network model. This separation is necessary because OMNeT++ models event timing and packet delivery but does not directly execute the fine-grained cryptographic and data-retrieval operations whose costs distinguish the compared schemes.

The evaluation focuses on a multi-instance setting in which a sender stores multiple offline states before flight and selects one of them when a message and receiver become available. The generic construction of Sun *et al.* [35] binds an offline output to receiver-dependent information, whereas the offline states of AOOSE and COOSE are receiver-independent. The proposed schemes can therefore select an available offline state directly, without first locating an entry associated with a particular receiver identity.

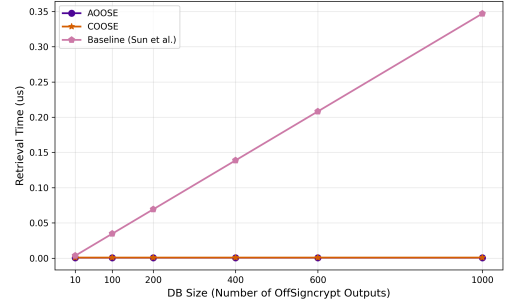
The cryptographic benchmarks are implemented in C using the Pairing-Based Cryptography (PBC) library version 1.0.0 [68] and GMP version 6.3.0 [69]. All pairing-based schemes use the same symmetric Type-I pairing over a supersingular elliptic curve. We instantiate the signature component with the Hess identity-based signature scheme [53] and the symmetric-encryption component with AES-256-GCM using a fresh 96-bit nonce and a 128-bit authentication tag. The message length is fixed at 20 bytes. To avoid assuming that a complete database management system is available on a small drone, the retrieval benchmark is implemented directly in C using four representative strategies: linear scan, stack-based pop, binary search, and hash-table lookup.

The network simulation runs on Ubuntu 24.04 under Windows Subsystem for Linux on a

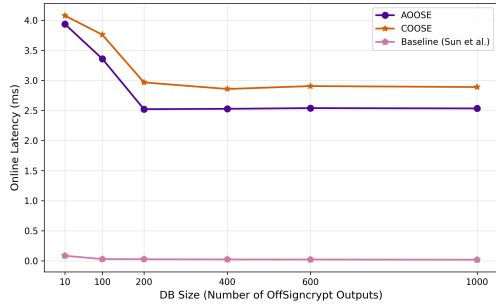




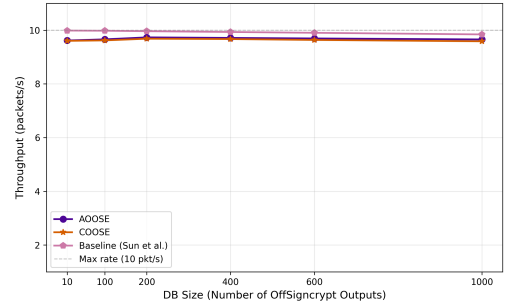
(a) Offline Phase Latency



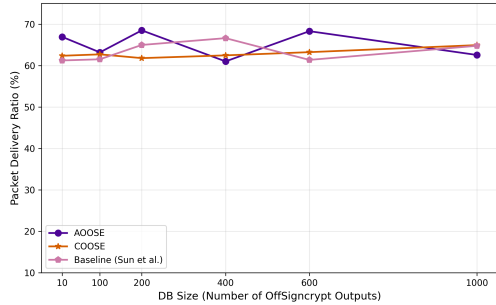
(b) Retrieval Latency in OnSigncrypt



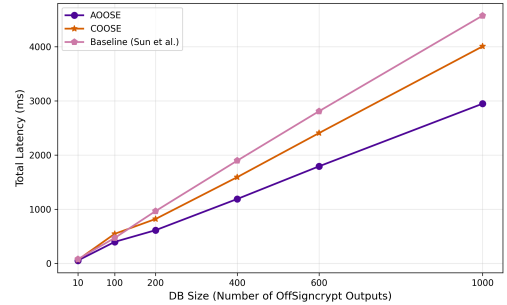
(c) Total OnSigncrypt Latency



(d) Sender Throughput



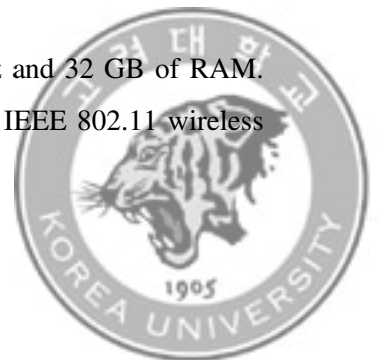
(e) Packet Delivery Ratio at the Receiver

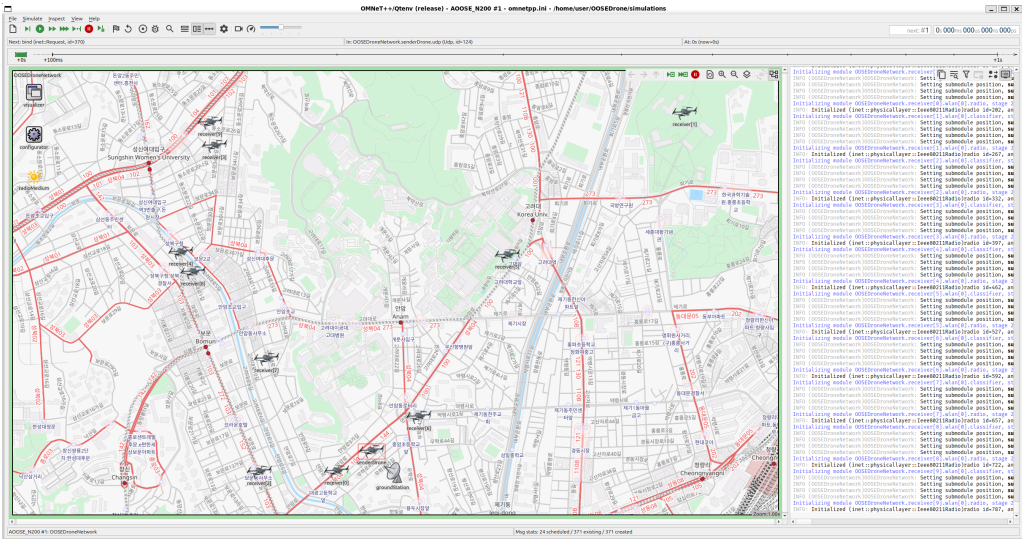


(f) End-to-End Latency

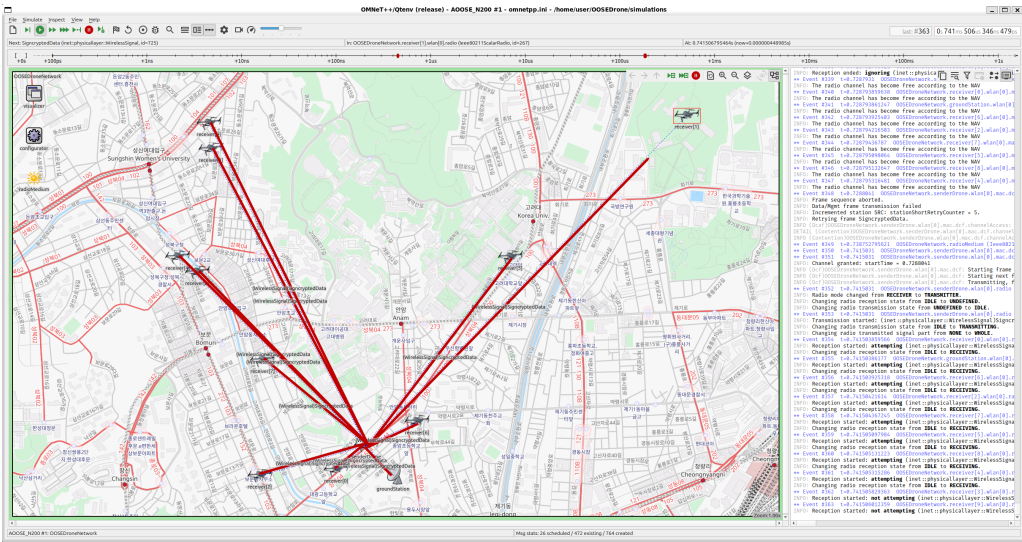
Figure 4.6: Performance Results for Different Numbers of Preloaded Offline Outputs

laptop equipped with an Intel Core Ultra 7 255H processor at 1.70 GHz and 32 GB of RAM. We use OMNeT++ 6.0.3 [70] with INET 4.5 [71]. The model employs IEEE 802.11 wireless communication and the mobility components provided by INET.





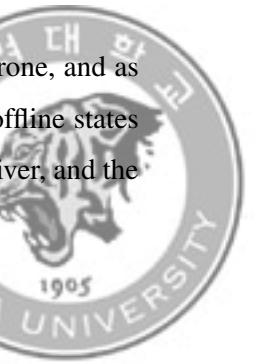
(a) Before Packet Transmission



(b) UDP Transmission to a Dynamically Selected Receiver

Figure 4.7: OMNeT++ Visualization of the Simulated IoD Network

The simulated topology contains one ground control station (GCS), one sender drone, and as many as ten receiver drones. Before the mission, the GCS generates and loads the offline states onto the sender. During flight, the sender executes OnSigncrypt after selecting a receiver, and the



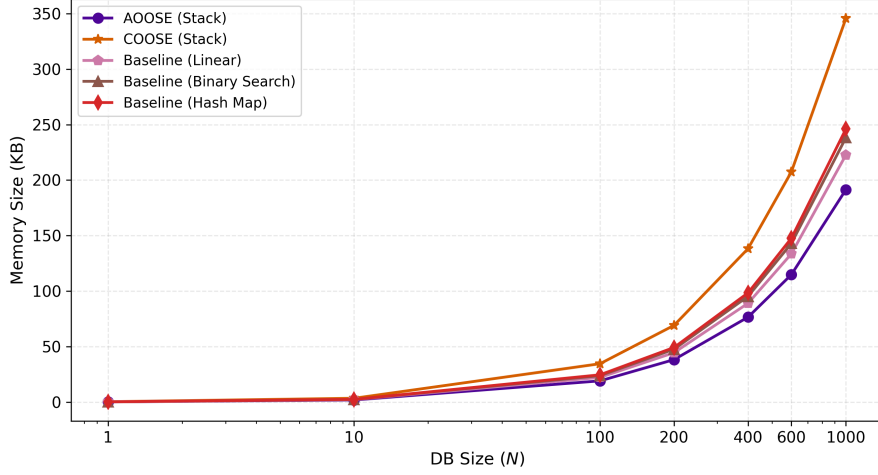


Figure 4.8: Memory Usage under Different Offline-State Retrieval Strategies

selected receiver applies UnSigncrypt to the delivered ciphertext. Receiver movement follows the MassMobility model. The implementation and simulation code are available at <https://github.com/youngkorea/OOSEDrone.git>.

Fig. 4.7 shows the network before transmission and while the sender is delivering UDP packets to a dynamically chosen receiver. The links in the visualization indicate the active transmission path. Fig. 4.6 reports the measurements obtained as the number of preloaded offline states increases from 10 to 1000. Let N_{off} denote this number, which is also referred to as the database (DB) size. The receiver count is held constant during this sweep so that the experiment isolates the effect of increasing the number of reusable offline instances.

As shown in Fig. 4.6(a), the accumulated offline latency grows more slowly for AOOSE and COOSE than for the baseline. In this experiment, offline latency is modeled as the benchmarked cost of one OffSigncrypt execution multiplied by N_{off} . The transfer of precomputed values from the GCS to the sender is excluded because the states are assumed to be loaded before flight. The lower growth rate of the proposed schemes reflects their design choice to defer receiver-specific key derivation and related protection mechanisms to the online phase.

Figs. 4.6(b) and 4.6(c) show the retrieval and total online latencies, respectively. Under a linear scan, locating a receiver-bound entry in the baseline requires $\mathcal{O}(N_{\text{off}})$ time. Binary search reduces



this cost to $\mathcal{O}(\log N_{\text{off}})$, while a hash table provides expected $\mathcal{O}(1)$ lookup at the expense of an auxiliary index and a theoretical worst case of $\mathcal{O}(N_{\text{off}})$. In contrast, the receiver-independent states of AOOSE and COOSE permit direct selection, such as a stack pop, in worst-case $\mathcal{O}(1)$ time without an identity-based index.

The memory implications of these retrieval strategies are shown in Fig. 4.8. The baseline can reduce lookup latency by introducing a sorted index or hash table, but these structures consume additional memory. The plotted memory values combine the serialized offline-state sizes in Table 4.6 with the auxiliary storage required by each retrieval method; the hash-table implementation, for example, allocates 32 bytes per entry for pointers and metadata. This trade-off is relevant to small aerial devices for which memory capacity may be as restrictive as computation time.

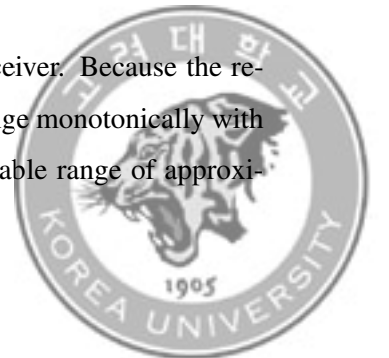
Fig. 4.6(c) combines retrieval and cryptographic processing in OnSigncrypt. Although AOOSE and COOSE perform additional verification or decryption operations, the measured increase remains limited: the largest observed difference is below 4 ms, and the gap approaches approximately 2.5 ms when $N_{\text{off}} = 1000$. These results indicate that the additional protection against offline-state exposure does not dominate online processing in the tested configuration.

Fig. 4.6(d) reports the sender throughput. For an arbitrarily selected receiver, the rate changes only marginally as the number of stored offline states increases, suggesting that the additional cryptographic and retrieval delays do not materially reduce packet generation in the simulated setting. We define the sender throughput in packets per second as

$$T_{\text{tx}} = \frac{N_{\text{sent}}}{T_{\text{sim}}}, \quad (4.10)$$

where N_{sent} is the number of transmitted packets and $T_{\text{sim}} = 300$ s is the simulation interval. This interval represents a bounded mission observation window rather than the full endurance of a drone.

Fig. 4.6(e) presents the packet delivery ratio (PDR) at the selected receiver. Because the receivers move and the wireless channel varies over time, PDR does not change monotonically with N_{off} . Across the evaluated configurations, the values remain in a comparable range of approxi-



mately 60–70%, and no systematic degradation attributable to AOOSE or COOSE is observed. PDR is calculated as

$$\text{PDR} = \frac{N_{\text{received}}}{N_{\text{sent}}} \times 100, \quad (4.11)$$

where N_{received} is the number of packets successfully delivered to the receiver.

Finally, Fig. 4.6(f) compares the end-to-end latency. In the reported simulation, AOOSE and COOSE remain below the baseline throughout the tested DB-size range. Although latency increases for all schemes as more offline states are stored, the proposed schemes exhibit a smaller rate of growth because their online retrieval does not depend on a receiver-specific search. The results therefore support the practical use of receiver-independent offline states in multi-instance mobile deployments while quantifying the additional cryptographic cost required for cross-phase protection.

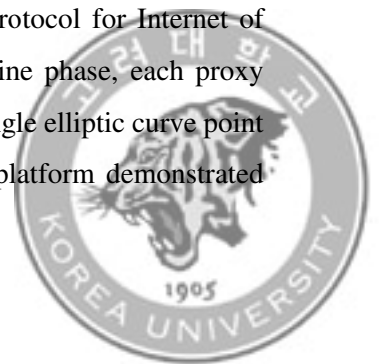


Chapter 5. Conclusion

This dissertation developed practical and provably secure identity-based on/offline cryptographic protocols for multi-user and multi-device mobile environments. In particular, it addressed two security risks that arise when cryptographic operations are distributed across users, devices, or execution phases: the exposure of auxiliary signing keys in multi-proxy signature schemes and the exposure of intermediate states retained between the offline and online phases of signcryption. By combining formal security analysis with implementation and network simulation, this dissertation showed that these risks can be mitigated while preserving the computational advantages of on/offline processing.

In Chapter 3, we proposed a provably secure identity-based multi-proxy signature (IBMPS) scheme constructed from bilinear pairings. A structural analysis of representative IBMPS schemes showed that many previous constructions are vulnerable to proxy signing key exposure: an adversary that obtains a proxy signing key together with the corresponding delegation values may recover the proxy signer’s long-term private key. The proposed scheme addresses this problem by incorporating independently selected randomness into the proxy signing key generation process, thereby preventing the exposed proxy signing key from directly revealing the corresponding long-term private key. We proved existential unforgeability against both Type I and Type II adversaries under the Computational Diffie–Hellman (CDH) assumption in the random oracle model.

We further extended the proposed IBMPS scheme to an on/offline protocol for Internet of Drones (IoD) applications. By moving expensive operations to the offline phase, each proxy signer can generate its partial multi-proxy signature online using only a single elliptic curve point addition. The implementation on a Raspberry Pi Zero W-based drone platform demonstrated



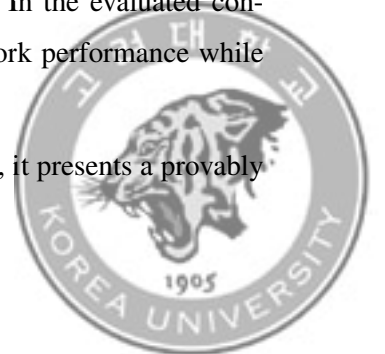
that the proposed scheme maintains a low growth rate in execution time as the number of proxy signers increases. The on/offline extension also reduced the measured computation time and energy consumption in the evaluated setting, while retaining partial-signature verification and formal security guarantees.

In Chapter 4, we introduced a security model for identity-based on/offline signcryption (OOSE) that explicitly captures memory corruption between the offline and online phases in multi-user and multi-instance settings. To the best of our knowledge, this is the first OOSE model to represent adversarial exposure of stored offline states through explicit MemCorrupt queries. Based on this model, we proposed two constructions. Authenticated OOSE (AOOSE) verifies the authenticity of an offline state before using it online and derives the receiver-dependent decryption key during the online phase. Confidentiality-Enhanced OOSE (COOSE) additionally encrypts the offline state before storage, preventing an adversary that merely obtains the stored state from recovering the values required to execute the online phase.

In the random oracle model, we established the IND-CCA2 confidentiality of the final ciphertexts generated by AOOSE and the confidentiality of the stored offline states generated by COOSE under the Gap Bilinear Diffie–Hellman (GBDH) assumption. We established unforgeability based on the existential unforgeability of the underlying identity-based signature scheme and the collision resistance of the chameleon hash function. The constructions also use separate master secrets and key material for identity-based signing and identity-based key agreement, reducing the risk that compromise of one cryptographic component directly exposes the secret material of the other.

The practical evaluation of the OOSE schemes combined native cryptographic and retrieval benchmarks with a discrete-event IoD network simulation using OMNeT++ and the INET framework. The results showed that protecting the offline state introduces only moderate additional processing during the online phase. At the same time, because the proposed offline outputs are independent of the receiver identity, AOOSE and COOSE support more efficient retrieval and reuse of precomputed states as the number of stored outputs increases. In the evaluated configurations, the schemes achieved favorable end-to-end latency and network performance while providing protection against cross-phase memory exposure.

In summary, this dissertation makes three principal contributions. First, it presents a provably



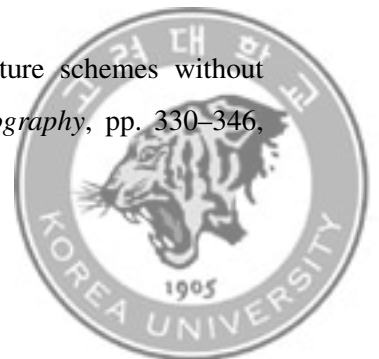
secure IBMPS scheme that resists proxy signing key exposure and an on/offline extension that requires only lightweight computation by each proxy signer during the online phase. Second, it formalizes cross-phase memory exposure in identity-based on/offline signcryption and presents two constructions that provide different levels of protection for exposed offline states. Third, it evaluates the proposed protocols through implementation on a Raspberry Pi-based drone platform and benchmark-driven simulation of a wireless IoD network, demonstrating that the proposed security mechanisms can be incorporated without eliminating the practical benefits of on/offline computation.

Several directions remain for future work. First, the security model for on/offline identity-based multi-proxy signatures (OIBMPS) should be extended beyond proxy signing key exposure to formally capture memory corruption that reveals intermediate states generated during the offline phase, thereby enabling a more comprehensive analysis of cross-phase attacks in multi-proxy and multi-device environments. Second, the proposed protocols should be extended to the post-quantum setting because the current constructions rely primarily on bilinear pairings and Diffie–Hellman-type assumptions; developing post-quantum OIBMPS and OOSE schemes based on lattice-based, hash-based, or other post-quantum primitives while preserving lightweight online computation is therefore an important direction. Third, the OOSE security model introduced in this dissertation primarily considers the full exposure of an offline state, and extending the framework to partial, continual, and side-channel leakage would provide stronger protection against a broader range of implementation-level threats by accounting for adversaries that repeatedly obtain limited information from intermediate states, long-term keys, or cryptographic computations across multiple offline and online executions.



Bibliography

- [1] A. Shamir, “Identity-based cryptosystems and signature schemes,” in *Proceedings of the Workshop on the Theory and Application of Cryptographic Techniques (CRYPTO)*, pp. 47–53, 1984.
- [2] D. Boneh and M. Franklin, “Identity-based encryption from the Weil pairing,” in *Proceedings of the Annual International Cryptology Conference (CRYPTO)*, pp. 213–229, 2001.
- [3] S. S. Al-Riyami and K. G. Paterson, “Certificateless public key cryptography,” in *Proceedings of the International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pp. 452–473, 2003.
- [4] S. Even, O. Goldreich, and S. Micali, “On-line/off-line digital signatures,” in *Proceedings of the Conference on the Theory and Application of Cryptology (CRYPTO)*, pp. 263–275, Springer, 1989.
- [5] A. Shamir and Y. Tauman, “Improved online/offline signature schemes,” in *Proceedings of the Annual International Cryptology Conference (CRYPTO)*, pp. 355–367, Springer, 2001.
- [6] H. Krawczyk and T. Rabin, “Chameleon hashing and signatures.” IACR Cryptology ePrint Archive, 1998.
- [7] K. Kurosawa and K. Schmidt-Samoa, “New online/offline signature schemes without random oracles,” in *International Workshop on Public Key Cryptography*, pp. 330–346, Springer, 2006.



- [8] C. Crutchfield, D. Molnar, D. Turner, and D. Wagner, “Generic on-line/off-line threshold signatures,” in *International Workshop on Public Key Cryptography*, pp. 58–74, Springer, 2006.
- [9] E. Bresson, D. Catalano, and R. Gennaro, “Improved on-line/off-line threshold signatures,” in *International Workshop on Public Key Cryptography*, pp. 217–232, Springer, 2007.
- [10] P. Yu and S. R. Tate, “An online/offline signature scheme based on the strong rsa assumption,” in *21st International Conference on Advanced Information Networking and Applications Workshops (AINAW’07)*, vol. 1, pp. 601–606, IEEE, 2007.
- [11] D. Catalano, M. Di Raimondo, D. Fiore, and R. Gennaro, “Off-line/on-line signatures: Theoretical aspects and experimental results,” in *International Workshop on Public Key Cryptography*, pp. 101–120, Springer, 2008.
- [12] C.-z. Gao, B. Wei, D. Xie, and C. Tang, “Divisible on-line/off-line signatures,” in *Cryptographers’ Track at the RSA Conference*, pp. 148–163, Springer, 2009.
- [13] M. H. Au, W. Susilo, and Y. Mu, “Is the notion of divisible on-line/off-line signatures stronger than on-line/off-line signatures?,” in *International Conference on Provable Security*, pp. 129–139, Springer, 2009.
- [14] M. Zheng, S.-J. Yang, W. Wu, J. Shao, and X. Huang, “A new design of online/offline signatures based on lattice,” in *International Conference on Information Security Practice and Experience*, pp. 198–212, Springer, 2018.
- [15] S. Xu, Y. Mu, and W. Susilo, “Online/offline signatures and multisignatures for aodv and dsr routing security,” in *Australasian conference on information security and privacy*, pp. 99–110, Springer, 2006.
- [16] M. Girault, G. Poupard, and J. Stern, “On the fly authentication and signature schemes based on groups of unknown order,” *Journal of Cryptology*, vol. 19, no. 4, pp. 463–487, 2006.



- [17] Y.-A. Shin, I.-R. Jeong, and J.-W. Byun, “Identity-based multiproxy signature with proxy signing key for Internet of Drones,” *IEEE Internet of Things Journal*, vol. 11, pp. 4191–4205, Feb 2024.
- [18] M. Mambo, K. Usuda, and E. Okamoto, “Proxy signatures: Delegation of the power to sign messages,” *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, vol. E79-A, no. 9, pp. 1338–1353, 1996.
- [19] S.-J. Hwang and C.-H. Shi, “A simple multi-proxy signature scheme,” *Communications of the CCISA*, vol. 8, no. 1, pp. 88–92, 2001.
- [20] X. Chen, F. Zhang, and K. Kim, “ID-based multi-proxy signature and blind multisignature from bilinear pairings,” in *Proceedings of the Korea Institute of Information Security and Cryptology (KIISC)*, vol. 3, pp. 11–19, 2003.
- [21] X. Li and K. Chen, “ID-based multi-proxy signature, proxy multi-signature and multi-proxy multi-signature schemes from bilinear pairings,” *Applied Mathematics and Computation*, vol. 169, no. 1, pp. 437–450, 2005.
- [22] F. Cao and Z. Cao, “A secure identity-based multi-proxy signature scheme,” *Computers & Electrical Engineering*, vol. 35, no. 1, pp. 86–95, 2009.
- [23] X. Gu, W. Jia, and J. Zhang, “Identity-based multi-proxy signature scheme in the standard model,” *Fundamenta Informaticae*, vol. 150, pp. 179–200, 2017.
- [24] Q. Chen, Z. Huang, Y. Ding, Y. Zhou, and H. Huang, “A new provably secure identity-based multi-proxy signature scheme,” in *Proceedings of the International Symposium on Cyberspace Safety and Security*, pp. 230–242, 2019.
- [25] J. K. Liu and J. Zhou, “An efficient identity-based online/offline encryption scheme,” in *Proceedings of the International Conference on Applied Cryptography and Network Security (ACNS)*, pp. 156–167, Springer, 2009.



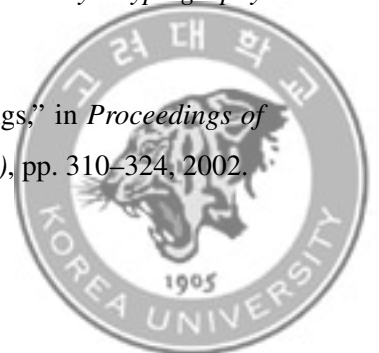
- [26] J. Lai, Y. Mu, and F. Guo, “Efficient identity-based online/offline encryption and signcryption with short ciphertext,” *International Journal of Information Security*, vol. 16, no. 3, pp. 299–311, 2017.
- [27] S. S. D. Selvi, S. S. Vivek, and C. P. Rangan, “Identity based online/offline encryption and signcryption schemes revisited,” in *Proceedings of the International Conference on Security Aspects in Information Technology*, pp. 111–127, Springer, 2011.
- [28] Y. Zheng, “Digital signcryption or how to achieve $\text{cost}(\text{signature} \ \& \ \text{encryption}) \ll \text{cost}(\text{signature}) + \text{cost}(\text{encryption})$,” in *Proceedings of the Annual International Cryptology Conference (CRYPTO)*, pp. 165–179, Springer, 1997.
- [29] J. H. An, Y. Dodis, and T. Rabin, “On the security of joint signature and encryption,” in *Proceedings of the International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT)*, pp. 83–107, Springer, 2002.
- [30] J. Malone-Lee, “Identity-based signcryption.” IACR Cryptology ePrint Archive, Report 2002/098, 2002.
- [31] B. Libert and J.-J. Quisquater, “A new identity based signcryption scheme from pairings,” in *Proceedings of the IEEE Information Theory Workshop*, pp. 155–158, 2003.
- [32] L. Chen and J. Malone-Lee, “Improved identity-based signcryption,” in *Proceedings of the Public Key Cryptography (PKC)*, pp. 362–379, 2005.
- [33] F. Zhang, Y. Mu, and W. Susilo, “Reducing security overhead for mobile networks,” in *Proceedings of the 19th International Conference on Advanced Information Networking and Applications (AINA)*, vol. 1, pp. 398–403, IEEE, 2005.
- [34] D. Sun, X. Huang, Y. Mu, and W. Susilo, “Identity-based on-line/off-line signcryption,” in *Proceedings of the IFIP International Conference on Network and Parallel Computing*, pp. 34–41, IEEE, 2008.



- [35] D. Sun, Y. Mu, and W. Susilo, "A generic construction of identity-based online/offline signcryption," in *Proceedings of the IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, pp. 707–712, IEEE, 2008.
- [36] J. K. Liu, J. Baek, and J. Zhou, "Online/offline identity-based signcryption revisited," in *Proceedings of the International Conference on Information Security and Cryptology (ICISC)*, pp. 36–51, Springer, 2010.
- [37] F. Li, M. K. Khan, K. Alghathbar, and T. Takagi, "Identity-based online/offline signcryption for low power devices," *Journal of Network and Computer Applications*, vol. 35, no. 1, pp. 340–347, 2012.
- [38] V. S. Naresh, S. Reddi, N. V. E. S. Murthy, P. S. Reddy, and A. Sanyasirao, "Practical identity based online/off-line signcryption scheme for secure communication in Internet of Things," *IEEE Access*, vol. 9, pp. 21267–21278, 2021.
- [39] I. Ali and et al., "IOOSC-U2G: An identity-based online/offline signcryption scheme for UAV to ground station communication," *IEEE Internet of Things Journal*, vol. 11, no. 18, pp. 29941–29955, 2024.
- [40] Y. Zhao, Y. Wang, Y. Liang, H. Yu, and Y. Ren, "Identity-based broadcast signcryption scheme for vehicular platoon communication," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 6, pp. 7814–7824, 2023.
- [41] Y. Hou, Y. Cao, H. Xiong, D. He, C.-H. Chi, and K.-Y. Lam, "Heterogeneous parallel key-insulated multi-receiver signcryption scheme for IoV," *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 5123–5137, 2025.
- [42] A. Mazumdar and et al., "On-line electronic payment system using signcryption," *Procedia Technology*, vol. 6, pp. 930–938, 2012.
- [43] H.-F. Yu, Z.-H. Wang, and B. Yang, "Decentralized electronic cash payment scheme using SM2-based blind signcryption," *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 48230–48241, 2025.



- [44] S. Sato and J. Shikata, “Lattice-based signcryption without random oracles,” in *Post-Quantum Cryptography*, vol. 10786 of *Lecture Notes in Computer Science*, pp. 331–351, Springer, 2018.
- [45] S. Xu and et al., “Efficient and secure post-quantum certificateless signcryption with linkability for IoMT,” *IEEE Transactions on Information Forensics and Security*, vol. 20, pp. 1119–1134, 2025.
- [46] S. Xu and et al., “Lattice-based forward secure certificateless encryption scheme for cloud data management,” in *Proceedings of the Database Systems for Advanced Applications (DASFAA)*, pp. 441–456, Springer, 2026.
- [47] J. W. Byun, “Security issues of ID-based on/offline signcryption revisited,” *The Journal of Korean Institute of Communications and Information Sciences*, vol. 50, no. 2, pp. 311–318, 2025.
- [48] S. Mishra, R. A. Sahu, S. Padhye, and R. S. Yadav, “Efficient ID-based multi-proxy signature scheme from bilinear pairing on k-plus problem,” in *Proceedings of the International Conference on Integrated Computing Technology (INTECH)*, vol. 165, pp. 113–122, 2011.
- [49] U. Rao and P. V. Reddy, “ID-based directed multi-proxy signature scheme from bilinear pairings,” *International Journal of Computer Science*, vol. 5, no. 1, pp. 107–117, 2011.
- [50] H. Xiong, J. Hu, Z. Chen, and F. Li, “On the security of an identity based multi-proxy signature scheme,” *Computers & Electrical Engineering*, vol. 37, no. 1, pp. 129–135, 2011.
- [51] A. Boldyreva, A. Palacio, and B. Warinschi, “Secure proxy signature schemes for delegation of signing rights,” *Journal of Cryptology*, vol. 25, pp. 57–115, 2012.
- [52] J. C. N. Schuldt, K. Matsuura, and K. G. Paterson, “Proxy signatures secure against proxy key exposure,” in *Proceedings of the International Workshop on Public Key Cryptography (PKC)*, pp. 141–161, 2008.
- [53] F. Hess, “Efficient identity based signature schemes based on pairings,” in *Proceedings of the International Workshop on Selected Areas in Cryptography (SAC)*, pp. 310–324, 2002.



- [54] M. Bellare, C. Namprempre, and G. Neven, “Security proofs for identity-based identification and signature schemes,” *Journal of Cryptology*, vol. 22, no. 1, pp. 1–61, 2009.
- [55] B. Libert and J.-J. Quisquater, “The exact security of an identity based signature and its applications.” IACR Cryptology ePrint Archive, Report 2004/102, 2004.
- [56] L. Chen and C. Kudla, “Identity based authenticated key agreement protocols from pairings,” in *Proceedings of the IEEE Computer Security Foundations Workshop*, pp. 219–233, 2003.
- [57] G. Xu, Y. Yang, L. Gu, and X. Niu, “ID-based multi-proxy sequential signature system from bilinear pairing,” in *2007 IEEE/WIC/ACM Int. Conf. Web Intell. and Intell. Agent Tech. Workshop*, pp. 315–318, Nov. 2007.
- [58] Y. A. Shin, J. H. Choi, I. R. Jeong, and J. W. Byun, “Towards unmanned proxy signature era: A design of unmanned on/offline multi-proxy signature and its implementation,” in *Conf. Inform. Sec. and Cryptogr.-Winter 2021 (CISC-W’21)*, vol. 31, (Republic of Korea), pp. 256–260, KIISC, 2021.
- [59] M. Bellare and P. Rogaway, “Random oracles are practical: A paradigm for designing efficient protocols,” in *Proc. 1st ACM Conf. Comput. and Commun. Sec. (CCS’93)*, (New York), pp. 62–73, Dec. 1993.
- [60] J. Bernoulli, “Quæstiones nonnullæ de usuris, cum solutione problematis de sorte aleorum, propositi in ephem. Gall. A. 168,” *Acta Eruditorum*, pp. 219–223, May 1690.
- [61] “RB071 AIR COPTER.” Daduino, <http://daduino.co.kr/index.html>.
- [62] M. O. Ozmen and A. A. Yavuz, “Dronecrypt: An efficient cryptographic framework for small aerial drones,” in *MILCOM 2018 IEEE Military Commu. Conf. (MILCOM)*, pp. 1–6, 2018.
- [63] M. O. Ozmen, R. Behnia, and A. A. Yavuz, “IoD-Crypt: A lightweight cryptographic framework for internet of drones.” ePrint Arch. arXiv:1904.06829, 2019.



- [64] S. H. Alsamhi, A. V. Shvetsov, S. Kumar, S. V. Shvetsova, M. A. Alhartomi, A. Hawbani, N. S. Rajput, S. Srivastava, A. Saif, and V. O. Nyangaresi, “UAV computing-assisted search and rescue mission framework for disaster and harsh environment mitigation,” *Drones*, vol. 6, Jun. 2021.
- [65] Y. Liu, X. Yin, and L. Qiu, “ID-based forward-secure signature scheme from the bilinear pairings,” in *Proceedings of the International Symposium on Electronic Commerce and Security*, pp. 179–183, IEEE, 2008.
- [66] J. Yu, R. Hao, F. Kong, X. Cheng, J. Fan, and Y. Chen, “Forward-secure identity-based signature: security notions and construction,” *Information Sciences*, vol. 181, no. 3, pp. 648–660, 2011.
- [67] J. Lai, Y. Mu, and F. Guo, “Efficient identity-based online/offline encryption and signcryption with short ciphertext,” *International Journal of Information Security*, vol. 16, no. 3, pp. 299–311, 2017.
- [68] B. Lynn, “The pairing-based cryptography (PBC) library.” <https://crypto.stanford.edu/pbc/>. Accessed: 2026-04-18.
- [69] T. Granlund and the GMP development team, “The GNU multiple precision arithmetic library.” <https://gmplib.org/>. Accessed: 2026-04-18.
- [70] T. Lamenza, M. Paulon, B. Perricone, B. Olivieri, and M. Endler, “Gradys-sim—a omnet++/inet simulation framework for internet of flying things,” *arXiv preprint arXiv:2202.08134*, 2022.
- [71] INET Framework development team, “INET Framework for OMNeT++.” <https://inet.omnetpp.org/>. Accessed: 2026-04-18.

